



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de autómatas y lenguajes formales — 2'2020

CONTROL 1

Indicaciones

- La duración del control es 1 hora y 30 minutos.
- Responda cada pregunta en una hoja separada y ponga su nombre en cada hoja de respuesta.
- Debe entregar una copia digital de cada pregunta por el buzón del curso, antes de las 23:59 horas del día del control.
- Debe preocuparse que la copia digital y su calidad sea legible. En caso de hacerla con papel y lápiz, se recomienda usar hojas blancas y un lápiz oscuro que sea visible en la versión digital. En caso de no ser legible, no podrá ser evaluada su solución.
- En caso de hacer el control fuera del horario, se recomienda tomar el tiempo (1 hora y 30 minutos) y entregarlo justo después de concluido el tiempo.
- Durante la evaluación puede hacer uso de sus apuntes o slides del curso.
- Esta es una evaluación estrictamente individual y, por lo tanto, no puede compartir información con sus compañeros o usar material fuera de sus apuntes o slides del curso. En caso de hacerlo, el control no reflejará su progreso en el curso, viéndose perjudicada su formación personal y profesional.
- **Al comienzo de cada pregunta debe escribir la siguiente oración y firmarla:**

“Doy mi palabra que la siguiente solución de la pregunta X fue desarrollada y escrita individualmente por mi persona según el código de honor de la Universidad.”

En caso de no escribir la oración o no firmarla, su solución no será evaluada.

Pregunta 1

Una autómata finito no-determinista (NFA) $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ se dice no-ambiguo, si para toda palabra $w \in \mathcal{L}(\mathcal{A})$ existe exactamente una ejecución de aceptación de $\mathcal{A}$ sobre w . Por ejemplo, un autómata finito determinista es un NFA no-ambiguo, pero existen autómatas que no son deterministas, pero si no-ambiguos.

1. [3 puntos] Para $i \geq 0$, considere el lenguaje L_i de todas las palabras $w = a_1 \dots a_n$ sobre $\{a, b\}$ con $n \geq i$ tal que $a_{n-i} = b$. Demuestre que para cada L_i existe un NFA no-ambiguo $\mathcal{A}$ con menos de $i + 2$ estados.
2. Demuestre que para todo lenguaje regular L con $\epsilon \notin L$, existe un NFA no-ambiguo $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ tal que $|I| = 1$, $|F| = 1$ y $L = \mathcal{L}(\mathcal{A})$.
[1 punto] por la construcción de $\mathcal{A}$ y demostrar que $\mathcal{A}$ es correcto y [2 puntos] por demostrar que $\mathcal{A}$ es no-ambiguo.

Pregunta 2

Sea Σ un alfabeto finito y sea R una expresión regular sobre Σ . Se define el operador:

$$R^{\Downarrow}$$

tal que $w \in \mathcal{L}(R^{\Downarrow})$ si, y solo si, existe una palabra $w' \in \mathcal{L}(R)$ que se puede descomponer como $w' = u_1v_1u_2v_2 \dots u_kv_k$ para algún $k \geq 1$ y con $u_i, v_i \in \Sigma^*$, y tal que $w = u_1u_2 \dots u_k$.

Demuestre que para toda expresión regular R , el resultado de $R^{\Downarrow}$ define un lenguaje regular.

[4 puntos] por la construcción de $R^{\Downarrow}$ y [2 puntos] por demostrar que es correcto.



PAUTA CONTROL 1

Pregunta 1

Pregunta 1.1

Podemos interpretar L_i como el lenguaje de las palabras tales que la $(n-i)$ -ésima letra de la palabra es una "b". Intuitivamente, podemos construir un NFA $\mathcal{A}_i = (Q, \Sigma, \Delta, I, F)$ para este lenguaje que se quede en el estado inicial leyendo cualquier letra, "adivine" la posición correcta de la b y luego avance i estados hasta el único estado final. Es claro que el autómata anterior define el lenguaje y tiene solamente $i+2$ estados, por lo que podemos formalizarlo de la siguiente manera:

- $Q = \{p_0, \dots, p_{i+1}\}$
- $\Sigma = \{a, b\}$
- $I = \{p_0\}$
- $F = \{p_{i+1}\}$
- $\Delta = \{(p_0, a, p_0), (p_0, b, p_0), (p_0, b, p_1)\} \cup \{(p_j, \alpha, p_{j+1}) \mid 0 < j < i \wedge p_j \in Q \wedge \alpha \in \Sigma\}$

Podemos ver que $L_i = \mathcal{L}(\mathcal{A}_i)$, pues si una palabra $w = a_1a_2\dots a_n$ pertenece a L_i significa que tiene una b en la posición $n - i$, lo que significa que existirá la ejecución:

$$\rho : p_0 \xrightarrow{a_1} p_0 \xrightarrow{a_2} p_0 \xrightarrow{a_3} \dots \xrightarrow{b=a_{n-i}} p_1 \xrightarrow{a_{n-(i-1)}} p_2 \xrightarrow{a_{n-(i-2)}} \dots \xrightarrow{a_{n-1}} p_i \xrightarrow{a_n} p_{i+1}$$

Similarmente, si una palabra es aceptada por $\mathcal{A}_i$, esta debe tener una ejecución de aceptación de la forma de ρ , lo que necesariamente significa que la palabra debe tener una b en la posición $n - i$ y por tanto pertenece a L_i .

Además $\mathcal{A}_i$ será no-ambiguo, pues si existiera otra ejecución para w , distinta de ρ , esto significaría que se realiza la transición a p_1 en otro momento de la ejecución. Si esta transición se realiza antes, quedarán menos de i letras por leer luego de esta, por lo que el autómata se detendrá antes de llegar al estado final y no aceptará la palabra; por otra parte, si esta transición se realiza después, quedará al menos una letra por leer luego de llegar al estado final y dado que este no tiene transiciones salientes, no se aceptará la palabra.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por realizar correctamente la construcción del autómata.
- **(0.5 puntos)** Por justificar que $L = \mathcal{L}(\mathcal{A})$.
- **(0.5 puntos)** Por justificar que $\mathcal{A}$ es no-ambiguo.

Pregunta 1.2

Podemos ver que la construcción de este autómata será la misma que se utilizó para la pregunta 2 de la tarea 1 del curso (no era necesario saberlo, este es solo un dato al margen para no escribir la misma construcción de nuevo en esta pauta), por lo que, repitiendo este proceso, tenemos que:

Dado que el lenguaje L es un lenguaje regular, sabemos que existirá un DFA $\mathcal{A}_0 = (Q, \Sigma, \delta, q_0, F)$, tal que $L = \mathcal{L}(\mathcal{A}_0)$ y a partir de este, podemos construir el DFA $\mathcal{A} = (Q', \Sigma, \Delta, \{q_0\}, \{q_f\})$, donde:

- $Q' = Q \uplus q_f$
- $\Delta = \delta \cup \{(p, a, q_f) \mid (p, a, q) \in \delta \wedge q \in F\}$

Luego, podemos demostrar que $L = \mathcal{L}(\mathcal{A})$ mostrando que $L \subseteq \mathcal{L}(\mathcal{A}) \wedge \mathcal{L}(\mathcal{A}) \subseteq L$. Los detalles de esta demostración los pueden revisar en la pauta de la Tarea 1.

Teniendo definido $\mathcal{A}$, podemos demostrar que es no ambiguo mirando sus ejecuciones y usando contradicción. Sea ρ una ejecución de aceptación de $\mathcal{A}$ sobre $w = a_1 a_2 \dots a_n$ de la forma:

$$\rho : p_0 = q_0 \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} p_{n-1} \xrightarrow{a_n} p_n = q_f$$

Puesto que $|I| = 1$ y $|F| = 1$, si suponemos que existe otra ejecución de aceptación para w , está tendrá la forma:

$$\rho' : p'_0 = q_0 \xrightarrow{a_1} p'_1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} p'_{n-1} \xrightarrow{a_n} p'_n = q_f$$

Dado que ρ y ρ' son diferentes, debe existir un primer estado p_k en el que difieren, es decir $p_k \neq p'_k$, lo que implica que:

$$(p_{k-1}, a_k, p_k) \in \Delta \wedge (p_{k-1}, a_k, p'_k) \in \Delta$$

Observando la construcción de Δ , podemos notar que esto corresponde a una contradicción, pues para que ocurra lo anterior debe cumplirse una de dos condiciones:

- $\delta(p_{k-1}, a_k) = p_k \wedge \delta(p_{k-1}, a_k) = p'_k$.

Esta condición no puede cumplirse porque δ corresponde a la función de transición de un autómata determinista, por lo que para un par estado letra existirá un único estado posible como resultado, lo que genera una contradicción.

- $p_k = p_n \wedge p'_k = p'_n$.

En caso de que la transición no provenga de δ , esta debe llegar al estado final, sin embargo, dado que tenemos un único estado final q_f y ambas ejecuciones son de aceptación, se debe cumplir que $p_n = p'_n = q_f$, lo que implica que $p_k = p'_k$, generando una contradicción.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por construir correctamente $\mathcal{A}$ y demostrar que $L = \mathcal{L}(\mathcal{A})$
 - **0.5 puntos** Por construir correctamente $\mathcal{A}$.
 - **0.5 puntos** Por demostrar que $L = \mathcal{L}(\mathcal{A})$.
- **(2 puntos)** Por demostrar que $\mathcal{A}$ es no-ambiguo.
 - **1 punto** Por plantear correctamente las ejecuciones de $\mathcal{A}$.
 - **1 punto** Por demostrar que la ejecución es única usando la construcción de Δ .

Pregunta 2

Pregunta 2

Una posible solución es la siguiente. Como $\mathcal{R}$ es una expresión regular, existe un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ tal que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{R})$. Notemos que el lenguaje $\mathcal{R}^\downarrow$ corresponde a las palabras ω tales que existe otra palabra ω' que cumple que $\omega' = u_1 v_1 \cdots u_k v_k$, y que $u_k, v_k \in \Sigma^*$ para todo $k \geq 1$, junto con que $\omega = u_1 u_2 \cdots u_k$.

En otras palabras $\mathcal{L}(\mathcal{R}^\downarrow)$ corresponde al lenguaje de las palabras que son una subpalabra (no necesariamente contigua o distinta de ϵ) de alguna palabra de $\mathcal{R}$. Ahora, teniendo claro lo anterior, una idea es partir del autómata $\mathcal{A}$ para $\mathcal{L}(\mathcal{R})$ y modificarlo de forma tal que “salte” algunos fragmentos de las palabras, para adivinar las partes que se añaden a ω para obtener palabras en $\mathcal{L}(\mathcal{R})$. De acá viene la intuición para utilizar ϵ -transiciones.

Construiremos el siguiente NFA con ϵ -transiciones, definido en base al autómata anterior. Específicamente, considere el autómata:

$$\mathcal{A}' = (Q, \Sigma, \Delta, \{q_0\}, F)$$

La relación de transición Δ será como sigue:

- Si $\delta(p, a) = q$ para $p, q \in Q, a \in \Sigma$, entonces $(p, a, q) \in \Delta$ (esto es, se “heredan” las transiciones de $\mathcal{A}$).
- Si existe una palabra $v \in \Sigma^*$ tal que al ser leída desde el estado p se llega al estado q (i.e. $\hat{\delta}(p, v) = q$), entonces podemos saltar directamente al estado q , es decir $(p, \epsilon, q) \in \Delta$.
- Notemos que como v puede ser la palabra vacía, estamos añadiendo transiciones de (p, ϵ, p) para cada estado $p \in Q$.

Ahora probaremos que el NFA con ϵ -transiciones que hemos definido acepta exactamente el lenguaje $\mathcal{L}(\mathcal{R}^\downarrow)$. Primero mostraremos que $\mathcal{L}(\mathcal{R}^\downarrow) \subseteq \mathcal{L}(\mathcal{A}')$. Sea $\omega = u_1 u_2 \cdots u_k$ tal que existe una palabra $\omega' = u_1 v_1 \cdots u_k v_k$ en $\mathcal{L}(\mathcal{R})$. Como $\omega' \in \mathcal{L}(\mathcal{R})$ entonces es aceptada por el autómata $\mathcal{A}$ y existe una ejecución ρ de $\mathcal{A}$ sobre ω' que tiene la siguiente forma.

$$\rho : q_0 \xrightarrow{u_1} p_0 \xrightarrow{v_1} q_1 \xrightarrow{u_2} p_1 \xrightarrow{v_2} q_2 \xrightarrow{u_3} p_2 \cdots \xrightarrow{v_k} q_k$$

donde q_k es un estado final.

Ahora notemos en esta ejecución que $\hat{\delta}(q_{i-1}, u_i) = p_{i-1}$ y $\hat{\delta}(p_{i-1}, v_i) = q_i$. Recordando como definimos el autómata $\mathcal{A}'$ a partir de $\mathcal{A}$ deducimos que una ejecución de $\mathcal{A}'$ sobre la palabra ω es como sigue.

$$q_0 \xrightarrow{u_1} p_0 \xrightarrow{\epsilon} q_1 \xrightarrow{u_2} p_1 \xrightarrow{\epsilon} q_2 \xrightarrow{u_3} p_2 \cdots \xrightarrow{\epsilon} q_k$$

Como el estado q_k también es final en $\mathcal{A}'$, tenemos que existe una ejecución de aceptación para ω en $\mathcal{A}'$, lo que prueba que $\mathcal{L}(\mathcal{R}^\downarrow) \subseteq \mathcal{L}(\mathcal{A}')$.

Finalmente, probaremos que $\mathcal{L}(\mathcal{A}') \subseteq \mathcal{L}(\mathcal{R}^\downarrow)$. Sea $\omega \in \mathcal{L}(\mathcal{A}')$. Existe (al menos) una ejecución de aceptación ρ de $\mathcal{A}'$ sobre la palabra ω . Este NFA admite transiciones en la palabra vacía, podemos asumir sin pérdida de generalidad, debido a la forma en que se definió la relación de transición Δ que esta ejecución no tiene dos ϵ -transiciones seguidas. Si existe un fragmento de la ejecución de forma $q_i \xrightarrow{\epsilon} p_i \xrightarrow{\epsilon} q_{i+1}$, se tiene que $\hat{\delta}(q_i, \epsilon) = q_{i+1}$ y luego por la forma en que se añadieron ϵ -transiciones a Δ , existe otra ejecución igual pero que realiza directamente $q_i \xrightarrow{\epsilon} q_{i+1}$. Ahora por lo anterior podemos considerar que ρ tiene la siguiente forma

$$q_0 \xrightarrow{u_1} p_0 \xrightarrow{\epsilon} q_1 \xrightarrow{u_2} p_1 \xrightarrow{\epsilon} q_2 \xrightarrow{u_3} p_2 \cdots \xrightarrow{\epsilon} q_k$$

Donde q_k es un estado final. Ahora viendo la relación entre las definiciones de los autómatas $\mathcal{A}$ y $\mathcal{A}'$ y mirando las palabras v_i que existen debido al segundo punto de la definición de Δ , deducimos que existe una ejecución de $\mathcal{A}$ sobre una palabra $\omega' = u_1 v_1 \cdots u_k v_k$ que tiene la siguiente forma

$$\rho : q_0 \xrightarrow{u_1} p_0 \xrightarrow{v_1} q_1 \xrightarrow{u_2} p_1 \xrightarrow{v_2} q_2 \xrightarrow{u_3} p_2 \cdots \xrightarrow{v_k} q_k$$

donde q_k es un estado final. De acá se concluye que $\omega' \in \mathcal{L}(\mathcal{R})$ y por ende $\omega \in \mathcal{L}(\mathcal{R}^{\downarrow})$.

La distribución del puntaje es la siguiente

- **(2 puntos)** Por definir un NFA $\mathcal{A}'$ a partir del autómata $\mathcal{A}$ y añadir las transiciones de $\mathcal{A}$ a la relación de transición.
- **(2 puntos)** Por añadir al autómata anterior ϵ -transiciones que permitan saltar entre estados si existe una palabra que permite ir de uno al otro
- **(1 punto)** Por probar que $\mathcal{L}(\mathcal{R}^{\downarrow}) \subseteq \mathcal{L}(\mathcal{A}')$
- **(1 punto)** Por probar que $\mathcal{L}(\mathcal{A}') \subseteq \mathcal{L}(\mathcal{R}^{\downarrow})$



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2° 2021

INTERROGACION 1

Preguntas en blanco: Preguntas entregadas en blanco se evaluarán con un 1.5.

Pregunta 1

De una expresión regular para el siguiente lenguajes sobre el alfabeto $\{a, b, c\}$.

“Todas las palabras que no contienen una subpalabra del lenguaje $a(b+c)c$.”

Explique por qué su expresión regular cumple con la especificación dada.

Pregunta 2

Sea Σ un alfabeto cualquiera. Para un lenguaje $L \subseteq \Sigma^*$ se define el lenguaje:

$$L - 2 = \{a_1 \dots a_n \in \Sigma^* \mid n \geq 1 \wedge \exists b \in \Sigma. a_1 \dots a_{n-1} \cdot b \cdot a_n \in L\}$$

Demuestre que para todo lenguaje regular L , el lenguaje $L - 2$ es regular.

Pregunta 3

Sea $\Sigma = \{a, b\}$. Para $w = a_1 \dots a_n \in \Sigma^*$ y $k \geq 1$, se define $w|_k = a_{n-(k-1)}a_{n-(k-2)} \dots a_n$ si $k \leq n$ y $w|_k = w$ si $k > n$. O sea, $w|_k$ son los últimos k símbolos, y es la palabra completa si el tamaño de w es menor que k .

Sea $\#$ un símbolo nuevo tal que $\# \notin \Sigma$. Para un lenguaje $L \subseteq \Sigma^*$, se define el lenguaje sobre el alfabeto $\{a, b, \#\}$ como:

$$\text{Window}(L) = \{a^k \# w \mid k \geq 1 \wedge w \in \Sigma^* \wedge w|_k \in L\}$$

En otras palabras, el lenguaje $\text{Window}(L)$ viene dado por palabras de la forma $a^k \# w$ donde primero viene una secuencia de k letras a , que definen el largo de una “ventana”, seguido de un símbolo separador $\#$, y terminado por una palabra $w \in \Sigma^*$ donde los últimos k símbolos forman la “ventana” que debe pertenecer al lenguaje L , esto es, $w|_k \in L$.

Demuestre que existe un lenguaje regular L tal que $\text{Window}(L)$ es no regular.



PAUTA INTERROGACIÓN 1

Pregunta 1

Podemos interpretar el lenguaje buscado como “Las palabras sobre el alfabeto $\{a, b, c\}$ que no contienen la subpalabra abc ni la subpalabra acc ”.

Una posible expresión regular que satisface lo que buscamos es:

$$(b + c)^* (a^+ \cdot (b + c) \cdot (b \cdot (b + c)^*)^?)^* a^*$$

La idea de la expresión es primero capturar las palabras que no contengan la letra a (primer paréntesis), luego, en caso de tener (al menos) una a seguida de una b o c , debe haber una letra b antes de la siguiente c para evitar que se formen los patrones (segundo paréntesis) y, finalmente, capturamos las palabras formadas solo por letras a (a^* final).

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por manejar correctamente las palabras que no contienen letras a . $((b + c)^*)$
- **(0.5 puntos)** Por manejar correctamente las palabras que contienen solamente letras a . (a^*)
- **(4.5 puntos)** Por manejar correctamente las palabras que contienen las 3 letras posibles.
 - **(0.5 puntos)** Por considerar que las palabras pueden tener varias letras a . (a^+)
 - **(2 puntos)** Por notar que el patrón se comienza a formar al leer una b o c luego de la última a . $((b + c))$
 - **(2 puntos)** Por evitar que se lea una c luego de leer ab y ac . $(b \cdot (b + c)^*)$
- **Nota:** Si la expresión no está completamente correcta, se descontará puntaje por las palabras que están en el lenguaje, pero no están definidas por la expresión y por las palabras definidas por la expresión que no estén en el lenguaje.

Otra forma, igualmente correcta, de resolver el problema era construir el autómata que definía el lenguaje y luego utilizar MNY para obtener la expresión asociada.

Idealmente, luego de completar el proceso se haría un análisis de la expresión encontrada para eliminar redundancias y luego se daría una explicación de cómo funciona la expresión encontrada.

Para este caso, la distribución de puntaje es la siguiente:

- **(6 puntos)** Si la expresión encontrada es correcta.
- **(3-5.5 puntos)** Si la expresión encontrada no produce “abc” ni “acc”, pero hay palabras en el lenguaje que no define. El puntaje dependerá de la explicación dada, la legibilidad de la expresión y de cuántos casos no estén cubiertos.
- **(0-4 puntos)** Si la expresión encontrada produce “abc” o “acc”. El puntaje dependerá de la explicación dada, la legibilidad de la expresión y de si existen palabras en el lenguaje que no están definidas por la expresión.
- **(0-2.5 puntos)** Si la expresión encontrada produce “abc” y “acc”. El puntaje dependerá de la explicación dada, la legibilidad de la expresión y de si existen palabras en el lenguaje que no están definidas por la expresión.

Pregunta 2

Para demostrar que el lenguaje es regular, basta con demostrar que existe un autómata que lo define.

Como L es regular, sabemos que existe un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ tal que $\mathcal{L}(\mathcal{A}) = L$.

A partir de este, podemos contruir el autómata $\mathcal{A}' = (Q', \Sigma, \Delta, q_0, q_f)$ con:

$$Q' = Q^1 \uplus Q^2 \uplus \{q_f\}, \text{ donde } Q^i = \{q^i \mid q \in Q\}$$

$$\begin{aligned} \Delta = & \{(p^1, a, q^1) \mid \delta(p, a) = q \wedge p^1, q^1 \in Q^1\} \\ & \cup \{(p^1, \varepsilon, q^2) \mid \exists a. \delta(p, a) = q \wedge p^1 \in Q^1 \wedge q^2 \in Q^2\} \\ & \cup \{(p^2, a, q_f) \mid \delta(p, a) \in F \wedge p^2 \in Q^2\} \end{aligned}$$

Intuitivamente, este autómata es igual al original (pero los estados finales dejan de serlo) y además tiene una copia de todos sus estados y un único estado final. Luego, el autómata intentará, de manera no determinista, adivinar que se encuentra donde debería estar la penúltima letra de la palabra y tomará la épsilon transición simulando que leyó esta letra; si tomó la transición en el momento correcto, debería quedar solamente una letra por leer que lleva a un estado final en el autómata original, por lo que simulamos este comportamiento con el último grupo de transiciones de Δ .

A continuación, debemos demostrar que $\mathcal{L}(\mathcal{A}') = L - 2$.

- $L - 2 \subseteq \mathcal{L}(\mathcal{A}')$:

Sea $w = a_1 \dots a_n \in L - 2$. Sabemos por definición que existe b tal que $a_1 \dots a_{n-1} b a_n \in L$, por lo que existirá la siguiente ejecución de aceptación de $\mathcal{A}$:

$$\rho_{\mathcal{A}} : q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} q_{n-1} \xrightarrow{b} q \xrightarrow{a_n} q_n \wedge q_n \in F$$

Sabemos por la construcción de $\mathcal{A}'$ que $(q_{n-1}^1, \varepsilon, q^2) \in \Delta$ y que $(q^2, a_n, q_f) \in \Delta$, por lo que podremos formar la siguiente ejecución en $\mathcal{A}'$:

$$\rho'_{\mathcal{A}} : q_0^1 \xrightarrow{a_1} q_1^1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} q_{n-1}^1 \xrightarrow{\varepsilon} q^2 \xrightarrow{a_n} q_f$$

Por tanto, $w \in \mathcal{L}(\mathcal{A}')$.

- $\mathcal{L}(\mathcal{A}') \subseteq L - 2$:

Similarmenle, sea $w = a_1 \dots a_n \in \mathcal{L}(\mathcal{A}')$. Por construcción sabes que una ejecución de aceptación de $\mathcal{A}'$ sobre w será de la forma:

$$\rho'_{\mathcal{A}} : q_0 \xrightarrow{a_1} q_1^1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} q_{n-1}^1 \xrightarrow{\varepsilon} q^2 \xrightarrow{a_n} q_f$$

Sabemos entonces por la construcción de $\mathcal{A}'$ que existe b tal que $\delta(q, b) = q^2$ y que $\delta(q^2, a_n) \in F$. Por lo que podemos construir la siguiente ejecución de $\mathcal{A}$:

$$\rho_{\mathcal{A}} : q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} q_{n-1} \xrightarrow{b} q \xrightarrow{a_n} q_n \wedge q_n \in F$$

Entonces $w \in L - 2$. Por tanto $\mathcal{L}(\mathcal{A}') = L - 2$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por definir correctamente los estados de $\mathcal{A}'$.
- **(3 puntos)** Por definir Δ correctamente:
 - **(1 punto)** Por simular las transiciones de $\mathcal{A}$.
 - **(1 punto)** Por simular la lectura de la penúltima letra de la palabra.
 - **(1 punto)** Por simular la lectura de la última letra de la palabra.
- **(1 punto)** Por demostrar $L - 2 \subseteq \mathcal{L}(\mathcal{A}')$.
- **(1 punto)** Por demostrar $\mathcal{L}(\mathcal{A}') \subseteq L - 2$.

Pregunta 3

El lenguaje $\text{Window}(L)$ está definido por:

$$\text{Window}(L) = \{a^k \# w \mid k \geq 1 \wedge w \in \Sigma^* \wedge w|_k \in L\}$$

Un posible lenguaje regular L que hace que $\text{Window}(L)$ no sea regular es:

$$L = \mathcal{L}(a^*)$$

Demostremos que $\text{Window}(L)$ no es un lenguaje regular. Sea un N cualquiera. Elegimos la palabra:

$$\underbrace{a^N \# b}_x \underbrace{a^N}_y \underbrace{\epsilon}_z$$

Tal que se cumpla:

$$a^N = \underbrace{a^i}_u \underbrace{a^j}_v \underbrace{a^k}_w$$

y tal que $i + j + k = N$, con $j \neq 0$. Al bombear esta palabra, tenemos lo siguiente:

$$a^N \# b a^i (a^j)^0 a^k = a^N \# b a^i a^k$$

Esta última palabra no pertenece a $\text{Window}(L)$, dado que $(b a^{i+k})|_N = b a^{i+k} \notin L$, por lo tanto según el lema de bombeo $\text{Window}(L)$ no es un lenguaje regular.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por encontrar un lenguaje L regular que cumpla con que $\text{Window}(L)$ no es regular.
- **(2 puntos)** Por encontrar una palabra perteneciente a $\text{Window}(L)$ para cada N y dividirla correctamente en x, y, z .
- **(1 punto)** Por bombear la palabra de forma que no pertenezca al lenguaje.
- **(1 punto)** Por utilizar correctamente el lema de bombeo.



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2' 2022

INTERROGACION 1

Preguntas en blanco: Preguntas o ítems entregados en blanco se evaluarán con 0.5 de 6 puntos.

Pregunta 1

Demuestre que para toda expresión regular R existe un ϵ -NFA $\mathcal{A}$ tal que $\mathcal{L}(R) = \mathcal{L}(\mathcal{A})$. Para esto, usted debe demostrar formalmente la construcción inductiva siguiendo la sintaxis de expresiones regulares. No es necesario demostrar la correctitud de cada construcción.

Pregunta 2

1. Escriba una expresión regular para el siguiente lenguaje sobre el alfabeto $\{a, b, c\}$:

$$L = \{w \in \{a, b, c\}^* \mid w \text{ no contiene tres } c \text{ seguidas}\}$$

Explique por qué su expresión regular define a L .

2. Construya un autómata finito determinista con el menor número de estados que defina a L . Demuestre, usando el algoritmo de minimización, que su autómata es mínimo.

Pregunta 3

Sea $\Sigma = \{a, b\}$. Considere el siguiente lenguaje:

$$\text{Eq} = \{w \cdot w \mid w \in \Sigma^+\}$$

Demuestre que el lenguaje $(\text{Eq})^+$ es no regular.

Pregunta 4

Sea $\Sigma = \{a, b\}$. Para dos lenguajes L_1 y L_2 , considere el operador “cortar y pegar” lenguajes:

$$L_1 \ddagger L_2 = \{u_1 \cdot v_2 \mid \exists u_2 \in \Sigma^* \exists v_1 \in \Sigma^* . u_1 \cdot u_2 \in L_1 \wedge v_1 \cdot v_2 \in L_2\}$$

En otras palabras, se “corta” un inicio (prefijo) de una palabra de L_1 y un final (sufijo) de una palabra en L_2 , y se “pega” (esto es, concatenan).

Demuestre que si L_1 y L_2 son lenguajes regulares, entonces $L_1 \ddagger L_2$ también es regular.



PAUTA INTERROGACIÓN 1

Pregunta 1

Pregunta 1

Una posible solución para esta pregunta es hacer inducción sobre el largo de la fórmula. Suponemos un estado inicial y uno final. Entonces, vemos por casos:

- $R = a$, donde el autómata que acepta a R es $A = (Q, \Sigma, \Delta, I, F)$, siendo:
 - $Q = \{1, 1\}$
 - $\Delta = \{(1, a, 2)\}$
 - $I = \{1\}$
 - $F = \{2\}$
- $R = \epsilon$, donde el autómata que acepta a R es $A = (Q, \Sigma, \Delta, I, F)$, siendo:
 - $Q = \{1\}$
 - $\Delta = \emptyset$
 - $I = \{1\}$
 - $F = \{1\}$
- $R = \emptyset$, donde el autómata que acepta a R es $A = (Q, \Sigma, \Delta, I, F)$, siendo:
 - $Q = \{1, 2\}$
 - $\Delta = \emptyset$
 - $I = \{1\}$
 - $F = \{2\}$

Sean R_1 y R_2 expresiones regulares que tienen un $\epsilon - NFA$ equivalente $A_1 = (Q_1, \Sigma, \Delta_1, \{q_0^1\}, \{q_f^1\})$ y $A_2 = (Q_2, \Sigma, \Delta_2, \{q_0^2\}, \{q_f^2\})$.

- $R = R_1 + R_2$, donde el autómata que acepta a R es $A = (Q, \Sigma, \Delta, I, F)$, siendo:
 - $Q = Q_1 \uplus Q_2 \uplus \{q_0, q_f\}$
 - $\Delta = \Delta_1 \uplus \Delta_2 \uplus \{(q_0, \epsilon, q_0^1), (q_0, \epsilon, q_0^2), (q_f^1, \epsilon, q_f), (q_f^2, \epsilon, q_f)\}$
 - $I = \{q_0\}$
 - $F = \{q_f\}$
- $R = R_1 \cdot R_2$, donde el autómata que acepta a R es $A = (Q, \Sigma, \Delta, I, F)$, siendo:

- $Q = Q_1 \uplus Q_2$
 - $\Delta = \Delta_1 \uplus \Delta_2 \uplus \{(q_f^1, \epsilon, q_0^2)\}$
 - $I = \{q_0^1\}$
 - $F = \{q_f^2\}$
- $R = R_1^*$, donde el autómata que acepta a R es $A = (Q, \Sigma, \Delta, I, F)$, siendo:
- $Q = Q_1 \uplus \{q_0\}$
 - $\Delta = \Delta_1 \uplus \{(q_0, \epsilon, q_0^1), (q_f^1, \epsilon, q_0)\}$
 - $I = \{q_0\}$
 - $F = \{q_0\}$

Dado que cualquier expresión regular se puede construir recursivamente a partir de las definiciones anteriores, podemos definir recursivamente un autómata para cada expresión usando las reglas anteriores.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** Por construir correctamente el autómata del primer caso.
- **(0.5 puntos)** Por construir correctamente el autómata del segundo caso.
- **(0.5 puntos)** Por construir correctamente el autómata del tercer caso.
- **(1.5 puntos)** Por construir correctamente el autómata del cuarto caso. Donde 0.5 corresponden a definir los conjuntos de estados iniciales y finales, 0.5 a definir correctamente el conjunto de estados y 0.5 a definir correctamente el conjunto de transiciones.
- **(1.5 puntos)** Por construir correctamente el autómata del quinto caso. Donde 0.5 corresponden a definir los conjuntos de estados iniciales y finales, 0.5 a definir correctamente el conjunto de estados y 0.5 a definir correctamente el conjunto de transiciones.
- **(1.5 puntos)** Por construir correctamente el autómata del sexto caso. Donde 0.5 corresponden a definir los conjuntos de estados iniciales y finales, 0.5 a definir correctamente el conjunto de estados y 0.5 a definir correctamente el conjunto de transiciones.

Pregunta 2

Pregunta 2.1

Para resolver la pregunta anterior, vamos a proponer una expresión regular que defina a L :

$$(c + cc)^?(a + b + (a + b)c + (a + b)cc)^*$$

En primer lugar, explicaremos la construcción del término de la derecha. La expresión regular no debe definir palabras en las que después de dos letras 'c' haya otra 'c'. Para ello, se incluye la expresión que tiene una 'a' o 'b' y después dos letras 'c' y luego puede haber una 'a' o 'b' o 'ac' o 'bc'. Pero nunca habrán tres letras 'c' seguidas.

Finalmente, se añade el término de la izquierda, porque el lenguaje puede incluir la palabra 'c' o 'cc'.

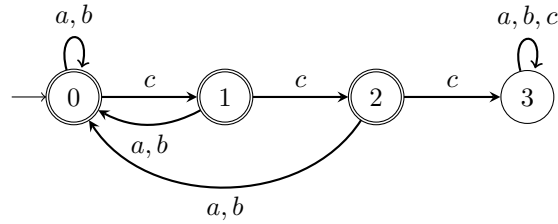
Notemos, que la expresión regular define correctamente a L , ya que incluye todas las palabras sobre el alfabeto $\{a, b, c\}$ que no contiene tres letras 'c' seguidas.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1 puntos) Por construir $(c + cc)^?$
- (1 puntos) Por construir $(a + b)^*$
- (1.5 puntos) Por construir $((a + b)c)^*$
- (1.5 puntos) Por construir $((a + b)cc)^*$
- (1 puntos) Por entregar una explicación con sentido

Pregunta 2.2

Un autómata determinista que defina a L puede ser el siguiente:



Para demostrar que el autómata es mínimo, realizaremos el algoritmo de minimización visto en clases. En primer lugar, construiremos una tabla con todos los pares de nodos sin marcar.

	0	1	2	3
0	-			
1	-	-		
2	-	-	-	
3	-	-	-	-

En segundo lugar, marcar todos los pares $\{p, q\}$ si $p \in F$ y $q \notin F$ o viceversa.

	0	1	2	3
0	-			X
1	-	-		X
2	-	-	-	X
3	-	-	-	-

Luego, si $\{p, q\}$ no están marcado y $\{\delta(p, a); \delta(q, a)\}$ si están marcados. Entonces, marcar $\{p, q\}$.

	0	1	2	3
0	-		c(1, 3)	X
1	-	-	c(2, 3)	X
2	-	-	-	X
3	-	-	-	-

	0	1	2	3
0	-	c(1,2)	X	X
1	-	-	X	X
2	-	-	-	X
3	-	-	-	-

	0	1	2	3
0	-	X	X	X
1	-	-	X	X
2	-	-	-	X
3	-	-	-	-

Cómo todas las entradas están marcadas, entonces para todo estado p, q en el autómata, p y q son distinguibles. Por lo tanto, el autómata es mínimo.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por construir el autómata mínimo
- **(1 punto)** Por manejar el método de minimización
- **(1 punto)** Por marcar el par estados finales y no finales
- **(2 punto)** Por realizar correctamente cada iteración (2 iteraciones)

Pregunta 3

Para realizar la demostración pedida, utilizaremos el contrapositivo del lema de bombeo. Una posible solución es la siguiente:

Sea un $N > 0$ arbitrario. Entonces, existe la siguiente palabra:

$$ab^N ab^N$$

La cual se puede dividir de la siguiente forma:

$$x = a$$

$$y = b^N$$

$$z = ab^N$$

Entonces, para todo $y = b^l b^m b^n$ con $u = b^l$, $v = b^m$, $w = b^n$ y $m > 0$, existe $i = 2$ tal que:

$$x \cdot u \cdot v^2 \cdot w \cdot z = ab^N b^m ab^N$$

Entonces buscamos demostrar que $ab^N b^m ab^N \notin Eq^+$. Debido a que la palabra a analizar comienza con a , y solo hay dos a presentes en esta, la única forma de separar la palabra en dos sería de la forma:

$$w \cdot w = ab^N b^m \cdot ab^N$$

Y como $m > 0$, tendremos que $ab^N b^m \neq ab^N$ y por ende, la palabra no pertenece a Eq^+ . Por lo tanto, Eq^+ no es regular.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por seleccionar un N y particiones x, y, z de la palabra adecuadas.
- **(1 punto)** Por dividir y en particiones u, v, w adecuadas, con $v \neq \epsilon$.
- **(1 punto)** Por encontrar un $i \geq 0$ adecuado para expandir v .
- **(2 puntos)** Por demostrar que la palabra resultante no pertenece a Eq^+ y por ende, que el lenguaje es no regular.

Pregunta 4

Una posible solución para esta pregunta es lo siguiente.

Como L_1 y L_2 son regulares, existen autómatas no deterministas $\mathcal{A}_1 = (Q_1, \Sigma, \Delta_1, I_1, F_1)$ y $\mathcal{A}_2 = (Q_2, \Sigma, \Delta_2, I_2, F_2)$, tales que $\mathcal{L}(\mathcal{A}_1) = L_1$ y $\mathcal{L}(\mathcal{A}_2) = L_2$.

La idea general, es construir un nuevo autómata no determinista, que al leer $w = u_1 \cdot v_2$ como input, ejecute $\mathcal{A}_1$ sobre el w , intente adivinar en qué momento termina u_1 , y mediante una ϵ -transición, pasar a ejecutar v_2 a partir de algún estado de $\mathcal{A}_2$. Si la ejecución termina en un estado final de $\mathcal{A}_2$, entonces el autómata acepta el input.

Para lograr esto, nuestro nuevo autómata debe contener a $\mathcal{A}_1$ y $\mathcal{A}_2$, y tener ϵ -transiciones que vayan de algunos estados de $\mathcal{A}_1$ a algunos estados de $\mathcal{A}_2$. La elección de cuáles estados van a tener ϵ -transiciones salientes o entrantes, viene por la definición de $L_1 \dot{+} L_2$.

Dado un $w = u_1 \cdot v_2$, para verificar si $w \in L_1 \dot{+} L_2$, es necesario comprobar que existe u_2 tal que $u_1 \cdot u_2 \in L_1$, y que existe v_1 tal que $v_1 \cdot v_2 \in L_2$. Para esto, construimos los siguientes conjuntos de estados:

$$Q_1^F = \{p \mid \exists u \in \Sigma^* \exists q \in F_1. (p, u) \vdash_{\mathcal{A}_1}^* (q, \epsilon)\}$$

$$Q_2^I = \{q \mid \exists v \in \Sigma^* \exists p \in I_2. (p, v) \vdash_{\mathcal{A}_1}^* (q, \epsilon)\}$$

Con esto, construimos el autómata no determinista $\mathcal{A}^\dagger = (Q^\dagger, \Sigma, \Delta^\dagger, I^\dagger, F^\dagger)$ donde

- $Q^\dagger = Q_1 \uplus Q_2$
- $I^\dagger = I_1$
- $F^\dagger = F_2$
- $\Delta^\dagger = \Delta_1 \uplus \Delta_2 \uplus \{(p, \epsilon, q) \mid p \in Q_1^F \wedge q \in Q_2^I\}$

Ahora, queda demostrar que $\mathcal{L}(\mathcal{A}^\dagger) = L_1 \dot{+} L_2$.

Primero demostraremos que $L_1 \dot{+} L_2 \subseteq \mathcal{L}(\mathcal{A}^\dagger)$. Sea $w \in L_1 \dot{+} L_2$. Por definición de $L_1 \dot{+} L_2$, existen $u_1, u_2, v_1, v_2 \in \Sigma^*$ tales que $w = u_1 \cdot v_2$, $u_1 \cdot u_2 \in L_1$, y $v_1 \cdot v_2 \in L_2$. Luego, existen $q_0^1 \in I_1, q_f^1 \in F_1, q_0^2 \in I_2, q_f^2 \in F_2$ tales que:

$$(q_0^1, u_1 \cdot u_2) \vdash_{\mathcal{A}_1}^* (q_f^1, \epsilon)$$

$$(q_0^2, v_1 \cdot v_2) \vdash_{\mathcal{A}_2}^* (q_f^2, \epsilon)$$

Existen $p \in Q_1, q \in Q_2$ tales que se cumple lo siguiente:

$$(q_0^1, u_1 \cdot u_2) \vdash_{\mathcal{A}_1}^* (p, u_2) \vdash_{\mathcal{A}_1}^* (q_f^1, \epsilon)$$

$$(q_0^2, v_1 \cdot v_2) \vdash_{\mathcal{A}_2}^* (q, v_2) \vdash_{\mathcal{A}_2}^* (q_f^2, \epsilon)$$

Por construcción de $\mathcal{A}^\dagger$, tenemos que $q_0^1 \in I^\dagger, q_f^2 \in F^\dagger$. También notamos que $p \in Q_1^F$, y que $q \in Q_2^I$. Luego, por definición de $\Delta^\dagger$, hay una ϵ -transición que lleva de p a q . Entonces, lo siguiente es una secuencia válida de configuraciones de $\mathcal{A}^\dagger$:

$$(q_0^1, u_1 \cdot v_2) \vdash_{\mathcal{A}^\dagger}^* (p, v_2) \vdash_{\mathcal{A}^\dagger}^* (q, v_2) \vdash_{\mathcal{A}^\dagger}^* (q_f^2, \epsilon)$$

De aquí queda claro que $(q_0^1, u_1 \cdot v_2) \vdash_{\mathcal{A}^\dagger}^* (q_f^2, \epsilon)$, por lo que $u_1 \cdot v_1 \in \mathcal{L}(\mathcal{A}^\dagger)$.

Luego, $L_1 \dot{+} L_2 \subseteq \mathcal{L}(\mathcal{A}^\dagger)$.

Ahora demostraremos que $\mathcal{L}(\mathcal{A}^\dagger) \subseteq L_1 \dot{+} L_2$. Sea $w \in \mathcal{L}(\mathcal{A}^\dagger)$. Luego, $\exists q_0 \in I_1, \exists q_f \in F_2$ tal que

$$(q_0, w) \vdash_{\mathcal{A}^\dagger}^* (q_f, \epsilon)$$

Por construcción, tenemos que $Q_1 \cap Q_2 = \emptyset$, por lo que en algún momento de la ejecución, se debe utilizar una ϵ -transición que lleve de los estados Q_1 a los estados Q_2 . Entonces, $\exists u_1, v_2 \in \Sigma^*$ tal que $w = u_1 \cdot v_2$, y $\exists p \in Q_1^F, q \in Q_2^I$ tal que:

$$(q_0, u_1 \cdot v_2) \vdash_{\mathcal{A}^\dagger}^* (p, v_2) \vdash_{\mathcal{A}^\dagger} (q, v_2) \vdash_{\mathcal{A}^\dagger}^* (q_f, \epsilon)$$

Por construcción, se tiene que $(q_0, u_1) \vdash_{\mathcal{A}_1}^* (p, \epsilon)$. Y, como $p \in Q_1^F$, existe u_2 tal que $(p, u_2) \vdash_{\mathcal{A}_1}^* (q_f^1, \epsilon)$ con $q_f^1 \in F_1$. Luego,

$$(q_0, u_1 \cdot u_2) \vdash_{\mathcal{A}_1}^* (q_f^1, \epsilon)$$

Por lo tanto, $u_1 \cdot u_2 \in L_1$.

Análogamente, se tiene que $(q, v_2) \vdash_{\mathcal{A}_2}^* (q_f, \epsilon)$. Y, como $q \in Q_2^I$, existe v_1 tal que $(q_0^2, v_1) \vdash_{\mathcal{A}_2}^* (q, \epsilon)$ con $q_0^2 \in I_2$. Luego,

$$(q_0^2, v_1 \cdot v_2) \vdash_{\mathcal{A}_2}^* (q_f, \epsilon)$$

Por lo tanto, $v_1 \cdot v_2 \in L_2$.

Como existen $u_2, v_1 \in \Sigma^*$ tales que $u_1 \cdot u_2 \in L_1$ y $u_1 \cdot u_2 \in L_1$, tenemos que $w \in L_1 \dot{+} L_2$.

Luego, $\mathcal{L}(\mathcal{A}^\dagger) \subseteq L_1 \dot{+} L_2$.

Como $L_1 \dot{+} L_2 \subseteq \mathcal{L}(\mathcal{A}^\dagger) \wedge \mathcal{L}(\mathcal{A}^\dagger) \subseteq L_1 \dot{+} L_2$, se cumple que $L_1 \dot{+} L_2 = \mathcal{L}(\mathcal{A}^\dagger)$, y concluimos que $L_1 \dot{+} L_2$ es regular.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por expresar la idea de ϵ -transiciones que conecten estados de $\mathcal{A}_1$ y $\mathcal{A}_2$
- **(1 punto)** Por construcción de $\mathcal{A}^\dagger$
- **(1 punto)** Por modelar correctamente de qué estados salen/entran las ϵ -transiciones en $\mathcal{A}^\dagger$
- **(1 punto)** Por demostrar $L_1 \dot{+} L_2 \subseteq \mathcal{L}(\mathcal{A}^\dagger)$
- **(1 punto)** Por demostrar $\mathcal{L}(\mathcal{A}^\dagger) \subseteq L_1 \dot{+} L_2$



IIC2223 – Teoría de Autómatas y Lenguajes Formales – 2' 2024
 IIC2224 – Autómatas y Compiladores

Pauta Interrogación 1

Preguntas en blanco: Preguntas o ítems entregados en blanco se evaluarán con 0.5 de 6 puntos.

Pregunta 1

Sean L_1 y L_2 lenguajes regulares. Demuestre que $L_1 \cdot L_2$ es un lenguaje regular.

Solución

Sean $A_1 = (Q_1, \Sigma, \Delta, I_1, F_1)$ y $A_2 = (Q_2, \Sigma, \Delta, I_2, F_2)$ dos NFAs tal que $L_1 = \mathcal{L}(A_1)$, $L_2 = \mathcal{L}(A_2)$

Sea el NFA $A = (Q, \Sigma, \Delta, I, F)$, tal que,

$$\begin{aligned} Q &= Q_1 \uplus Q_2 \\ I &= I_1 \\ F &= F_2 \\ \Delta &= \Delta_1 \uplus \Delta_2 \uplus \{(p, \varepsilon, q) \cdot p \in F_1, q \in I_2\} \end{aligned}$$

Por demostrar. $L_1 \cdot L_2 = \mathcal{L}(A)$, es decir: $w \in L_1 \cdot L_2 \Leftrightarrow w \in \mathcal{L}(A)$

- $w \in L_1 \cdot L_2 \Rightarrow w \in \mathcal{L}(A)$

Sea $w = w_1 w_2 \in L_1 \cdot L_2$ tal que $w_1 \in L_1, w_2 \in L_2$. Como $\mathcal{L}(A_1) = L_1$ y $\mathcal{L}(A_2) = L_2$, se tiene, necesariamente, que:

$$(q_0^1, w_1) \vdash_{\Delta_1}^* (q_f^1, \varepsilon) \wedge (q_0^2, w_2) \vdash_{\Delta_2}^* (q_f^2, \varepsilon)$$

Con $q_0^1 \in I_1, q_f^1 \in F_1, q_0^2 \in I_2, q_f^2 \in F_2$. Por construcción y dado que $(q_f^1, \varepsilon, q_0^2) \in \Delta$, se tiene que:

$$(q_0^1, w_1 w_2) \vdash_{\Delta}^* (q_f^1, w_2) \vdash_{\Delta} (q_0^2, w_2) \vdash_{\Delta}^* (q_f^2, \varepsilon)$$

Así, se concluye que $(q_0^1, w_1 w_2) \vdash_{\Delta}^* (q_f^2, \varepsilon) \Rightarrow w_1 w_2 \in \mathcal{L}(A)$, con $q_0^1 \in I, q_f^2 \in F$.

- $w \in \mathcal{L}(A) \Rightarrow w \in L_1 \cdot L_2$

Sea $w \in \mathcal{L}(A)$, se tiene que:

$$(q_0, w) \vdash_{\Delta}^* (q_f, \varepsilon)$$

Con, $q_0 \in I = I_1, q_f \in F = F_2$. Por construcción, se tiene que en algún momento se debe saltar de Q_1 a Q_2 , esto es,

$$(q_0, w_1 w_2) \vdash_{\Delta}^* (q_1, w_2) \vdash_{\Delta} (q_2, w_2) \vdash_{\Delta}^* (q_f, \varepsilon)$$

Por la existencia de la ε -transición, $q_1 \in F_1, q_2 \in I_2$. Como $q_0, q_1 \in Q_1$ y $q_2, q_f \in Q_2$, se tiene que:

$$(q_0, w_1) \vdash_{\Delta_1}^* (q_1, \varepsilon) \wedge (q_2, w_2) \vdash_{\Delta_2}^* (q_f, \varepsilon)$$

Por lo tanto, $w_1 \in L_1, w_2 \in L_2$.

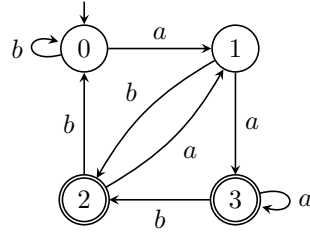
Distribución de puntajes

Los 6 puntos de la pregunta se encuentran distribuidos de la siguiente forma:

- **(1 pto.)** Por definir los estados del autómata A , -0.5 si no considera la unión disjunta.
- **(1 pto.)** Por definir los conjuntos de estados iniciales I y finales F del autómata A .
- **(1 pto.)** Por definir el conjunto Δ de transiciones del autómata A , -0.5 si no considera la unión disjunta.
- **(1.5 ptos)** Por la demostración hacia el lado derecho de la doble implicancia.
 - **(0.5 ptos)** Por el planteamiento y escribir las transiciones de estados.
 - **(0.5 ptos)** Por utilizar la construcción del autómata A .
 - **(0.5 ptos)** Por lograr concluir la demostración utilizando lo anterior.
- **(1.5 ptos)** Por la demostración hacia el lado izquierdo de la doble implicancia. (una buena argumentación de por qué los pasos anteriores son reversibles también se considerará correcta).
 - **(0.5 ptos)** Por el planteamiento y escribir las transiciones de estados.
 - **(0.5 ptos)** Por utilizar la construcción del autómata A .
 - **(0.5 ptos)** Por lograr concluir la demostración utilizando lo anterior.

Pregunta 2

Considere el siguiente autómata finito determinista $\mathcal{A}$ sobre el alfabeto $\Sigma = \{a, b\}$.



1. Minimice el número de estados de $\mathcal{A}$ utilizando el algoritmo de minimización de estados.
2. Construya una expresión regular equivalente a $\mathcal{L}(\mathcal{A})$. Justifique su respuesta.

Solución

1. Como se solicita en el enunciado, seguiremos el algoritmo de minimización de estados. Realizamos los siguientes pasos:

- (a) Paso 1 del algoritmo: Construimos la tabla con los pares $\{p, q\}$ sin marcar (dejamos con negrita los estados finales, es decir, 2 y 3):

	0	1	2	3
0	-			
1	-	-		
2	-	-	-	
3	-	-	-	-

- (b) Paso 2 del algoritmo: Marcamos $\{p, q\}$ si $p \in F$ y $q \notin F$ o viceversa:
Debido a lo anterior, tenemos que marcar los siguientes pares $\{0, 2\}$, $\{0, 3\}$, $\{1, 2\}$, $\{1, 3\}$. Obtenemos la siguiente tabla:

	0	1	2	3
0	-		✓	✓
1	-	-	✓	✓
2	-	-	-	
3	-	-	-	-

- (c) Paso 3 del algoritmo (Lo repetimos hasta que no hayan cambios): Si $\{p, q\}$ no está marcado y $\{\delta(p, a), \delta(q, a)\}$ está marcado para algún $a \in \Sigma$, entonces marcamos $\{p, q\}$.

Siguiendo lo anterior, notemos que debemos marcar los siguientes pares:

- El par $\{0, 1\}$ debido a que $\{\delta(0, a), \delta(1, a)\} \rightarrow \{1, 3\}$, que resulta en un par que está marcado. Notemos que si seguimos $\{\delta(0, b), \delta(1, b)\} \rightarrow \{0, 2\}$ que también está marcado. Por lo que marcamos el par $\{0, 1\}$.
- El par $\{2, 3\}$ debido a que $\{\delta(2, a), \delta(3, a)\} \rightarrow \{1, 3\}$, que resulta en un par que está marcado. Notemos que si seguimos $\{\delta(2, b), \delta(3, b)\} \rightarrow \{0, 2\}$ que también está marcado. Por lo que marcamos el par $\{2, 3\}$.

Obtenemos la siguiente tabla:

	0	1	2	3
0	-	✓	✓	✓
1	-	-	✓	✓
2	-	-	-	✓
3	-	-	-	-

Notemos que la tabla ya está completa, por lo que no es necesario repetir el paso 3. Ahora, notemos que todos los pares de estados $\{p, q\}$ de la tabla están marcados, por lo que todos los pares de estados son distinguibles. Debido a lo anterior podemos concluir que el autómata ya se encuentra minimizado, es decir, el resultado del algoritmo es el autómata inicial.

Distribución de puntajes

Los 6 puntos de la pregunta se encuentran distribuidos de la siguiente forma:

- **(1 pto.)** Por completar la tabla o explicar el proceso del llenado de esta siguiendo el algoritmo.
 - **(1 pto.)** Por marcar los pares de estados $\{p, q\}$ si $p \in F$ y $q \notin F$ o viceversa, es decir, por realizar correctamente el paso 1.
 - **(1.5 pto.)** Por marcar el par $\{0, 1\}$ debido a $\{\delta(0, a), \delta(1, a)\} \rightarrow \{1, 3\}$ (o siguiendo la otra transición).
 - **(1.5 pto.)** Por marcar el par $\{2, 3\}$ debido a que $\{\delta(2, a), \delta(3, a)\} \rightarrow \{1, 3\}$ (o siguiendo la otra transición).
 - **(1 pto.)** Por concluir correctamente luego de finalizar el algoritmo.
2. Para esta pregunta, existen diversas formas de llegar a una expresión regular equivalente a $\mathcal{L}(\mathcal{A})$, por lo que desarrollaremos 2 de ellas a continuación.

(a) La primera estrategia consiste en analizar el autómata:

- Si vemos el estado 0, podemos notar que su función es recordar que las últimas dos letras son bb o que el largo de la palabra es < 2 .
- Luego, el estado 1 recuerda que las últimas dos letras son ba o a .
- A continuación, el estado 2 recuerda que las últimas dos letras son ab .
- Por último, el estado 3 recuerda que las últimas dos letras son aa .

Si juntamos todo lo anterior podemos concluir que el autómata acepta todas las palabras donde la **penúltima** letra es una a . Eso nos da la siguiente expresión regular:

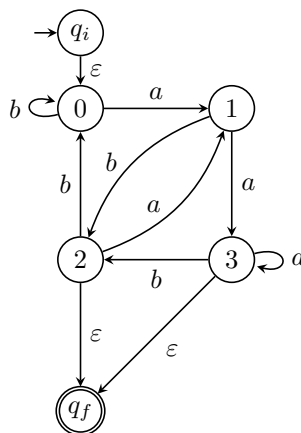
$$(a + b)^* \cdot a \cdot (a + b)$$

Distribución de puntajes

Si se siguió esta estrategia, los 6 puntos de la pregunta se encuentran distribuidos de la siguiente forma:

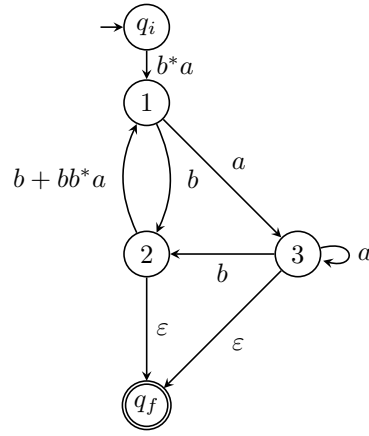
- **(2 pto.)** Por explicar que hace cada estado y/o explicar cuales son las palabras que acepta el autómata.
 - **(4 pto.)** Por construir la expresión regular correctamente relacionándolo con lo anterior.
- (b) La segunda estrategia es utilizando el algoritmo de eliminación de estados:

Antes de comenzar con el método de eliminación de estados, primero se debe convertir el DFA a un GNFA. Para ello, debemos agregar 2 nuevos estados q_i y q_f . q_i debemos conectarlo al estado 0 y los estados 2 y 3 deben ser conectados a q_f mediante ε -transiciones, resultando en el siguiente GNFA:

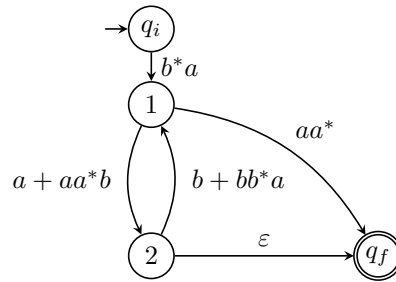


A continuación, iniciamos el método de eliminación de estados. Por simplicidad, las expresiones regulares se irán simplificando a medida que se avanza en el método, pero no se mostrará explícitamente. Por ejemplo, (εa) se simplificará a (a) y $(\emptyset + a)$ se simplificará a (a) .

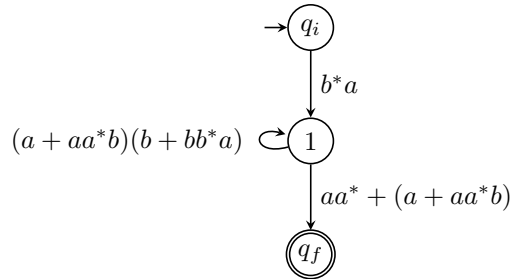
Paso 1: Eliminar el estado 0.



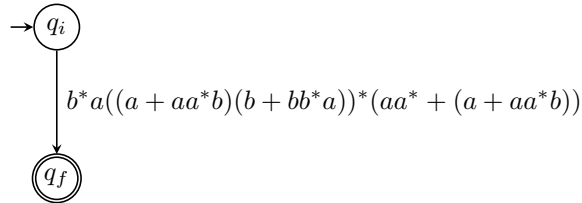
Paso 2: Eliminar el estado 3.



Paso 3: Eliminar el estado 2.



Paso 4: Eliminar el estado 1.



Dado que solo tenemos 2 estados, el inicial y el final, la expresión regular resultante es la expresión correspondiente a la transición entre estos dos estados, es decir, $b^*a((a + aa^*b)(b + bb^*a))^*(aa^* + (a + aa^*b))$.

Distribución de puntajes

Si se siguió esta estrategia, los 6 puntos de la pregunta se encuentran distribuidos de la siguiente forma:

- **(1 pto.)** Por convertir el NFA/DFA a GNFA.
- **(1 pto.)** Por eliminar el estado 3.
- **(1 pto.)** Por eliminar el estado 0.
- **(1 pto.)** Por eliminar el estado 2.
- **(1 pto.)** Por eliminar el estado 1.
- **(1 pto.)** Por concluir con la eliminación de estados y obtener la expresión regular.

Problema 3

Sea $\Sigma = \{a, b, c\}$. Demuestre que los siguientes lenguajes no son regulares:

1. $\text{Prod} = \{a^i b^j c^{i \cdot j} \in \Sigma^+ \mid i \geq 0 \wedge j \geq 0\}$
2. $\text{Mod} = \{a^i b^j c^{i \bmod j} \in \Sigma^+ \mid i \geq 0 \wedge j \geq 2\}$

Solución

1. Necesitamos demostrar que el lenguaje $\text{Prod} = \{a^i b^j c^{i \cdot j} \in \Sigma^+ \mid i \geq 0 \wedge j \geq 0\}$ no es regular. Para ello, podemos utilizar el contra positivo del lema de bombeo.
 - Para todo $N > 0$, elegimos la palabra $P = a^N b^N c^{N^2}$.
 - Elegimos x, y, z tal que $P = xyz$, $x = a^N b^N$, $y = c^{N^2}$ ($|y| \geq N$) y $z = \varepsilon$.
 - Para toda descomposición $y = c^k c^l c^m$ con $k + l + m = N^2$ y $l > 0$.
 - Tenemos nuestra nueva palabra $P' = a^N b^N c^{k+i \cdot l+m}$.
 - Elegimos bombear con $i = 0$, obteniendo que $N^2 = k + 0 \cdot l + m$, pero $N^2 > k + m$, ya que $l > 0$.
 - Por lo tanto, $N^2 \neq k + m$ y por ende, Prod no es regular.
2. Similar a antes, necesitamos demostrar que el lenguaje $\text{Mod} = \{a^i b^j c^{i \bmod j} \in \Sigma^+ \mid i \geq 0 \wedge j \geq 2\}$ no es regular. Para ello, nuevamente utilizamos el contra positivo del lema de bombeo.
 - Para todo $N > 0$, elegimos la palabra $M = a^N b^{2N} c^N$.
 - Elegimos x, y, z tal que $M = xyz$, $x = a^N b^{2N}$, $y = c^N$ ($|y| \geq N$) y $z = \varepsilon$.
 - Para toda descomposición $y = c^k c^l c^m$ con $k + l + m = N$ y $l > 0$.
 - Tenemos nuestra nueva palabra $M' = a^N b^{2N} c^{k+i \cdot l+m}$.
 - Elegimos bombear con $i = 0$, obteniendo que $N = k + 0 \cdot l + m$, pero $N > k + m$, ya que $l > 0$.
 - Por lo tanto, $N \neq k + m$ y por ende, Mod no es regular.

Distribución de puntajes

Ambos ítems valen 6 puntos cada uno, distribuidos de la siguiente forma:

- (1 pto.) Por el correcto uso del lema de bombeo.
- (1 pto.) Por la correcta elección de la palabra P o M , esta debe estar bien definida, dadas las restricciones del lenguaje.
- (1 pto.) Por la correcta elección de x , y y z .
- (1 pto.) Por la correcta elección de i al bombear.
- (2 ptos) Por la correcta demostración de que la palabra bombeada no pertenece al lenguaje.

Problema 4

Sea $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ un autómata finito no-determinista (NFA). Se define el lenguaje universal de $\mathcal{A}$ como:

$$\mathcal{L}_\forall(\mathcal{A}) = \{w \in \Sigma^* \mid \text{ toda ejecución de } \mathcal{A} \text{ sobre } w \text{ es de aceptación} \}$$

Demuestre que para todo NFA $\mathcal{A}$, el lenguaje $\mathcal{L}_\forall(\mathcal{A})$ es siempre regular.

Solución

Sea $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ un NFA. Tenemos el siguiente lenguaje:

$$\mathcal{L}_\forall(\mathcal{A}) = \{w \mid \text{ todas las ejecuciones de } \mathcal{A} \text{ sobre } w \text{ que son de aceptación}\}$$

Para demostrar que el lenguaje es regular, definimos el siguiente autómata:

$$\mathcal{A}_\forall^{det} = (2^Q, \Sigma, \Delta^{det}, q_0^{det}, F_\forall^{det})$$

- $2^Q = \{S \mid S \subseteq Q\}$
- $q_0^{det} = I$
- $\Delta^{det}(S, a) = \{\exists p \in S. (p, a, q) \in \Delta\}$
- $F_\forall^{det} = \{S \subseteq Q \mid S \subseteq F\} = 2^F$

En palabras simples, como funciona el autómata es igual que cuando se determiniza un NFA, solamente que lo que cambia es que ahora los estados finales son todas las combinaciones que se pueden hacer con los estados finales de $\mathcal{A}$, con esto nos aseguramos de que todas las ejecuciones sean aceptadas por dicho autómata.

Es importante ver que aca $F_\forall^{det}$ contiene al vacío. Esto, dado que si es que no existen ejecuciones de $\mathcal{A}$ sobre w , entonces automáticamente se cumple la propiedad (dado que es un para todo).

Sea $w = a_1 a_2 a_3 \dots a_n \in \Sigma^*$ cualquiera. Como $\mathcal{A}_\forall^{det}$ es determinista. Sea ρ la ejecución de $\mathcal{A}_\forall^{det}$ sobre w , tal que:

$$\rho : S_0 \xrightarrow{a_1} S_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} S_n$$

En clases se vio en la demostración de determinización que:

Lema 1. Si $\rho : q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} q_n$ es una ejecución de $\mathcal{A}$ sobre w , entonces $\forall i. p_i \in S_i$.

Lema 2. $\forall i \leq n. \forall p \in S_i$, existe una ejecución $\rho : q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_i} q_i$ de $\mathcal{A}$ sobre w tal que $p_i = p$.

Usaremos ambos lemas demostrados en clases para demostrar la correctitud de la construcción.

Por demostrar. $w \in \mathcal{L}(\mathcal{A}_\forall^{det}) \iff w \in \mathcal{L}_\forall(\mathcal{A})$

Como sabemos, si $w \in \mathcal{L}(\mathcal{A}_\forall^{det})$ entonces $S_n \subseteq 2^F$, por lo tanto, la siguiente demostración es equivalente:

$$S_n \subseteq 2^F \iff w \in \mathcal{L}_\forall(\mathcal{A})$$

($\rightarrow$) Suponga que $S_n \subseteq 2^F$. Sea $\rho : q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} q_n$ una ejecución de aceptación de $\mathcal{A}$ sobre w cualquiera. Por lema 1: $p_n \in S_n$, por lo tanto $p_n \in F$. Con esto tenemos que ρ es de aceptación. Como esto sirve para cualquier ejecución ρ , entonces todas las ejecuciones son de aceptación. Con esto concluimos que $w \in \mathcal{L}_\forall(\mathcal{A})$.

($\leftarrow$) Vamos a demostrar por contrapositivo que $S_n \not\subseteq 2^F \rightarrow w \notin \mathcal{L}_\forall(\mathcal{A})$.

Suponga que existe $p \in S_n$ tal que $p \notin F$. Por lema 2: $\exists \rho : p_0 \xrightarrow{a_1} \dots \xrightarrow{a_n} p_n$ de $\mathcal{A}$ sobre w tal que $p_n = p$. Como $p_n = p \notin F$, entonces ρ no es de aceptación. Por lo tanto $w \notin \mathcal{L}(\mathcal{A}_\forall^{det})$.

Distribución de puntajes

- (2 ptos.) Por definir correctamente la determinización de $\mathcal{A}$.
- (2 ptos.) Por definir correctamente $F_\forall^{det}$.
- (1 pto.) Por demostrar que $w \in \mathcal{L}(\mathcal{A}_\forall^{det}) \rightarrow w \in \mathcal{L}_\forall(\mathcal{A})$.
- (1 pto.) Por demostrar que $w \in \mathcal{L}(\mathcal{A}_\forall^{det}) \leftarrow w \in \mathcal{L}_\forall(\mathcal{A})$.



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de autómatas y lenguajes formales — 2'2020

CONTROL 2

Indicaciones

- La duración del control es 1 hora y 30 minutos.
- Responda cada pregunta en una hoja separada y ponga su nombre en cada hoja de respuesta.
- Debe entregar una copia digital de cada pregunta por el buzón del curso, antes de las 23:59 horas del día del control.
- Debe preocuparse que la copia digital y su calidad sea legible. En caso de hacerla con papel y lápiz, se recomienda usar hojas blancas y un lápiz oscuro que sea visible en la versión digital. En caso de no ser legible, no podrá ser evaluada su solución.
- En caso de hacer el control fuera del horario, se recomienda tomar el tiempo (1 hora y 30 minutos) y entregarlo justo después de concluido el tiempo.
- Durante la evaluación puede hacer uso de sus apuntes o slides del curso.
- Esta es una evaluación estrictamente individual y, por lo tanto, no puede compartir información con sus compañeros o usar material fuera de sus apuntes o slides del curso. En caso de hacerlo, el control no reflejará su progreso en el curso, viéndose perjudicada su formación personal y profesional.
- **Al comienzo de cada pregunta debe escribir la siguiente oración y firmarla:**

“Doy mi palabra que la siguiente solución de la pregunta X fue desarrollada y escrita individualmente por mi persona según el código de honor de la Universidad.”

En caso de no escribir la oración o no firmarla, su solución no será evaluada.

Pregunta 1

Sea $\Sigma = \{a, b\}$. Recuerde que $w \in \Sigma^*$ es un palíndromo si $w = w^R$ donde w^R es la palabra reverso de w . Un palíndromo se dice no trivial si es de largo mayor o igual a dos. Sea Pal_Σ el conjunto de todas las palabras $w \in \Sigma^*$ tal que w es un palíndromo no trivial y $(\text{Pal}_\Sigma)^*$ la clausura de Kleene de Pal_Σ .

1. Demuestre que el lenguaje $(\text{Pal}_\Sigma)^*$ es libre de contexto.
2. Demuestre que el lenguaje $(\text{Pal}_\Sigma)^*$ es no regular.

Pregunta 2

Para una gramática $\mathcal{G} = (V, \Sigma, P, S)$ y $X \in V$, se define el lenguaje de X como $\mathcal{L}(X) = \{w \in \Sigma^* \mid X \Rightarrow_{\mathcal{G}}^* w\}$.

1. Demuestre que, si $\mathcal{L}(\mathcal{G})$ es un lenguaje finito, entonces existe una gramática $\mathcal{G}' = (V', \Sigma, P', S')$ tal que $\mathcal{L}(\mathcal{G}') = \mathcal{L}(\mathcal{G})$ y $\mathcal{G}'$ tiene una sola variable, esto es, $V' = \{S'\}$.
2. Demuestre que, si $\mathcal{L}(\mathcal{G})$ es un lenguaje infinito, entonces existe una gramática $\mathcal{G}' = (V', \Sigma, P', S')$ tal que $\mathcal{L}(\mathcal{G}') = \mathcal{L}(\mathcal{G})$ y todas sus variables generan un lenguaje infinito, esto es, para todo $X \in V'$ se tiene que $\mathcal{L}(X)$ es infinito.



PAUTA CONTROL 2

Pregunta 1

Pregunta 1.1

Una posible solución es definir la gramática $\mathcal{G} = (V, \Sigma, P, S)$ con $\Sigma = \{a, b\}$, $V = \{X, Y, S\}$ con las siguientes producciones:

$$\begin{aligned} S &\rightarrow YS \mid \varepsilon \\ X &\rightarrow aXa \mid bXb \mid a \mid b \mid \varepsilon \\ Y &\rightarrow aXa \mid bXb \end{aligned}$$

y explicar la correctitud de esta gramática. Dado lo anterior, la distribución de puntaje es la siguiente:

- (1 punto) Por definir bien X, es decir la producción asociada a los palíndromos comunes.
- (1 punto) Por definir la variable inicial tal que genere la clausura de Kleene.
- (0.5 puntos) Por definir bien Y o la producción que asegure la no trivialidad de los palíndromos.
- (0.5 puntos) Por explicar la correctitud de la gramática.

Pregunta 1.2

Una posible solución es usar el lema de bombeo con $w = \underbrace{a}_x \underbrace{b^N}_y \underbrace{ab^N a}_z$. Luego dada una división arbitraria para y :

$$b^n b^m b^l = b^N$$

Bombeamos a^m con $i = 2$ nos queda $w' = ab^{N+m}ab^N a$ y podemos notar que no es un palíndromo ni es una concatenación de palíndromos.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1.5 puntos) Por elegir de manera correcta la palabra a bombear con sus subdivisiones correspondientes.
- (1 punto) Por bombear correctamente.
- (0.5 puntos) Por usar el lema de bombeo correctamente.

Pregunta 2

Pregunta 2.1

Dado que $\mathcal{L}(\mathcal{G})$ es un lenguaje finito, podemos enumerar las palabras pertenecientes al lenguaje, $\mathcal{L}(\mathcal{G}) = \{w_1, w_2, \dots, w_n\}$. Luego, sea $\mathcal{G}' = (\{S'\}, \Sigma, P', S')$, tal que $S' \rightarrow w_i \in P'$ para todo $w_i \in \mathcal{L}(\mathcal{G})$. Es fácil ver que $\mathcal{L}(\mathcal{G}) = \mathcal{L}(\mathcal{G}')$ ya que $w \in \mathcal{L}(\mathcal{G})$ si, y solo si, $S' \rightarrow w \in P'$ si, y solo si, $w \in \mathcal{L}(\mathcal{G})'$. Finalmente $\mathcal{G}'$ es una gramática con una sola variable y cumple que $\mathcal{L}(\mathcal{G}) = \mathcal{L}(\mathcal{G}')$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** Por una correcta utilización de que el lenguaje $\mathcal{L}(\mathcal{G})$ es finito.
- **(2 puntos)** Por dar una contrucción correcta de $\mathcal{G}'$.
- **(0.5 puntos)** Por demostrar que $\mathcal{L}(\mathcal{G}) = \mathcal{L}(\mathcal{G}')$.

Pregunta 2.2

Sea $V_\infty = \{X \in V \mid \mathcal{L}(X) \text{ es infinito}\}$ y $V_{<\infty} = \{X \in V \mid \mathcal{L}(X) \text{ es finito}\}$. Dado que $\mathcal{L}(\mathcal{G})$ es infinito, $\mathcal{L}(S)$ también lo es, por lo tanto, podemos concluir que $V_\infty \neq \emptyset$.

Sea $\mathcal{G}' = (V_\infty, \Sigma, P', S)$ tal que para cada regla de la forma $X \rightarrow \alpha_0 X_1 \alpha_1 X_2 \dots X_k \alpha_k \in P$, con $X \in V_\infty, \alpha_i \in \{V_\infty \cup \Sigma\}^*, X_j \in V_{<\infty} i \in \{0, 1, \dots, k\}$, agregamos una regla $X \rightarrow \alpha_0 w_1 \alpha_1 \dots w_k \alpha_k \in P'$ para todo $w_j \in \mathcal{L}(X_j)$. Cabe destacar en esta definición de P' , que cuando $k = 0$, $X \rightarrow \alpha_0$, con $\alpha_0 \in \{V_\infty \cup \Sigma\}^*$ pertenecen a P' , por lo que las reglas que no involucran a las variables de lenguaje finito que pertenecen a P , también lo están en P' .

Ahora demostraremos que el lenguaje de cada variable $X \in V_\infty$ es igual con respecto a cada gramática, es decir $\mathcal{L}_{\mathcal{G}}(X) = \mathcal{L}_{\mathcal{G}'}(X)$.

- Por demostrar $\mathcal{L}_{\mathcal{G}}(X) \subseteq \mathcal{L}_{\mathcal{G}'}(X)$. Sabemos que $w \in \mathcal{L}_{\mathcal{G}}(X) \rightarrow X \Rightarrow_{\mathcal{G}}^* w$ Por inducción sobre el largo de la derivación.

Caso base. Para una derivación de largo 1 $X \Rightarrow_{\mathcal{G}} w$ entonces $X \rightarrow w \in P$ y, por lo tanto, $X \rightarrow w \in P'$. De acá podemos deducir que $X \Rightarrow_{\mathcal{G}'} w$ y $w \in \mathcal{L}_{\mathcal{G}'}(X)$.

Inducción. Suponemos que para todo $w \in \mathcal{L}_{\mathcal{G}}(X)$ tal que $X \Rightarrow_{\mathcal{G}}^* w$ con una derivación de largo menor o igual a n , entonces $w \in \mathcal{L}_{\mathcal{G}'}(X)$.

Mostraremos que para todo $w \in \mathcal{L}_{\mathcal{G}}(X)$ tal que $X \Rightarrow_{\mathcal{G}}^* w$ con una derivación de largo igual a $n + 1$, entonces $w \in \mathcal{L}_{\mathcal{G}'}(X)$. Sea $w \in \mathcal{L}_{\mathcal{G}}(X)$ tal que $X \Rightarrow_{\mathcal{G}}^* w$ con una derivación de largo igual a $n + 1$. Entonces $X \Rightarrow_{\mathcal{G}} \beta \Rightarrow_{\mathcal{G}}^* w$ con $\beta \in \{V \cup \Sigma\}$ y $\beta \Rightarrow_{\mathcal{G}}^* w$ es una derivación de largo n .

Se pueden dar dos casos:

1. Si $\beta \in \{V_\infty \cup \Sigma\}$, entonces $X \rightarrow \beta \in P'$ y tenemos que $X \Rightarrow_{\mathcal{G}'} \beta$. Luego, por hipótesis de inducción $X \Rightarrow_{\mathcal{G}'} \beta \Rightarrow_{\mathcal{G}'}^* w$ y $w \in \mathcal{L}_{\mathcal{G}'}(X)$.
2. Si $\beta = \beta_0 X_1 \beta_1 \dots X_m \beta_m$ con cada $X_j \in V_{<\infty}$ (es decir β tiene al menos una variable perteneciente a $V_{<\infty}$), entonces por construcción de $\mathcal{G}'$, $X \rightarrow \beta_0 w_1 \beta_1 \dots w_m \beta_m \in P'$ con cada $w_i \in \mathcal{L}(X_i)$. Por lo tanto $X \Rightarrow_{\mathcal{G}'} \beta_0 w_1 \beta_1 \dots w_m \beta_m \in P'$ para cada $w_i \in \mathcal{L}(X_i)$. Por hipótesis de inducción $X \Rightarrow_{\mathcal{G}'}^* w$ y concluimos que $w \in \mathcal{L}_{\mathcal{G}'}(X)$.

Finalmente, $\mathcal{L}_{\mathcal{G}}(X) \subseteq \mathcal{L}_{\mathcal{G}'}(X)$.

- Por demostrar $\mathcal{L}_{\mathcal{G}'}(X) \subseteq \mathcal{L}_{\mathcal{G}}(X)$. Sabemos que $w \in \mathcal{L}_{\mathcal{G}'}(X)$ y entonces $X \Rightarrow_{\mathcal{G}'}^* w$. De nuevo, por inducción sobre el largo de la derivación:

Caso base. Para una derivación de largo 1 sabemos que $X \Rightarrow_{\mathcal{G}'} w$ con $X \rightarrow w \in P'$. Pueden ocurrir dos casos:

1. Si $X \rightarrow w \in P$, entonces $X \Rightarrow_{\mathcal{G}} w$ y $w \in \mathcal{L}_{\mathcal{G}}(X)$.
2. Si $X \rightarrow w \notin P$, por construcción de $\mathcal{G}'$, $X \rightarrow \beta_0 X_1 \beta_1 \dots X_m \beta_m \in P$ con cada $X_j \in V_{<\infty}$ y $\beta_0 X_1 \beta_1 \dots X_m \beta_m \Rightarrow_{\mathcal{G}}^* w$. Por lo tanto $X \Rightarrow_{\mathcal{G}}^* w$ y $w \in \mathcal{L}_{\mathcal{G}}(X)$.

Inducción. Suponemos que para todo $w \in \mathcal{L}_{\mathcal{G}'}(X)$ tal que $X \Rightarrow_{\mathcal{G}'}^* w$ con una derivación de largo menor o igual a n , entonces se cumple que $w \in \mathcal{L}_{\mathcal{G}}(X)$. Demostraremos que para todo $w \in \mathcal{L}_{\mathcal{G}'}(X)$ tal que $X \Rightarrow_{\mathcal{G}'}^* w$ con una derivación de largo igual a $n+1$, entonces $w \in \mathcal{L}_{\mathcal{G}}(X)$.

Sea $w \in \mathcal{L}_{\mathcal{G}'}(X)$ tal que $X \Rightarrow_{\mathcal{G}'}^* w$ con una derivación de largo igual a $n+1$. Entonces $X \Rightarrow_{\mathcal{G}'} \beta \Rightarrow_{\mathcal{G}'}^* w$ con $\beta \in \{V_{\infty} \cup \Sigma\}$ y $\beta \Rightarrow_{\mathcal{G}'}^* w$ es una derivación de largo n . Aquí se pueden dar dos casos:

1. Si $X \rightarrow \beta \in P$ entonces $X \Rightarrow_{\mathcal{G}} \beta$. Por hipótesis de inducción $X \Rightarrow_{\mathcal{G}} \beta \Rightarrow_{\mathcal{G}}^* w$ y $w \in \mathcal{L}_{\mathcal{G}}(X)$.
2. Si $X \rightarrow \beta \notin P$, por construcción de $\mathcal{G}'$, $X \rightarrow \beta_0 X_1 \beta_1 \dots X_m \beta_m \in P$ con cada $X_j \in V_{<\infty}$ y $\beta_0 X_1 \beta_1 \dots X_m \beta_m \Rightarrow_{\mathcal{G}}^* \beta$. Por hipótesis de inducción $\beta \Rightarrow_{\mathcal{G}}^* w$, por lo tanto $X \Rightarrow_{\mathcal{G}}^* w$ y $w \in \mathcal{L}_{\mathcal{G}}(X)$.

Finalmente, concluimos que $\mathcal{L}_{\mathcal{G}'}(X) \subseteq \mathcal{L}_{\mathcal{G}}(X)$.

De los dos puntos anteriores se concluye que $\mathcal{L}_{\mathcal{G}'}(X) = \mathcal{L}_{\mathcal{G}}(X)$. Más aún, sabemos que para todo $X \in V_{\infty}$ se tiene que $\mathcal{L}(X)$ es infinito, y $\mathcal{L}(\mathcal{G}) = \mathcal{L}_{\mathcal{G}}(S) = \mathcal{L}_{\mathcal{G}'}(S) = \mathcal{L}(\mathcal{G}')$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 punto)** Por la construcción de la gramática.
- **(0.25 puntos)** Por la demostración de que para toda variable perteneciente a la nueva gramática, su lenguaje es infinito.
- **(0.25 puntos)** Por demostrar que el lenguaje de la nueva gramática es igual al lenguaje de la gramática original.



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2° 2021

INTERROGACION 2

Preguntas en blanco: Preguntas entregadas en blanco se evaluarán con un 1.5.

Pregunta 1

Considere el siguiente lenguaje L sobre el alfabeto $\Sigma = \{a, b\}$:

$$L = \{a^i b^j a^k \mid j = i + k\}$$

1. Presente una gramática libre de contexto $\mathcal{G}$ tal que $\mathcal{L}(\mathcal{G}) = L$.
2. Demuestre que su gramática $\mathcal{G}$ del punto anterior es correcta, esto es, demuestre que $\mathcal{L}(\mathcal{G}) = L$.

Pregunta 2

Para esta pregunta considere transductores de la forma $\mathcal{T} = (Q, \Sigma, \Omega, \Delta, I, F)$ con $\Delta \subseteq Q \times \Sigma \times (\Omega \cup \{\epsilon\}) \times Q$, esto es, los transductores NO tienen transiciones de lectura con ϵ . Considere el siguiente problema:

PROBLEMA: Evaluación de transductor
INPUT: Un transductor $\mathcal{T}$ y una palabra u
OUTPUT: Una palabra $v \in \Omega^*$ cualquiera tal que $v \in \llbracket \mathcal{T} \rrbracket(u)$

Escriba un algoritmo para el problema anterior. Su algoritmo debe tomar tiempo $\mathcal{O}(|\mathcal{T}| \cdot |u|)$.

Pregunta 3

Para una gramática libre de contexto $\mathcal{G} = (V, \Sigma, P, S)$ decimos que $\mathcal{G}$ tiene un *loop* si existe una variable $X \in V$ tal que $X \xRightarrow{*} \alpha X \beta$ para algún $\alpha, \beta \in (V \cup \Sigma)^*$. Demuestre que si $\mathcal{G}$ no tiene un loop, entonces $\mathcal{L}(\mathcal{G})$ es un lenguaje regular.



PAUTA INTERROGACIÓN 2

Pregunta 1

Pregunta 1.1

Una posible solución para esta pregunta es tomar en cuenta la siguiente propiedad:

$$i + k = j \text{ si y solo si } a^i b^j a^k = a^i b^i b^k a^k$$

Así, se propone la siguiente gramática libre de contexto:

$$\begin{aligned} S &\longrightarrow AB \\ A &\longrightarrow aAb | \epsilon \\ B &\longrightarrow bBa | \epsilon \end{aligned}$$

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1 punto) Por definir la propiedad.
- (1.5 puntos) Por definir la gramática libre de contexto.
- (0.5 punto) Por considerar los casos bordes.

Pregunta 1.2

Se demuestra que la gramática libre de contexto definida es correcta mediante la demostración de $\mathcal{L}(\mathcal{G}) = L$.

Por demostrar: $L \subseteq \mathcal{L}(\mathcal{G})$.

Sea $a^i b^j a^k \in L$. Luego, como $i + k = j$, entonces $a^i b^j a^k = a^i b^i b^k a^k$.

Ahora, a partir de la CFG propuesta, se tiene que:

$$\begin{aligned} A &\xRightarrow{*} a^i b^i \\ B &\xRightarrow{*} b^k a^k \end{aligned}$$

De esta manera, existe la siguiente derivación:

$$S \Rightarrow AB \xRightarrow{*} a^i b^i B \xRightarrow{*} a^i b^i b^k a^k$$

Y como $a^i b^j a^k = a^i b^i b^k a^k$, entonces $a^i b^j a^k \in \mathcal{L}(\mathcal{G})$.

Por demostrar: $\mathcal{L}(\mathcal{G}) \subseteq L$.

Sea $w \in \mathcal{L}(\mathcal{G})$, por lo que existe una derivación de la forma:

$$S \xRightarrow{*} w$$

Se analiza la forma que tiene esta derivación. En primer lugar, desde S, en un paso, solo hay una posible derivación:

$$S \Rightarrow AB$$

Luego, la derivación completa por la izquierda debe tener la siguiente forma:

$$S \Rightarrow AB \xRightarrow{lm} aAbB \xRightarrow{lm} aaAbB \xRightarrow{lm} a^i Ab^i B \xRightarrow{lm} a^i b^i B \xRightarrow{lm} a^i b^i bBa \xRightarrow{lm} a^i b^i b^k Ba^k \xRightarrow{lm} a^i b^i b^k a^k$$

Así, como $i + k = j$, entonces $w = a^i b^i b^k a^k = a^i b^j a^k$, por lo que $w \in L$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1.5 puntos) Por demostrar que $L \subseteq \mathcal{L}(\mathcal{G})$.
- (1.5 puntos) Por demostrar que $\mathcal{L}(\mathcal{G}) \subseteq L$.

Pregunta 2

Modificando el algoritmo visto en clases para la evaluación de NFA *on-the-fly*, tenemos que para un transductor $\mathcal{T} = (Q, \Sigma, \Omega, \Delta, I, F)$ tal que $\Delta \subseteq Q \times \Sigma \times (\Omega \cup \{\varepsilon\}) \times Q$ tenemos:

```

Function Eval-Transducer( $\mathcal{T}, w = a_1 \dots a_n$ )
   $S := \text{hash}(Q, \text{null})$ 
  foreach  $q \in I$  do
     $S[q] := \varepsilon$ 
  for  $i := 1$  to  $n$  do
     $S_{old} := S$ 
     $S := \text{hash}(Q, \text{null})$ 
    foreach  $p \in Q$  do
      if  $S_{old}[p] \neq \text{null}$  then
        foreach  $(p, a_i, b, q) \in \Delta$  do
          if  $S[q] = \text{null}$  then
             $S[q] = S_{old}[p] \cdot b$ 
    foreach  $p \in F$  do
      if  $S[q] \neq \text{null}$  then
        return  $S[q]$ 
  return null

```

La idea del algoritmo es llevar una tabla de hash con las palabras escritas por las ejecuciones actuales que se encuentran en cada estado, e ir completándola según nos indican las transiciones del transductor.

Estas tablas comienzan con un valor *null* **distinto de epsilon**, indicando que no hay ninguna ejecución en proceso, y a continuación se inicializan con ε las entradas de los estados iniciales, indicando que hay una ejecución activa, pero que no ha escrito nada aún. Teniendo lo anterior, se recorre la palabra letra por letra, visitando los estados que tengan alguna ejecución activa ($S_{old}[p] \neq \text{null}$), se revisan todas las transiciones salientes de este estado (para la letra actual) y en caso de encontrar una transición válida se concatena

la letra que indica la transición a la palabra asociada al estado de llegada. Cabe señalar que solamente se concatena esta letra si es la primera transición válida que encontramos ($S[q] \neq null$), pues buscamos que el algoritmo retorne solamente una palabra, por lo que no es necesario llevar registro de todas las posibles.

Finalmente, luego de haber leído la palabra completa, visitamos todos los estados finales, retornando la palabra escrita por el transductor en caso de encontrar alguna y retornando *null* en caso de que ninguna ejecución llegue a un estado final.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por llevar una tabla de hash o similar para las ejecuciones (en lugar de un conjunto).
- **(1 punto)** Por crear la tabla con nulos e inicializar con ε para las variables iniciales.
- **(1.5 puntos)** Por iterar sobre los estados que llevan alguna ejecución (no nulos).
- **(1.5 puntos)** Por seguir las transiciones y actualizar la tabla correctamente.
- **(1 punto)** Por retornar correctamente el output.
 - **(0.5 punto)** Por retornar una palabra escrita por el transductor.
 - **(0.5 punto)** Por retornar *null* (o similar) cuando no hay output.

Pregunta 3

Por demostrar que si una gramática libre de contexto no tiene loops, entonces define un lenguaje regular. Sea $\mathcal{G} = (V, \Sigma, P, S)$ una CFG sin loops y $G_{\mathcal{G}} = (V, E)$ un grafo tal que $|V| = n$ y $(X, Y) \in E$ si, y solo si, $X \rightarrow \alpha Y \beta \in P$.

Por demostrar que si $\mathcal{G}$ no tiene loops, entonces $G_{\mathcal{G}}$ tampoco. Por contrapositivo, si $G_{\mathcal{G}}$ tiene loops, entonces existe un camino $X = X_1 \rightarrow X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_m = X$. Luego, existen las producciones $X_1 \rightarrow \alpha_2 X_2 \beta_2$, $X_2 \rightarrow \alpha_3 X_3 \beta_3, \dots, X_{m-1} \rightarrow \alpha_m X_m \beta_m$. Componiéndolas se obtiene $X \xRightarrow{\pm} \alpha X \beta$.

Como $G_{\mathcal{G}}$ no tiene loops, entonces define un grafo dirigido acíclico (DAG). Así, se puede definir la secuencia $X_1, X_2, \dots, X_n$ de las variables en V ordenadas topológicamente tal que, con $i < j \leq n$, desde X_i no se puede llegar a X_j (al revés de la definición convencional).

Sea $\mathcal{G}_X = (V, \Sigma, P, X)$. Por demostrar, utilizando inducción fuerte, que $\forall i \in \{1, n\}. \mathcal{L}(\mathcal{G}_{X_i})$ es finito.

- **Caso base:** Desde X_1 no se puede llegar a ninguna variable, es decir, todas las son de la forma $X \rightarrow w$ con $w \in \Sigma^*$. Como P es finito entonces, $\mathcal{L}(\mathcal{G}_{X_1})$ también lo es.
- **Caso inductivo:** Sea X_i una variable en la secuencia ordenada. Por hipótesis de inducción $\forall k < i. \mathcal{L}(\mathcal{G}_{X_k})$ es finito. Gracias al orden topológico y que $\mathcal{G}$ no tiene loops, entonces las producciones de X_i son de la forma $X_i \rightarrow \alpha$ con $\alpha \in (\Sigma \cup \{X_1, \dots, X_{i-1}\})^*$. Luego, considerando las producciones de X_i de la forma $X_i \rightarrow \alpha_1 \alpha_2 \dots \alpha_l$, si se reemplaza cada aparición de X_k con $k < i$ en cada $\alpha_1, \alpha_2, \dots, \alpha_l$ por toda posible palabra en $\mathcal{L}(\mathcal{G}_{X_k})$ se logra definir $\mathcal{L}(\mathcal{G}_{X_i})$ a través de producciones de la forma $X_i \rightarrow w$ con $w \in \Sigma^*$. Como los $\mathcal{L}(\mathcal{G}_{X_k})$ son finitos y las producciones originales de X_i son finitas, entonces las producciones con reemplazo son finitas. Por lo tanto, $\mathcal{L}(\mathcal{G}_{X_i})$ es finito.

Finalmente, $\mathcal{L}(\mathcal{G}_{X_S}) = \mathcal{L}(\mathcal{G})$ es finito y todo lenguaje finito es regular.

Dado lo anterior, la distribución de puntaje es la siguiente: .

- **(1 punto)** Por plantear la demostración de que el grafo de una CFG sin ciclos tampoco los tiene.
- **(1 punto)** Por la demostración de que el grafo de una CFG sin ciclos tampoco los tiene.

- (1 punto) Por ordenar las variables para facilitar la inducción.
- (1 punto) Por el caso base.
- (1 punto) Por el caso inductivo.
- (1 punto) Por la conclusión y mencionar que todo lenguaje finito es regular.



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2° 2022

INTERROGACION 2

Preguntas en blanco: Preguntas o ítems entregados en blanco se evaluarán con 0.5 de 6 puntos.

Pregunta 1

Demuestre que existen transductores $\mathcal{T}_1$ y $\mathcal{T}_2$ sobre alfabetos Σ y Ω tal que la relación $\llbracket \mathcal{T}_1 \rrbracket \cap \llbracket \mathcal{T}_2 \rrbracket = \{(u, v) \in \Sigma^* \times \Omega^* \mid (u, v) \in \llbracket \mathcal{T}_1 \rrbracket \wedge (u, v) \in \llbracket \mathcal{T}_2 \rrbracket\}$ no es una relación racional.

Pregunta 2

Considere los siguientes lenguajes sobre el alfabeto $\{a, b, c, d\}$:

$$\begin{aligned} L_1 &= \{a^i b^j c^k d^\ell \mid 2i = \ell \wedge 3j = k\} \\ L_2 &= \{a^i b^j c^k d^\ell \mid 2i = k \wedge 3j = \ell\} \end{aligned}$$

Uno de los lenguajes es libre de contexto y el otro no. ¿Cuál es cuál? Diga cuál lenguaje es libre de contexto y cuál no, y demuestre ambas afirmaciones.

Pregunta 3

Sean Σ y Ω alfabetos finitos. Una función $h : \Sigma^* \rightarrow \Omega^*$ *respeto la concatenación* si para todo $u, v \in \Sigma^*$ se cumple que $h(u \cdot v) = h(u) \cdot h(v)$. Para $L \subseteq \Sigma^*$, se define el lenguaje $h(L) = \{h(w) \mid w \in L\}$.

Demuestre que, si $L \subseteq \Sigma^*$ es un lenguaje libre de contexto y $h : \Sigma^* \rightarrow \Omega^*$ es una función que respeta la concatenación, entonces $h(L)$ es un lenguaje libre de contexto.

Pregunta 4

Considere el siguiente problema:

Problema: #SUFFIX-DFA
Input: Un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ y $w = a_1 \dots a_n \in \Sigma^*$.
Output: $|\{i \in \{1, \dots, n\} \mid a_i \dots a_n \in \mathcal{L}(\mathcal{A})\}|$.

Esto es, el problema #SUFFIX-DFA consiste en, dado un autómata finito determinista $\mathcal{A}$ y dado una palabra w , contar todos los sufijos de w que son aceptados por $\mathcal{A}$.

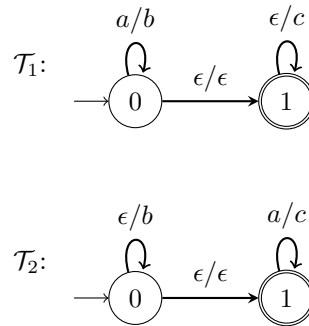
Escriba un algoritmo que resuelva #SUFFIX-DFA en tiempo $\mathcal{O}(|\mathcal{A}| \cdot |w|)$ donde $|\mathcal{A}|$ es el número de estados y transiciones de $\mathcal{A}$. Demuestre la correctitud de su algoritmo.



PAUTA INTERROGACIÓN 2

Pregunta 1

Una posible solución para esta pregunta es construyendo los siguientes transductores $\mathcal{T}_1$ y $\mathcal{T}_2$, con $\Sigma = \{a\}$ su alfabeto de input y $\Omega = \{b, c\}$ su alfabeto de output:



Es fácil ver que: $[\mathcal{T}_1] = \{(a^n, b^n c^m) \mid n, m \geq 0\}$, y que $[\mathcal{T}_2] = \{(a^n, b^m c^n) \mid n, m \geq 0\}$. Luego, tenemos que $[\mathcal{T}_1] \cap [\mathcal{T}_2] = \{(a^n, b^n c^n) \mid n \geq 0\}$.

Por contradicción, supongamos que $[\mathcal{T}_1] \cap [\mathcal{T}_2]$ es una relación racional. Se tiene que $\pi_2([\mathcal{T}_1] \cap [\mathcal{T}_2]) = \{b^n c^n \mid n \geq 0\}$, y notamos que **no** es un lenguaje regular. Por teorema visto en clases, sabemos que las proyecciones de las relaciones racionales son lenguajes regulares. Luego, llegamos a una contradicción, y queda demostrado que $[\mathcal{T}_1] \cap [\mathcal{T}_2]$ no es una relación racional.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1.5 puntos)** Por construir $\mathcal{T}_1$
- **(1.5 puntos)** Por construir $\mathcal{T}_2$
- **(1 puntos)** Por expresar la relación definida por $[\mathcal{T}_1] \cap [\mathcal{T}_2]$
- **(1 puntos)** Por identificar que $\pi_2([\mathcal{T}_1] \cap [\mathcal{T}_2])$ no es un lenguaje regular
- **(1 puntos)** Por concluir que $[\mathcal{T}_1] \cap [\mathcal{T}_2]$ no es una relación racional

Nota: Es correcto proponer otros transductores $\mathcal{T}_1$ y $\mathcal{T}_2$, mientras cumplan que $\pi_1([\mathcal{T}_1] \cap [\mathcal{T}_2])$ no sea un lenguaje regular, o que $\pi_2([\mathcal{T}_1] \cap [\mathcal{T}_2])$ no sea un lenguaje regular.

Pregunta 2

Sean los lenguajes L_1 y L_2 definidos de la siguiente forma:

$$L_1 = \{a^i b^j c^k d^l \mid 2i = l \wedge 3j = k\}$$

$$L_2 = \{a^i b^j c^k d^l \mid 2i = k \wedge 3j = l\}$$

En este caso, L_1 es un lenguaje libre de contexto mientras que L_2 no lo es. A continuación se demuestran ambas afirmaciones.

1) L_1 es LLC:

Para demostrar que L_1 es libre de contexto, procedemos a diseñar una gramática para el lenguaje y explicar su correctitud. La gramática generada es de la forma:

$$\begin{aligned} \mathcal{G} : X &\rightarrow aXdd \mid Y \\ Y &\rightarrow bYccc \mid \epsilon \end{aligned}$$

La gramática se construye de forma recursiva desde los extremos hasta el centro. Nos aseguramos con las derivaciones de X de agregar las letras a y d en los extremos de la palabra correspondientes, procurando seguir la proporción pedida. Luego, al cambiar de variable a Y , agregamos en el centro las letras b y c que también siguen su propia proporción pedida. Finalmente, solo es posible seguir agregando b y c , o terminar la palabra.

2) L_2 no es LLC:

Para demostrar que L_2 no es LLC, usaremos el contrapositivo del lema de bombeo para lenguajes libres de contexto. Sea un $N > 0$, definimos la siguiente palabra z perteneciente al lenguaje:

$$z = a^N b^N c^{2N} d^{3N}$$

Y sea la siguiente descomposición cualquiera para z :

$$z = uvwxy$$

Con $vx \neq \epsilon$ y $|vwx| \leq N$. Según la estructura que tome dicha descomposición, podemos tener la siguientes tres posibilidades:

1. $vwx \in a^*b^*$. En este caso, sea $i = 0$ de forma que la palabra queda como:

$$uv^0wx^0y = a^{N_1}b^{N_2}c^{2N}d^{3N} \quad (1)$$

Lo que fuerza a que $N_1 < N$ ó $N_2 < N$, por lo que la palabra nueva no pertenece a L_2 .

2. $vwx \in b^*c^*$. Es este caso, sea $i = 0$:

$$uv^0wx^0y = a^Nb^{N_2}c^{N_3}d^{3N} \quad (2)$$

Lo que fuerza a que $N_2 < N$ ó $N_3 < 2N$, por lo que la palabra nueva no pertenece a L_2 .

3. $vwx \in c^*d^*$. Es este caso, sea $i = 0$:

$$uv^0wx^0y = a^Nb^Nc^{N_3}d^{N_4} \quad (3)$$

Lo que fuerza a que $N_3 < 2N$ ó $N_4 < 3N$, por lo que la palabra nueva no pertenece a L_2 .

Al ser analizadas todas las posibilidades de descomposición para la palabra z , se demuestra que L_2 no es un lenguaje libre de contexto.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** Por indicar que L_1 es LLC y que L_2 no lo es.
- **(2 puntos)** Por generar la gramática correspondiente para L_1 .
- **(0.75 puntos)** Por explicar la correctitud de la gramática generada.
- **(1 punto)** Por seleccionar una palabra z adecuada para el lema de bombeo de L_2 .
- **(0.25 puntos)** Por seleccionar la descomposición $z = uvwxy$ adecuada, junto a sus restricciones.
- **(1.5 puntos)** Por analizar correctamente todas las posibilidades de la descomposición anterior de z .

Pregunta 3

Pregunta 3

Sea L un lenguaje libre de contexto y el lenguaje $h(L)$ definido como:

$$h(L) = \{h(w) \mid w \in L\}.$$

Sea $\mathcal{G} = \{V, \Sigma, P, S\}$ en forma normal de Chomsky (CNF) tal que $L(\mathcal{G}) = L \setminus \{\epsilon\}$, por teorema visto en clases. Se define $h(\mathcal{G}) = (V \cup \{S'\}, \Omega, P', S')$ tal que:

$$\begin{aligned} P' = & \{X \rightarrow YZ \mid X \rightarrow YZ \in P\} \\ & \cup \{X \rightarrow h(a) \mid X \rightarrow a \in P\} \\ & \cup \{S' \rightarrow S\} \\ & \cup \{S' \rightarrow \epsilon \mid \epsilon \in L\} \end{aligned}$$

Por demostrar: $h(L) = L(h(\mathcal{G}))$

($\subseteq$) Si $u \in h(L)$, sea $w \in L$ tal que $h(w) = u$. Como $w \in L(\mathcal{G})$ sea T un árbol de derivación de $\mathcal{G}$ sobre w .

Sea $h(T)$ el árbol T donde reemplazando cada hoja $a \in \Sigma$ por $h(a)$. Entonces, por construcción es fácil ver que $h(T)$ es árbol de derivación de $h(\mathcal{G})$ con $h(w)$.

Si $u = \epsilon \Rightarrow S \Rightarrow \epsilon$ y $\epsilon \in L(h(\mathcal{G}))$.

($\supseteq$) Sea $u \in L(h(\mathcal{G}))$ y sea T árbol de derivación de $h(\mathcal{G})$ sobre u . Sea T' el árbol tal que para cada nodo de la forma $X(b_1 \dots b_k)$ lo reemplazamos por $X(a)$ con $a \in \Sigma$ tal que $h(a) = b_1 \dots b_k$. Sea w la palabra en las hojas de T' :

- $h(w) = u$
- T' es árbol de derivación de $\mathcal{G}$ sobre w

Por lo tanto, $u \in h(L)$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** Por manejar correctamente la función h (respeta la concatenación)
- **(1.5 puntos)** Por conservar las derivaciones del tipo $X \rightarrow YZ$
- **(1.5 puntos)** Por conservar las derivaciones del tipo $X \rightarrow a$
- **(0.5 puntos)** Por incluir el ϵ en la gramática
- **(2 puntos)** Por demostrar correctamente cada lado (1 punto c/u)

Pregunta 4

Una posible solución para esta pregunta es modificar el algoritmo *eval-NFAonthefly* visto en clases. Específicamente, en lugar de llevar dos conjuntos S y S_{old} para llevar los estados en los que van las ejecuciones al leer la i -ésima letra de w , se llevará un **contador** de los sufijos que hay en cada estado al leer la i -ésima letra. En el nuevo algoritmo, S y S_{old} corresponderán a arreglos de tamaño $|Q| = m$, donde cada entrada del arreglo le corresponde a un estado $j \in Q$, y esta llevará la cuenta de sufijos de la palabra que al ejecutar $\mathcal{A}$ llegan al estado j .

Sin pérdida de generalidad, suponga que el autómata $\mathcal{A}$ tiene como conjunto de estados $Q = \{0, 1, \dots, m-1\}$. Si los m estados de $\mathcal{A}$ no son números, estos pueden ser ordenados arbitrariamente y ser asignados a números. También suponemos, sin pérdida de generalidad, que $\mathcal{A}$ es un autómata sin ϵ -transiciones. Luego, escribimos el siguiente algoritmo:

```
#SUFFIX-DFA( $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ ,  $w = a_1 a_2 \dots a_n$ )
 $S, S_{old} \leftarrow$  Arreglos de largo  $m$ , y entradas con valor 0
for  $i = 1$  to  $n$  do
     $S_{old} \leftarrow S$ 
     $S \leftarrow [0]$  // arreglo de 0s
     $S_{old}[0] \leftarrow S_{old}[0] + 1$ 
    for  $j = 0$  to  $m - 1$  do
         $S[\delta(j, a_i)] \leftarrow S[\delta(j, a_i)] + S_{old}[j]$ 
    end for
end for
 $C \leftarrow 0$ 
for  $j \in F$  do
     $C \leftarrow C + S[j]$ 
end for
return  $C$ 
```

Para la demostración de la correctitud del algoritmo, usaremos inducción sobre el tamaño de w , para mostrar que en toda iteración el arreglo S lleva correctamente la cantidad de sufijos de $a_1 \dots a_{i-1}$ que llegan a cada estado.

CB: Antes de empezar a leer letras de w , se da que el estado inicial de $\mathcal{A}$ no tiene sufijos y no se ha pasado por ningún otro estado. Esto se ve representado correctamente al inicializar las entradas de S como 0.

HI: Suponemos que luego de procesar $a_1 \dots a_i$ se cumple que, para todo estado j , $S[j]$ es igual al número de sufijos de $\mathcal{A}$ sobre $a_1 \dots a_i$ que llegan al estado j .

TI: Por HI, S lleva correctamente la cantidad de ejecuciones por estado en la i -ésima iteración. En la iteración $i + 1$, se asigna a S_{old} los valores de S , y S se reinicializa con valores 0. Luego, por cada transición (j_1, a_{i+1}, j_2) del autómata, a $S[j_2]$ se le suma la cantidad de sufijos de j_1 . Al realizar esto sobre todas las transiciones del autómata, se cumple que $S[j]$ es igual al número de sufijos de $a_1 \dots a_{i+1}$ al ejecutar $\mathcal{A}$ que llegan a $j \in Q$.

Ya demostrado que el S lleva correctamente la cantidad de sufijos que llegan a cada estado para cualquier largo de w . La cantidad de sufijos de w que son aceptados por $\mathcal{A}$ será la suma de la cantidad de sufijos de las ejecuciones que llegaron a estados finales del autómata después de leer las n letras de w . Esto se guarda en la variable C al final del algoritmo, y se retorna dicho valor. Luego, el algoritmo propuesto retorna correctamente la cantidad de sufijos de w que son aceptados por $\mathcal{A}$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 puntos)** Por llevar un contador para cada estado.
- **(1.5 puntos)** Por actualizar correctamente S y S_{old}
- **(1.5 puntos)** Por actualizar correctamente $S[\delta(j, a_i)]$
- **(1 puntos)** Por contar los sufijos.
- **(1 puntos)** Por demostrar correctitud.

Nota 1: No era correcto si:

- el algoritmo tiene un tiempo de ejecución mayor al pedido, por tanto se asignaron 0 puntos a esta solución.
- si solo se escribe el algoritmo NFA-onthefly⁴ sin cambios, por tanto se asignaron 0 puntos a esta solución.

Nota 2: Si el autómata se “invierte” para leer la palabra de derecha a izquierda, había que tener cuidado en darse cuenta que al invertir el autómata queda no-determinista. Para esto debían demostrar que el autómata invertido acepta el mismo lenguaje y después aprovechar el autómata invertido correctamente. En muchos casos esto no ocurrió y por tanto se asignaron 0 puntos a esta solución. Se recomienda consultar al profesor, si su solución esta en este caso.



Pauta Interrogación 2

Pregunta 1

Usando el teorema de Myhill-Nerode, demuestre que $\mathcal{L}(\mathcal{A})$ es regular para todo 2DFA $\mathcal{A}$.

Solución

Sea $\mathcal{A} = (Q, \Sigma, \vdash, \dashv, \delta, q_0, q_f)$ un 2DFA arbitrario. Para un input $u \in \Sigma^*$ cualquiera, definiremos la función de comportamiento T_u como se hizo en clases:

$$T_u : Q \cup \{\bullet\} \longrightarrow Q \cup \{\perp\}$$

tal que

- $T_u(p) = q$ si y solo si desde $(p, |u|)$ cruza en la config. $(q, |u| + 1)$.
- $T_u(p) = \perp$ si y solo si desde $(p, |u|)$ nunca cruza de u .
- $T_u(\bullet) = q$ si y solo si desde $(q_0, 0)$ cruza por primera vez con $(q, |u| + 1)$.
- $T_u(\bullet) = \perp$ si y solo si desde $(q_0, 0)$ nunca cruza de u .

Consideremos la relación $\equiv_T$ tal que se cumpla:

$$u \equiv_T v \quad \text{ssi} \quad T_u = T_v$$

Es fácil ver que $\equiv_T$ es relación de equivalencia sobre Σ^* (es refleja, simétrica y transitiva). Por Teorema de Myhill-Nerode, demostrar que $\mathcal{L}(\mathcal{A})$ es lenguaje regular es equivalente a demostrar que $\equiv_T$ es una relación de Myhill-Nerode.

Por demostrar: $\equiv_T$ es relación de Myhill-Nerode

Debemos verificar que $\equiv_T$ cumpla las siguientes 3 propiedades:

1. $\equiv_T$ es congruencia por la derecha
2. $\equiv_T$ refine $\mathcal{L}(\mathcal{A})$
3. La cantidad de clases de equivalencia de $\equiv_T$ es finita

1. $\equiv_T$ es congruencia por la derecha

Esto implica demostrar que

$$u \equiv_T v \implies u \cdot w \equiv_T v \cdot w \quad \forall w \in \Sigma^*$$

Ahora, tenemos que $u \equiv_T v \implies T_u = T_v = T$. Si $\mathcal{A}$ parte en $(p, |uw| + 1)$ sobre uw tenemos dos posibilidades:

- (1) Se lee w y nunca se entra a u . En este caso $T_{uw}(p) = T_{vw}(p)$.
- (2) Se lee w y entra a u en un estado q . Luego, como $T_u = T_v$ entonces el comportamiento de $\mathcal{A}$ sobre u sería igual al comportamiento sobre v . Por lo que $T_{uw}(p) = T_{vw}(p)$.

Ocurrirá algo similar en el caso de $\bullet$.

- (1) Si $T_u(\bullet) = T_v(\bullet) = q$ entonces se cruza desde $(q_0, 0)$ por primera vez a la primera letra de w con estado q . Luego la lectura de uw y vw será igual en adelante, ya que su sufijo w es el mismo, por lo que $T_{uw}(\bullet) = T_{vw}(\bullet)$.
- (2) Si $T_u(\bullet) = T_v(\bullet) = \perp$, entonces nunca se cruza de u ni v . Por lo tanto, tampoco cruzará de uw ni vw . Luego, $T_{uw}(\bullet) = T_{vw}(\bullet) = \perp$.

2. $\equiv_T$ refine $\mathcal{L}(\mathcal{A})$

Esto implica demostrar que

$$u \equiv_T v \implies (u \in \mathcal{L}(\mathcal{A}) \Leftrightarrow v \in \mathcal{L}(\mathcal{A}))$$

Al ser u y v indistinguibles tendremos:

- (1) Si $T(\bullet) = \perp$ entonces $\mathcal{A}$ nunca sale a la derecha de u ni de v . Recordemos que para que una palabra u sea aceptada su ejecución debe terminar con $(q_f, |u| + 1)$. Por lo que tendremos que $u \notin \mathcal{L}(\mathcal{A})$ y $v \notin \mathcal{L}(\mathcal{A})$.
- (2) Si $T(\bullet) = p$ entonces nos encontramos con dos casos:
 - (a) $p = q_f$, luego $u \in \mathcal{L}(\mathcal{A})$ y $v \in \mathcal{L}(\mathcal{A})$, ya que llegaríamos a una ejecución de aceptación.
 - (b) $p \neq q_f$ y $\delta(p, \neg) = (q, \leftarrow)$, es decir, la ejecución podría continuar en la dirección opuesta. Por un lado, si $T(q) = \perp$ se repite el caso (1). Por otro, si $T(q) \in Q$, se repite el caso (2).

3. Cantidad finita de clases de equivalencia

Se cumple que

$$|\Sigma^* / \equiv_T| \leq |\{T : Q \cup \{\bullet\} \longrightarrow Q \cup \{\perp\}\}|$$

$$|\Sigma^* / \equiv_T| \leq (|Q| + 1)^{|Q|+1}$$

por lo que la cantidad de clases de equivalencia de $\equiv_T$ está acotada por un límite superior finito. Finalmente podemos concluir que, por Teorema de Myhill-Nerode, $\mathcal{L}(\mathcal{A})$ es un lenguaje regular.

Distribución de puntajes

- **(0.5 ptos.)** Por definir correctamente la función de comportamiento T_u .
- **(2 ptos.)** Por definir correctamente la relación de equivalencia $\equiv_T$.
- **(1 pto.)** Por demostrar que $\equiv_T$ es congruencia por la derecha.
- **(1 pto.)** Por demostrar que $\equiv_T$ refine el lenguaje.
- **(1 pto.)** Por demostrar que $\equiv_T$ tiene cantidad finita de clases de equivalencia.
- **(0.5 ptos.)** Por usar el Teorema de Myhill-Nerode correctamente para demostrar lo pedido.

Pregunta 2

Demuestre una gramática libre de contexto para cada uno de los siguientes lenguajes.

1. $L_1 = \{a^i b^j c^k \mid i \neq j \vee j \neq k\}$
2. $L_2 = \{a^i b^j c^k \mid j = i + k\}$

Para cada una de las gramáticas, explique su correctitud.

Solución

1. $L_1 = \{a^i b^j c^k \mid i \neq j \vee j \neq k\}$

Se define la siguiente gramática libre de contexto:

$$\begin{aligned} S &\rightarrow X_L \mid X_R \\ X_L &\rightarrow Y_L \cdot C \\ Y_L &\rightarrow aY_L b \mid aA \mid bB \\ X_R &\rightarrow A \cdot Y_R \\ Y_R &\rightarrow bY_R c \mid bB \mid cC \\ A &\rightarrow aA \mid \varepsilon \\ B &\rightarrow bB \mid \varepsilon \\ C &\rightarrow cC \mid \varepsilon \end{aligned}$$

Desglosemos la solución. Primero tenemos S que define tanto a X_L como a X_R . X_L representa cuando en la palabra se cumple que $i \neq j$, mientras que X_R cubre el otro caso $j \neq k$. Antes de ver X_L vemos los últimos tres casos A, B y C . Cada uno de estos permite generar la cantidad de veces que se quiera

la letra específica o parar de construir la palabra utilizando la palabra vacía ε . Ahora veamos Y_L . Esta parte genera aY_Lb o una cantidad de a 's o b 's cualquiera (usando A o B mencionadas anteriormente). Como vemos, Y_L se encarga de que cada vez que se quieran generar nuevas letras en la palabra se generen la misma cantidad de a 's y de b 's y solo se puede terminar la palabra si es que se escoge luego aA o bB . En esto último esta la clave, dado que con esto nos aseguramos de que por lo menos hay una a o una b más (es decir el caso $i \neq j$: $j < i$ si se elige aA y $i < j$ si se elige bB). Finalmente vemos X_L , este se encarga de generar Y_L que como sabemos es una palabra con $i \neq j$ junto a una cantidad arbitraria de c 's. Debido a que X_L solo evalúa el caso en que $i \neq j$ entonces la cantidad de c 's que tenga es irrelevante. Analogamente tenemos lo mismo que para X_R y Y_R , que evalúa el otro caso posible $j \neq k$.

- **(1 pto.)** Por establecer la diferencia entre X_L y X_R .
- **(1 pto.)** Por establecer el caso aY_Lb .
- **(1 pto.)** Por establecer el caso $aA \mid bB$ en Y_L .
- **(1 pto.)** Por establecer el caso bY_Rc .
- **(1 pto.)** Por establecer el caso $bB \mid cC$ en Y_R .
- **(1 pto.)** Por establecer los casos A, B y C .

2. $L_2 = \{a^i b^j c^k \mid j = i + k\}$

Se define la siguiente gramática libre de contexto:

$$\begin{aligned} S &\rightarrow X_L \cdot X_R \\ X_L &\rightarrow aX_Lb \mid \varepsilon \\ X_R &\rightarrow bX_Rc \mid \varepsilon \end{aligned}$$

Desglosemos la solución. Similar al inciso anterior S define a X_L y X_R , solamente que en este caso los concatena. El caso X_L se encarga de que cada vez que se genere una a también se genere una b . Analogamente X_R se encarga de que cada vez que se genera una c también se genere una b . Finalmente, como en X_L tenemos la misma cantidad de a 's que b 's (la cual es i) y en X_R tenemos la misma cantidad de b 's que c 's (la cual es k), entonces tenemos que la concatenación S cumple que $j = i + k$.

- **(2 ptos.)** Por establecer la concatenación entre X_L y X_R .
- **(2 ptos.)** Por establecer el caso X_L .
- **(2 ptos.)** Por establecer el caso X_R .

Problema 3

Sea $\mathcal{T} = (Q, \Sigma, \Omega, \Delta, I, F)$ un transductor. Decimos que $\mathcal{T}$ es *síncrono* si $\Delta \subseteq Q \times \Sigma \times \Omega \times Q$. Para un lenguaje $L \subseteq \Sigma^*$, se define el lenguaje $\llbracket \mathcal{T} \rrbracket(L) = \{v \in \Omega^* \mid \exists u \in L. (u, v) \in \llbracket \mathcal{T} \rrbracket\}$.

Demuestre que para todo transductor síncrono $\mathcal{T} = (Q, \Sigma, \Omega, \Delta, I, F)$ y para todo lenguaje regular $L \subseteq \Sigma^*$ se tiene que $\llbracket \mathcal{T} \rrbracket(L)$ es regular.

Solución 1

Sabemos que $\mathcal{L}$ es un lenguaje regular, por lo que existe un autómata finito determinista que lo define, sea $\mathcal{A} = (Q', \Sigma, \delta', q'_0, F')$ este autómata. Definimos $\mathcal{T}_{\mathcal{A}} = (Q_{\mathcal{A}}, \Omega, \Delta_{\mathcal{A}}, I_{\mathcal{A}}, F_{\mathcal{A}})$, donde:

- $Q_{\mathcal{A}} = Q \times Q'$.
- $\Delta_{\mathcal{A}} = \{(p, p'), b, (q, q') \mid \exists a \in \Sigma^* [\delta'(p', a) = q' \wedge (p, a, b, q) \in \Delta]\}$.
- $I_{\mathcal{A}} = I \times \{q'_0\}$.
- $F_{\mathcal{A}} = F \times F'$.

PD. $\llbracket \mathcal{T} \rrbracket(\mathcal{L}) = L(\mathcal{T}_{\mathcal{A}})$.

- $\llbracket \mathcal{T} \rrbracket(\mathcal{L}) \subseteq L(\mathcal{T}_{\mathcal{A}})$

Sea $v \in \llbracket \mathcal{T} \rrbracket(\mathcal{L})$, entonces existe $u \in \mathcal{L}$ tal que $(u, v) \in \mathcal{T}$. Como $\mathcal{T}$ es síncrono, entonces $|u| = |v| = n$. Sea $u = a_1 \dots a_n$ y $v = b_1 \dots b_n$. Como $u \in \mathcal{L}$, entonces existe una ejecución ρ' de aceptación en $\mathcal{A}$ tal que:

$$\rho' : p'_0 \xrightarrow{a_1} p'_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} p'_n$$

Como $(u, v) \in \mathcal{T}$, entonces existe una ejecución ρ en $\mathcal{T}$ de aceptación tal que:

$$\rho : (p_0, p'_0) \xrightarrow{(a_1, b_1)} (p_1, p'_1) \xrightarrow{(a_2, b_2)} \dots \xrightarrow{(a_n, b_n)} (p_n, p'_n)$$

Por ende, la secuencia ρ'' , definida como:

$$\rho'' : (p_0, p'_0) \xrightarrow{b_1} (p_1, p'_1) \xrightarrow{b_2} \dots \xrightarrow{b_n} (p_n, p'_n)$$

es una ejecución de aceptación en $\mathcal{T}_A$, dado que $(p_i, a_i, b_i, p_{i+1}) \in \Delta$ y $\delta'(p'_i, a_i) = p'_{i+1}$ para todo i , con $0 \leq i < n$, $(p_0, p'_0) = I_A$ y $(p_n, p'_n) \in F_A$, por lo que $v \in L(\mathcal{T}_A)$.

- $L(\mathcal{T}_A) \subseteq \llbracket \mathcal{T} \rrbracket(\mathcal{L})$

Sea $v = b_1 \dots b_n \in L(\mathcal{T}_A)$, entonces existe una ejecución ρ de aceptación en $\mathcal{T}_A$ tal que:

$$\rho : (p_0, p'_0) \xrightarrow{b_1} (p_1, p'_1) \xrightarrow{b_2} \dots \xrightarrow{b_n} (p_n, p'_n)$$

Como $\mathcal{T}$ es síncrono, entonces $|u| = |v| = n$. Sea $u = a_1 \dots a_n$. Como ρ es una ejecución de aceptación en $\mathcal{T}_A$, por construcción sabemos de dado que $((p_i, p'_i), b_i, (p_{i+1}, p'_{i+1})) \in \Delta_A$, entonces existe $a_i \in \Sigma^*$ tal que $\delta'(p'_i, a_i) = p'_{i+1}$ y $(p_i, a_i, b_i, p_{i+1}) \in \Delta$ para todo i , con $0 \leq i < n$. Por ende, existen ejecuciones ρ' y ρ'' en $\mathcal{A}$ y $\mathcal{T}$ respectivamente, tales que:

$$\begin{aligned} \rho' : p'_0 &\xrightarrow{a_1} p'_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} p'_n \\ \rho'' : (p_0, p'_0) &\xrightarrow{(a_1, b_1)} (p_1, p'_1) \xrightarrow{(a_2, b_2)} \dots \xrightarrow{(a_n, b_n)} (p_n, p'_n) \end{aligned}$$

Por ende, $(u, v) \in \mathcal{T}$ y $v \in \llbracket \mathcal{T} \rrbracket(\mathcal{L})$.

Distribución de puntaje

- **(1 pto.)** Por definir Q_A .
- **(2 ptos.)** Por definir Δ_A .
- **(1 pto.)** Por definir I_A y F_A .
- **(1 pto.)** Por demostrar que $\llbracket \mathcal{T} \rrbracket(\mathcal{L}) \subseteq L(\mathcal{T}_A)$.
- **(1 pto.)** Por demostrar que $L(\mathcal{T}_A) \subseteq \llbracket \mathcal{T} \rrbracket(\mathcal{L})$.

Solución 2

En clases se vio que si $\mathcal{T}_1$ y $\mathcal{T}_2$ son transductores, entonces $\llbracket \mathcal{T}_1 \rrbracket \cap \llbracket \mathcal{T}_2 \rrbracket$ no es necesariamente una relación racional. Sin embargo, si $\mathcal{T}_1$ y $\mathcal{T}_2$ son síncronos, entonces $\llbracket \mathcal{T}_1 \rrbracket \cap \llbracket \mathcal{T}_2 \rrbracket$ sí define una relación racional.

Sea $\mathcal{T}_i = (Q_i, \Sigma, \Omega, \Delta_i, I_i, F_i)$ para $i \in \{1, 2\}$. Definimos $\mathcal{T}_1 \times \mathcal{T}_2 = (Q^\times, \Sigma, \Omega, \Delta^\times, I^\times, F^\times)$, donde:

- $Q^\times = Q_1 \times Q_2$.
- $\Delta^\times = \{((p_1, p_2), a, b, (q_1, q_2)) \mid (p_1, a, b, q_1) \in \Delta_1 \wedge (p_2, a, b, q_2) \in \Delta_2\}$.
- $I^\times = I_1 \times I_2$.
- $F^\times = F_1 \times F_2$.

La demostración de esta afirmación es análoga a la demostración del producto de autómatas, por lo que se omitirá.

A continuación, dado que $\mathcal{L}$ es regular, existe un autómata finito no determinista $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ que lo define, es decir, $L(\mathcal{A}) = \mathcal{L}$.

Definimos $\mathcal{T}_A = (Q, \Sigma, \Omega, \Delta', I, F)$ como la $\mathcal{T}$ -extensión de $\mathcal{A}$, donde

$$\Delta' = \{(p, a, b, q) \mid (p, a, q) \in \Delta \wedge b \in \Omega\}$$

Es claro ver que $\mathcal{T}_A$ es síncrono y que $\llbracket \mathcal{T}_A \rrbracket = \{(u, v) \mid u \in \mathcal{L} \wedge v \in \Omega^* \wedge |u| = |v|\}$.

Así, como sabemos que $\mathcal{T}$ y $\mathcal{T}_A$ son síncronos, podemos notar que $\llbracket \mathcal{T} \rrbracket(\mathcal{L}) = \pi_2(\llbracket \mathcal{T} \rrbracket \cap \llbracket \mathcal{T}_A \rrbracket)$. Esto se debe a que $\llbracket \mathcal{T} \rrbracket \cap \llbracket \mathcal{T}_A \rrbracket$ es una relación racional, por lo que su proyección sobre el segundo componente será un lenguaje regular.

Distribución de puntaje

- (1 pto.) Por plantear la intersección de dos relaciones racionales.
- (1 pto.) Por definir $\mathcal{T}_1 \times \mathcal{T}_2$ junto a una breve explicación.
- (1 pto.) Por mencionar al autómata $\mathcal{A}$.
- (1 pto.) Por definir $\mathcal{T}_{\mathcal{A}}$.
- (2 ptos.) Por demostrar que $\llbracket \mathcal{T} \rrbracket(\mathcal{L})$ es regular.

Problema 4

Considere el siguiente problema:

Problema: LONGEST-SUFFIX-NFA
Input: Un NFA $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ y $w = a_1 \dots a_n \in \Sigma^*$
Output: $\min \{i \in \{1, \dots, n\} \mid a_i \dots a_n \in \mathcal{L}(\mathcal{A})\}$

Esto es, el problema LONGEST-SUFFIX-NFA consiste en, dado un autómata finito no-determinista $\mathcal{A}$ y una palabra $w = a_1 \dots a_n$, retornar la posición i del sufijo más largo $a_i \dots a_n$ que es aceptado por $\mathcal{A}$. En caso que ese sufijo no exista, el algoritmo debe entregar NULL.

Escriba un algoritmo que resuelva LONGEST-SUFFIX-NFA en tiempo $\mathcal{O}(|\mathcal{A}| \cdot |w|)$ donde $|\mathcal{A}|$ es el número de estados y transiciones de $\mathcal{A}$. Demuestre la correctitud de su algoritmo.

Solución 1

Para resolver el problema LONGEST-SUFFIX-NFA, se requiere diseñar un algoritmo que encuentre el sufijo más largo de una palabra $w = a_1, \dots, a_n$ que sea aceptado por un autómata finito no determinista (NFA) $A = (Q, \Sigma, \delta, I, F)$. La idea es recorrer la palabra desde el final hacia el principio, verificando si algún sufijo es aceptado por el NFA. Para ello, se puede utilizar un algoritmo de simulación de NFA, que permita determinar si un estado es alcanzable desde otro estado, dado un símbolo de entrada. A continuación, se presenta el pseudocódigo del algoritmo:

Algorithm 1: find-longest-suffix-NFA(N, w)

Input: Un NFA $A = (Q, \Sigma, \delta, I, F)$ y una palabra $w = a_1, \dots, a_n$

Output: El sufijo más largo de w que es aceptado por A

```
1  $S \leftarrow F$  ; // Conjunto de estados finales
2  $m \leftarrow \infty$  ; // Longitud del sufijo más largo
3 for  $i = n$  to 1 do
    // Recorre la palabra desde el final
4      $S_{old} \leftarrow S$  ; // Copia de los estados alcanzables
5      $S \leftarrow \emptyset$  ; // Conjunto de estados alcanzables
    // Notar que este ciclo tiene complejidad  $\mathcal{O}(|\Delta|)$ 
6     foreach  $p \in S_{old}$  do
        // Recorre los estados alcanzables
7          $S \leftarrow S \cup \{q \mid (q, a_i, p) \in \Delta\}$  ; // Estados alcanzables con  $a_i$ 
    // Notar que esta verificación se cumple en  $\mathcal{O}(|Q|)$ 
8     if  $I \cap S \neq \emptyset$  then
        // Si hay estados alcanzables desde el inicial
9          $m \leftarrow i$  ; // Actualiza la longitud del sufijo más largo
10 if  $m = \infty$  then
11     return NULL ; // No hay sufijo aceptado
12 else
13     return  $i$  ; // Retorna la posición del sufijo más largo
```

Explicación del Algoritmo

1. **Inicialización:** Se inicializa el conjunto de estados S con los estados finales F y la longitud del sufijo más largo m con infinito.
2. **Recorrido de la palabra:** Se recorre la palabra desde el final hacia el principio, verificando si algún sufijo es aceptado por el NFA.
3. **Simulación del NFA:** Para cada símbolo de la palabra, se simula el NFA para determinar los estados alcanzables desde los estados alcanzables anteriores.
4. **Verificación de estados iniciales:** Se verifica si hay algún estado alcanzable desde el estado inicial. Si es así, se actualiza la longitud del sufijo más largo.
5. **Retorno de resultados:** Si la longitud del sufijo más largo es infinito, no hay sufijo aceptado. En caso contrario, se retorna la posición del sufijo más largo.

Complejidad del Algoritmo: La complejidad del algoritmo es $O(|A| \cdot |w|)$ en el peor caso, donde $|A|$ es el tamaño del alfabeto y $|w|$ es la longitud de la palabra. La complejidad se debe a la simulación del NFA, que tiene una complejidad de $O(|\Delta|)$, donde Δ es el conjunto de transiciones del NFA. En el peor caso, se debe recorrer la palabra completa, lo que implica una complejidad de $O(|w|)$.

Justificación de la correctitud

1. **Recorrido completo de la palabra:** El bucle inverso asegura que verificamos todos los posibles sufijos de w .
2. **Simulación del NFA:** La simulación del NFA permite determinar los estados alcanzables desde los estados alcanzables anteriores, dado un símbolo de entrada.
3. **Verificación de estados iniciales:** La verificación de estados iniciales permite determinar si hay algún estado alcanzable desde el estado inicial.
4. **Retorno de resultados:** El algoritmo retorna la longitud del sufijo más largo que es aceptado por el NFA.

Distribución de puntaje

- 1 punto por explicar el algoritmo de simulación del NFA.
- 1 punto por inicializar correctamente el conjunto de estados S y la longitud del sufijo más largo m .
- 1 punto por recorrer los estados alcanzables y actualizar el conjunto de estados S .
- 1 punto por verificar si hay algún estado alcanzable desde el estado inicial y actualizar la longitud del sufijo más largo.
- 1 punto por retornar la longitud del sufijo más largo.
- 1 punto por justificar la correctitud del algoritmo y la complejidad del mismo.

Solución 2

Alternativamente, se puede resolver el problema LONGEST-SUFFIX-NFA recorriendo la palabra desde el principio hacia el final, verificando si algún sufijo es aceptado por el NFA. Para ello, se puede utilizar un algoritmo de simulación de NFA, que permita determinar si un estado es alcanzable desde otro estado, dado un símbolo de entrada. A continuación, se presenta el pseudocódigo del algoritmo:

Algorithm 2: find-longest-suffix-NFA(N, w)

Input: Un NFA $A = (Q, \Sigma, \delta, I, F)$ y una palabra $w = a_1, \dots, a_n$

Output: El sufijo más largo de w que es aceptado por A

```
1  $S \leftarrow \{(q, 1) | q \in I\} \cup \{(q, \infty) | q \notin I\}$ ; //  $S$  es una tabla de hash con longitud de sufijo más largo
2 for  $i = 1$  to  $n$  do
    // Recorre la palabra desde el principio
3      $S_{old} \leftarrow S$ ; // Copia de los estados alcanzables
4      $S \leftarrow \{(q, i) | q \in I\} \cup \{(q, \infty) | q \notin I\}$ ; // Conjunto de estados alcanzables
5     foreach  $p \in Q$  do
        // Notar que este ciclo tiene complejidad  $O(|\Delta|)$ 
6         foreach  $(p, a_i, q) \in \Delta$  do
7              $S[q] \leftarrow \min(S[q], i)$ ; // Actualiza la longitud del sufijo más largo
8      $m \leftarrow \min S[p] | p \in F$ ; // Longitud del sufijo más largo
9     if  $m = \infty$  then
10        return NULL; // No hay sufijo aceptado
11    else
12        return  $m$ ; // Retorna la posición del sufijo más largo
```

Explicación del Algoritmo

1. **Inicialización:** Se inicializa la tabla de hash S con los estados iniciales y finales del NFA.
2. **Recorrido de la palabra:** Se recorre la palabra desde el principio hacia el final, verificando si algún sufijo es aceptado por el NFA.
3. **Simulación del NFA:** Para cada símbolo de la palabra, se simula el NFA para determinar los estados alcanzables desde los estados alcanzables anteriores.
4. **Verificación de estados finales:** Se verifica si hay algún estado final alcanzable desde los estados alcanzables.
5. **Retorno de resultados:** Si la longitud del sufijo más largo es infinito, no hay sufijo aceptado. En caso contrario, se retorna la longitud del sufijo más largo.

Complejidad del Algoritmo: La complejidad del algoritmo es $O(|A| \cdot |w|)$ en el peor caso, donde $|A|$ es el tamaño del alfabeto y $|w|$ es la longitud de la palabra. La complejidad se debe a la simulación del NFA, que tiene una complejidad de $O(|\Delta|)$, donde Δ es el conjunto de transiciones del NFA. En el peor caso, se debe recorrer la palabra completa, lo que implica una complejidad de $O(|w|)$.

Justificación de la correctitud

1. **Recorrido completo de la palabra:** El bucle asegura que verificamos todos los posibles sufijos de w .
2. **Simulación del NFA:** La simulación del NFA permite determinar los estados alcanzables desde los estados alcanzables anteriores, dado un símbolo de entrada.
3. **Verificación de estados finales:** La verificación de estados finales permite determinar si hay algún estado final alcanzable desde los estados alcanzables.
4. **Retorno de resultados:** El algoritmo retorna la longitud del sufijo más largo que es aceptado por el NFA.

Distribución de puntaje

- 1 punto por explicar el algoritmo de simulación del NFA.
- 1 punto por inicializar correctamente la tabla de hash S .
- 1 punto por recorrer los estados alcanzables y actualizar la tabla de hash S .
- 1 punto por verificar si hay algún estado final alcanzable y actualizar la longitud del sufijo más largo.
- 1 punto por retornar la longitud del sufijo más largo.
- 1 punto por justificar la correctitud del algoritmo y la complejidad del mismo.



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de autómatas y lenguajes formales — 2'2020

Examen

Indicaciones

- La duración del examen es 3 horas pero puede tomarse más tiempo si estima necesario.
- Responda cada pregunta en una hoja separada y ponga su nombre, sección y número de lista en cada hoja de respuesta.
- Debe entregar una copia digital de cada pregunta por el buzón del curso, antes de las 23:59 horas del día del examen.
- Debe preocuparse que la copia digital y su calidad sea legible. Se recomienda usar hojas blancas y un lápiz oscuro que sea visible en la versión digital. En caso de no ser legible, no podrá ser evaluada su solución.
- Durante la evaluación puede hacer uso de sus apuntes o slides del curso.
- Esta es una evaluación estrictamente individual y, por lo tanto, no puede compartir información con sus compañeros o usar material fuera de sus apuntes o slides del curso. En caso de hacerlo, el examen no reflejará su progreso en el curso, viéndose perjudicada su formación personal y profesional.
- **Al comienzo de cada pregunta debe escribir la siguiente oración y firmarla:**

“Doy mi palabra que la siguiente solución de la pregunta X fue desarrollada y escrita individualmente por mi persona según el código de honor de la Universidad.”

En caso de no escribir esto al comienzo de cada pregunta, su solución no será evaluada.

Pregunta 1

Sea $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ un autómata finito no-determinista. Considere el lenguaje:

$$\mathcal{S}(\mathcal{A}) = \{ p \cdot q \cdot w \in I \cdot F \cdot \Sigma^* \mid \text{existe una ejecución de } \mathcal{A} \text{ sobre } w \text{ desde el estado } p \text{ al estado } q \}$$

Demuestre que $\mathcal{S}(\mathcal{A})$ es un lenguaje regular.

Pregunta 2

1. Sea $\Sigma = \{a, \#\}$ y considere el lenguaje L de todas las palabras de la forma $u_1 \# u_2 \# \dots \# u_n$ con $n \geq 1$ tal que $u_i \in \{a\}^*$ para todo $i \leq n$ y $u_j \neq u_k$ para todo $j, k \leq n$ con $j \neq k$.
Demuestre que L NO es regular.
2. Demuestre la siguiente versión del lema de bombeo para lenguajes libres de contexto:

“Existe un $N > 0$ tal que para toda palabra $z \in L$ con $|z| \geq N$ existe una descomposición $z = uvwxy$ con $v \neq \epsilon$ tal que para todo $i \geq 0$, se cumple que $u \cdot v^i \cdot w \cdot x^i \cdot y \in L$.”

Pregunta 3

Demuestre que para todo lenguaje regular L , existe una gramática libre de contexto $\mathcal{G}$ tal que $L = \mathcal{L}(\mathcal{G})$ y $\mathcal{G}$ es una gramática $\text{LR}(k)$ para algún k .

Para esta pregunta, usted NO puede hacer su solución demostrando que L es $\text{LL}(k)$ y, después, que toda gramática $\text{LL}(k)$ es también $\text{LR}(k)$.

Pregunta 4

Para un lenguaje $L \subseteq \Sigma^*$, se define $\mathbf{first}_k(L) = \{ w|_k \mid w \in L \}$.

Escriba un algoritmo que reciba como entrada un autómata finito no-determinista $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$, una palabra $w \in \Sigma^*$ y $k > 0$, y responda **TRUE** si, y solo si, $w \in \mathbf{first}_k(\mathcal{L}(\mathcal{A}))$. Su algoritmo debe tomar tiempo lineal en $|Q| + |\Delta|$, $|w|$ y k . Por último, demuestre la correctitud de su algoritmo.



PAUTA EXAMEN

Pregunta 1

Una posible solución es la siguiente. Sea $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ el autómata finito no-determinista. A partir de $\mathcal{A}$, construimos $\mathcal{A}' = (Q', \Sigma \cup Q, \Delta', I', F')$ tal que:

- $Q' = \{q_0\} \cup Q \cup Q \times Q$ donde q_0 es un estado nuevo (esto es $q_0 \notin Q$)
- $\Delta' = \{(q_0, p, p) \mid p \in I\} \cup \{(p, q, (p, q)) \mid q \in F\} \cup \{((p, q), a, (p', q)) \mid (p, a, p') \in \Delta\}$
- $I' = \{q_0\}$
- $F' = \{(q, q) \mid q \in F\}$

Por último, demostramos que $\mathcal{S}(\mathcal{A}) = \mathcal{L}(\mathcal{A}')$, una dirección a la vez.

1. ($\mathcal{S}(\mathcal{A}) \subseteq \mathcal{L}(\mathcal{A}')$) Sea $w \in \mathcal{S}(\mathcal{A})$. Por la definición de $\mathcal{S}(\mathcal{A})$, sabemos que w tiene la forma $p \cdot q \cdot a_1 \dots a_n$, con $p \in I$, $q \in F$, $a_i \in \Sigma$ y existe una ejecución de aceptación $\rho : p = p_0 \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} p_n = q$ de $\mathcal{A}$ sobre $a_1 a_2 \dots a_n$. Por la construcción de $\mathcal{A}'$ podemos verificar que la siguiente es una ejecución de $\mathcal{A}'$ sobre w :

$$q_0 \xrightarrow{p} p \xrightarrow{q} (p, q) \xrightarrow{a_1} (p_1, q) \dots \xrightarrow{a_n} (p_n, q)$$

Luego, dado que $p_n = q$, w llega a un estado final, entonces w es aceptado por $\mathcal{A}'$ y $w \in \mathcal{L}(\mathcal{A}')$.

2. ($\mathcal{L}(\mathcal{A}') \subseteq \mathcal{S}(\mathcal{A})$) Sea $w' \in \mathcal{L}(\mathcal{A}')$. Por construcción de $\mathcal{A}'$, podemos ver que w' tiene que tener la forma $w' = p \cdot q \cdot a_1 \dots a_n$ y existe una ejecución de aceptación

$$\rho' : q_0 \xrightarrow{p} p \xrightarrow{q} (p, q) \xrightarrow{a_1} (p_1, q) \dots \xrightarrow{a_n} (q, q)$$

de $\mathcal{A}'$ sobre w' . Por la definición de $\mathcal{A}'$, podemos obtener la ejecución $\rho : p = p_0 \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} p_n = q$ a partir de las transiciones del estilo $((p, q), a, (p', q))$ de Δ' . Dado que $p \in I$, $q \in F$ y $(p_i, a_i, p_{i+1}) \in \Delta$, entonces ρ es una ejecución de aceptación sobre $a_1 \dots a_n$. Esto es $w' \in \mathcal{S}(\mathcal{A})$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** Por definir correctamente el alfabeto del autómata $\mathcal{A}'$.
- **(0.5 puntos)** Por definir correctamente el conjunto de estados finales del autómata $\mathcal{A}'$.
- **(1 punto)** Si el autómata lea el estado p y lo almacena.
- **(1 punto)** Si el autómata lea el estado q y lo almacena.
- **(1 punto)** Si el autómata simule las transiciones del autómata $\mathcal{A}$.
- **(1 punto)** Por demostrar correctamente que $\mathcal{S}(\mathcal{A}) \subseteq \mathcal{L}(\mathcal{A}')$.
- **(1 puntos)** Por demostrar correctamente que $\mathcal{L}(\mathcal{A}') \subseteq \mathcal{S}(\mathcal{A})$.

Pregunta 2

Pregunta 2.1

Una posible solución consiste en aplicar el contrapositivo del lema de bombeo para lenguajes regulares de la siguiente manera. Para $N > 0$ cualquiera, podemos escoger la siguiente palabra

$$a^N \# a^{N+1} \# a^{N+2} \# \dots \# a^{2N}$$

la cuál subdividimos como $x = \varepsilon$, $y = a^N$, $z = \# a^{N+1} \# a^{N+2} \# \dots \# a^{2N}$, se cumple claramente que $|xyz| \leq N$. Ahora, nuestra competencia, el demonio, elige una descomposición cualquiera de $y = \underbrace{a^j}_u \underbrace{a^k}_v \underbrace{a^l}_w$, en

donde se tiene que $k \neq 0$ y además $j + k + l = N$.

Ahora escogemos $i = 2$ para bombear y se tiene una palabra

$$xuv^2wz = a^j a^{2k} a^l \# a^{N+1} \# a^{N+2} \# \dots \# a^{2N}$$

con $N + 1 \leq j + 2k + l \leq 2N$, pues $k \neq 0$. Luego existen 2 subpalabras $u_1 = a^j a^{2k} a^l$ y u_r que coinciden, de forma tal que la palabra xuv^2wz no pertenece al lenguaje L , pues no son todos los fragmentos de forma a^m son distintos de a pares. De acá se concluye que el lenguaje no es regular.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1,5 puntos) Por elegir correctamente una palabra xyz de largo mayor a N .
- (0,5 puntos) Por elegir correctamente una subdivisión de y .
- (0,5 puntos) Por elegir correctamente la cantidad de veces i a bombear.
- (0,5 puntos) Por argumentar que la palabra obtenida no pertenece al lenguaje.

Pregunta 2.2

La idea general es modificar la demostración de la versión clásica del lema. Si tenemos una palabra ω de largo mayor a un N suficientemente grande, podemos repetir un subarbol dentro del árbol de derivación. Dado el lenguaje libre de contexto L del enunciado, podemos suponer que existe una gramática en forma normal de Greibach que lo genera, es decir todas sus producciones son de forma:

$$A \rightarrow aA_1 \dots A_n$$

Donde a es una terminal y lo que sigue a la derecha es una secuencia de variables no terminales.

Ahora la demostración sigue la misma estrategia vista en clases. Elegimos $N = 2^{|V|+1}$ y, como cada producción tiene la forma de arriba, todo árbol de derivación de esta gramática tiene una rama con al menos $\log_2(N)$ nodos internos. Ahora si tomamos una palabra ω de largo mayor a N , por lo anterior su árbol de derivación tiene una rama de altura mayor a $|V|$. Luego por el principio del palomar, en esa rama existe una variable al menos que se repite, llamemosla X , por ende podemos bombear, luego la palabra uv^iwx^iy pertenece al lenguaje L . Como las reglas son de la forma $A \rightarrow aA_1 \dots A_n$ entonces el subarbol desde X a X tendrá al menos una letra a la derecha, lo que implica que $v \neq \epsilon$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1 punto) Por usar la forma normal de Greibach.
- (0,5 puntos) Por usar la forma normal para argumentar que $v \neq \epsilon$.
- (0,5 puntos) Por elegir correctamente el valor de N .
- (0,5 puntos) Por usar el principio del palomar.
- (0,5 puntos) Por explicar como bombear.

Pregunta 3

Demostraremos que para todo lenguaje regular L , existe una gramática libre de contexto $\mathcal{G}$ tal que $L = \mathcal{L}(\mathcal{G})$ y $\mathcal{G}$ es $\text{LR}(k)$ para algún k . Dado que el lenguaje L es regular, sabemos que existirá un DFA $A = (Q, \Sigma, \delta, q_0, F)$ tal que $L = \mathcal{L}(A)$; por lo que podemos construir la gramática $\mathcal{G} = (Q, \Sigma, P, \{q_0\})$ a partir de A con:

$$P = \{p \rightarrow aq \mid p, q \in Q, a \in \Sigma \wedge \delta(p, a) = q\} \cup \{p \rightarrow \varepsilon \mid p \in F\}$$

Como se vio en clases, esta gramática definirá el mismo lenguaje que el autómata en el que se basa, es decir $L = \mathcal{L}(A) = \mathcal{L}(\mathcal{G})$, por lo que ahora debemos demostrar que existe un k tal que $\mathcal{G}$ es $\text{LR}(k)$.

Una gramática libre de contexto será $\text{LR}(k)$, si para toda derivación:

- $\alpha \cdot \beta \cdot v_1 \xleftarrow[rm]{*} \alpha \cdot X \cdot v_1 \xleftarrow[rm]{*} S'$
- $\alpha \cdot \beta \cdot v_2 \xleftarrow[rm]{*} \alpha' \cdot Y \cdot v_2' \xleftarrow[rm]{*} S'$
- $v_1|_k = v_2|_k$

se cumple que $\alpha = \alpha'$, $X = Y$ y $v_2 = v_1$. Observando ahora las producciones de la gramática, podemos ver que tendremos solamente dos opciones para sus derivaciones, y son las siguientes:

- $S' \xrightarrow[rm]{*} a_1 a_2 \dots a_k \cdot p \cdot \varepsilon \xrightarrow[rm]{*} a_1 a_2 \dots a_k \cdot aq \cdot \varepsilon$, donde $\alpha = a_1 a_2 \dots a_k$, $X = p$, $v_1 = \varepsilon$ y $\beta = aq$. Luego, tomando una derivación diferente tenemos: $S' \xrightarrow[rm]{*} a_1 a_2 \dots a_j \cdot p' \cdot \varepsilon \xrightarrow[rm]{*} a_1 a_2 \dots a_k \cdot aq \cdot \varepsilon$, donde $\alpha' = a_1 a_2 \dots a_j$, $Y = p'$, $v_2 = \varepsilon$, $v_2' = \varepsilon$ y α, β deben ser los mismos que en la derivación anterior. Debido a la construcción de la gramática, cada producción puede generar una sola letra, lo que significa $\alpha = \alpha'$. Además, como está basada en un DFA, solamente existe un estado p que leyendo la letra a llegue al estado q , lo que significa que $p = p'$, esto es, $X = Y$. Finalmente, tenemos que $v_1 = v_2 = v_2' = \varepsilon$, lo que significa no solo que la gramática será $\text{LR}(k)$ para algún k , si no también que podría serlo para todo $k \geq 0$, pues $\varepsilon|_k = \varepsilon$ para todo $k \geq 0$.
- $S' \xrightarrow[rm]{*} a_1 a_2 \dots a_k \cdot p \cdot \varepsilon \xrightarrow[rm]{*} a_1 a_2 \dots a_k \cdot \varepsilon \cdot \varepsilon$, donde $\alpha = a_1 a_2 \dots a_k$, $X = p$, $v_1 = \varepsilon$ y $\beta = \varepsilon$.

A primera vista, pareciera que este caso es análogo al anterior, pues podemos tomar una derivación de la forma: $S' \xrightarrow[rm]{*} a_1 a_2 \dots a_j \cdot p' \cdot \varepsilon \xrightarrow[rm]{*} a_1 a_2 \dots a_k \cdot \varepsilon \cdot \varepsilon$, donde $\alpha' = a_1 a_2 \dots a_j$, $Y = p'$, $v_2 = \varepsilon$ y α, β deben ser los mismos que en la derivación anterior; y luego podemos utilizar el mismo argumento para $\alpha = \alpha'$, $v_1 = v_2 = v_2'$ y $p = p'$. Sin embargo existe otra derivación posible. Consideremos la siguiente derivación:

$$S' \xrightarrow[rm]{*} a_1 a_2 \dots a_k b_1 b_2 \dots b_n \cdot p'' \cdot \varepsilon \xrightarrow[rm]{*} a_1 a_2 \dots a_k \cdot \varepsilon \cdot b_1 b_2 \dots b_n$$

dónde $\alpha' = a_1 a_2 \dots a_k b_1 b_2 \dots b_n$, $Y = p''$, $v_2' = \varepsilon$, $v_2 = b_2 \dots b_n$ y α, β deben ser los mismos que en la derivación anterior.

Pareciera que este caso no aporta a la demostración, pues es claro que $v_1 \neq v_2$ y $v_2 \neq v_2'$, sin embargo sí se cumple que $v_1|_0 = v_2|_0$ y, puesto que $v_2 \neq v_2'$, tendremos que esta gramática no es $\text{LR}(0)$. Ahora bien, como si tenemos que $v_1|_1 \neq v_2|_1$, esta propiedad si se cumple para $\text{LR}(1)$.

Por lo tanto, la gramática $\mathcal{G}$ será $\text{LR}(1)$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por construir correctamente la gramática.
- **(3 puntos)** Por demostrar que la gramática es $\text{LR}(k)$ para algún k , usando las producciones con $\beta \neq \varepsilon$.
 - **(1 puntos)** Por utilizar correctamente $\mathcal{G}$ para construir una derivación de la forma $S' \xrightarrow[rm]{*} a_1 a_2 \dots a_k \cdot p \cdot \varepsilon$.
 - **(2 puntos)** Por demostrar que las demás derivaciones cumplen con $\alpha = \alpha'$, $X = Y$, $v_2 = v_2'$.
- **(1 punto)** Por considerar el caso $\beta = \varepsilon$, incluso si no se descarta que la gramática sea $\text{LR}(0)$.

Pregunta 4

Para un autómata finito no-determinista $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$, considere el conjunto:

$$F^* = \{q \mid \text{existe una ejecución de aceptación } \rho \text{ desde } q \text{ sobre } A\}$$

es decir todos los estados tal que desde ellos se puede llegar a algún estado final con alguna ejecución. Luego el algoritmo propuesto es el siguiente.

Algorithm 1: Determina si $w \in \text{first}_k(\mathcal{L}(\mathcal{A}))$

Input : $\mathcal{A} = (Q, \Sigma, \Delta, I, F), w = a_1 \dots a_n, k$
Output: true si, y solo si, w esta en $\text{first}_k(\mathcal{L}(\mathcal{A}))$
 $F^* := \text{DFS}(\mathcal{A});$
if $|w| > k$ **then**
 | **return** *false*
end
 $S := I, i := 1;$
while $i \leq |w|$ & $i \leq k$ **do**
 | $S := \text{next}(\Delta, S, a_i);$
 | $i := i + 1;$
end
return $S \cap F^* \neq \emptyset$

Donde $\text{DFS}(\mathcal{A})$ es una búsqueda en profundidad de a partir cada estado del autómata hasta encontrar un estado final lo que retornará solo los estados para los cuales se pueda alcanzar un estado final desde ellos. Además **next** se define como se vió en clases para el algoritmo de evaluación de un NFA.

Finalmente, se debe justificar la correctitud del algoritmo y del tiempo del algoritmo que es:

$$\mathcal{O}((|\Delta| + |Q|) \cdot \min(k, |w|) + |\Delta| \cdot |Q|).$$

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1 punto) Por utilizar la evaluación `on_the_fly` para el automata.
- (1 punto) Por utilizar k para no revisar la palabra completa.
- (2 puntos) Por utilizar F^* o bien la noción de estados que alcanzan estados finales.
- (1 punto) Por explicar la correctitud del algoritmo.
- (1 punto) Por justificar el que su algoritmo cumple con el tiempo pedido.



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2° 2021

Examen

Preguntas en blanco: Preguntas entregadas en blanco se evaluarán con un 1.5.

Pregunta 1

Sea $\Sigma = \{a, b\}$. Para lenguajes $L_1, L_2 \subseteq \Sigma^*$ se define:

$$L_1 \mid L_2 = \{uv \in L_1 \mid v \in L_2\}$$

Demuestre que si L_1 y L_2 son lenguajes regulares, entonces $L_1 \mid L_2$ también es regular.

Pregunta 2

1. Demuestre que el siguiente lenguaje es libre de contexto:

$$R = \{a^i b^j c^k \mid i < j \vee j < k\}$$

Para esto, demuestre una gramática o un PDA que defina el lenguaje y explique su correctitud.

2. Demuestre que el siguiente lenguaje NO es libre de contexto:

$$S = \{a^i b^j c^k \mid i < j \wedge j < k\}$$

Pregunta 3

Sea $G = (V, \Sigma, P, S)$ una gramática en forma normal de Greibach sin variables inútiles. Demuestre que G es una gramática LL(1) si, y solo si, para todo par de reglas distintas $X \rightarrow a\gamma$ y $X \rightarrow a'\gamma'$ en P se tiene que, si $a = a'$, entonces $\gamma = \gamma'$.

Pregunta 4

Para un lenguaje $L \subseteq \Sigma^*$ y $a \in \Sigma$, se define $\text{follow}_k(a) = \{v|_k \mid u \cdot a \cdot v \in L\}$.

Escriba un algoritmo que reciba como entrada un autómata finito no-determinista $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$, una letra $a \in \Sigma$, una palabra $w \in \Sigma^*$ y $k > 0$, y responda TRUE si, y solo si, $w \in \text{follow}_k(a)$. Su algoritmo debe tomar tiempo $\mathcal{O}(|\mathcal{A}| \cdot |w|)$. Por último, explique la correctitud de su algoritmo.



PAUTA EXAMEN

Pregunta 1

Como L_1 y L_2 son lenguajes regulares, sabemos que existirán los DFA $\mathcal{A}_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ y $\mathcal{A}_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ tales que $\mathcal{L}(\mathcal{A}_1) = L_1$ y $\mathcal{L}(\mathcal{A}_2) = L_2$. Luego, para demostrar que $L_1|L_2$ es regular, bastará con encontrar un autómata que lo defina.

Sea $\mathcal{A} = (Q_1 \cup Q_2, \Sigma, \Delta, \{q_{01}\}, F_1 \times F_2)$, con Δ dado por:

$$\begin{aligned} \Delta = & \{(p_1, a, q_1) \mid \delta(p_1, a) = q_1\} \cup \\ & \{(p_1, \varepsilon, (p_1, q_{02})) \mid p_1 \in Q_1\} \cup \\ & \{((p_1, p_2), a, (q_1, q_2)) \mid \delta(p_1, a) = q_1 \wedge \delta(p_2, a) = q_2\} \end{aligned}$$

Ahora debemos demostrar que $\mathcal{L}(\mathcal{A}) = L_1|L_2$.

1. $L_1|L_2 \subseteq \mathcal{L}(\mathcal{A})$:

Sea $w = uv \in L_1|L_2$. Sabemos entonces que $w \in L_1 \wedge v \in L_2$, con $w = a_1...a_n$ y $v = a_{i+1}...a_n$. Luego, sabemos que existirán ejecuciones:

$$\begin{aligned} \rho_1 : q_{01} &\xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} p_n \wedge p_n \in F_1 \\ \rho_2 : q_{02} &\xrightarrow{a_i} q_i \xrightarrow{a_{i+1}} \dots \xrightarrow{a_n} q_n \wedge q_n \in F_2 \end{aligned}$$

A partir de ρ_1 y ρ_2 podemos construir:

$$\rho : q_{01} \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_{i-1}} p_{i-1} \xrightarrow{\varepsilon} (p_{i-1}, q_{02}) \xrightarrow{a_i} (p_i, q_i) \xrightarrow{a_{i+1}} \dots \xrightarrow{a_n} (p_n, q_n) \wedge p_n \in F_1 \wedge q_n \in F_2$$

Es claro que ρ es una ejecución de aceptación de $\mathcal{A}$ sobre w , es decir, $w \in \mathcal{L}(\mathcal{A})$ y por lo tanto $L_1|L_2 \subseteq \mathcal{L}(\mathcal{A})$.

2. $\mathcal{L}(\mathcal{A}) \subseteq L_1|L_2$:

Sea $w \in \mathcal{L}(\mathcal{A})$. Por la construcción de $\mathcal{A}$, sabemos que existe una ejecución de $\mathcal{A}$ sobre w que tendrá la siguiente forma:

$$\rho : q_{01} \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_{i-1}} p_{i-1} \xrightarrow{\varepsilon} (p_{i-1}, q_{02}) \xrightarrow{a_i} (p_i, q_i) \xrightarrow{a_{i+1}} \dots \xrightarrow{a_n} (p_n, q_n) \wedge p_n \in F_1 \wedge q_n \in F_2$$

Luego, por definición de Δ , sabemos que para toda transición de la forma $p_{k-1} \xrightarrow{a_k} p_k$ se cumplirá $\delta_1(p_{k-1}, a_k) = p_k$ y para toda transición de la forma $(p_{k-1}, q_{k-1}) \xrightarrow{a_k} (p_k, q_k)$ se cumplirá $\delta_1(p_{k-1}, a_k) = p_k$ y $\delta_2(q_{k-1}, a_k) = q_k$. Utilizando esto, podemos generar las siguientes ejecuciones de $\mathcal{A}_1$ y $\mathcal{A}_2$

$$\begin{aligned} \rho_1 : q_{01} &\xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} p_n \wedge p_n \in F_1 \\ \rho_2 : q_{02} &\xrightarrow{a_i} q_i \xrightarrow{a_{i+1}} \dots \xrightarrow{a_n} q_n \wedge q_n \in F_2 \end{aligned}$$

Sabemos entonces que $w = a_1...a_n \in L_1$ y que $v = a_{i+1}...a_n \in L_2$, lo que a su vez significa que $\exists u. uv \in L_1 \wedge v \in L_2$ y, por tanto $w \in L_1|L_2$.

Por lo tanto $\mathcal{L}(\mathcal{A}) = L_1|L_2$ y $L_1|L_2$ es un lenguaje regular.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por construir los estados de $\mathcal{A}$ correctamente.
 - **(1 punto)** Por el conjunto de estados, $Q_1 \cup Q_1 \times Q_2$.
 - **(0.5 puntos)** Por los estados iniciales, $\{q_{01}\}$.
 - **(0.5 puntos)** Por los estados finales, $F_1 \times F_2$.
- **(3 puntos)** Por construir la relación de transición de $\mathcal{A}$ correctamente.
 - **(1 punto)** Por el simular $\mathcal{A}_1$ (primer conjunto de Δ).
 - **(1 punto)** Por pasar de $\mathcal{A}_1$ a $\mathcal{A}_1 \times \mathcal{A}_2$ (segundo conjunto de Δ).
 - **(1 punto)** Por el simular $\mathcal{A}_1 \times \mathcal{A}_2$ (tercer conjunto de Δ).
- **(1 punto)** Por demostrar correctamente que $\mathcal{L}(\mathcal{A}) = L_1|L_2$.
 - **(0.5 puntos)** Por la dirección $\mathcal{L}(\mathcal{A}) \subseteq L_1|L_2$.
 - **(0.5 puntos)** Por la dirección $L_1|L_2 \subseteq \mathcal{L}(\mathcal{A})$.

Pregunta 2

Pregunta 2.1

Sea $G = (\{S, X, C, X', B, Y, A', Y'\}, \{a, b, c\}, P, S)$ una Gramática Libre de Contexto, donde P contiene las siguientes producciones:

$$\begin{aligned}
 S &\longrightarrow X \mid Y \\
 X &\longrightarrow X'C \\
 C &\longrightarrow cC \mid \epsilon \\
 X' &\longrightarrow aX'b \mid bB \\
 B &\longrightarrow bB \mid \epsilon \\
 Y &\longrightarrow AY' \\
 A' &\longrightarrow aA \mid \epsilon \\
 Y' &\longrightarrow bY'c \mid cC
 \end{aligned}$$

Hay varias gramáticas que resuelven el problema. Se elige este formato para mostrar las ideas que deben estar en la solución. Soluciones con las mismas ideas tendrán todo el puntaje.

Explicación: Se escoge cual de las 2 condiciones cumplir. Al elegir una, se agrega una cantidad arbitraria de la letra que no influye en la condición. Después, se agrega una cantidad igual de letras tal que $i = j$ o $j = k$ dependiendo del caso. Finalmente, se agrega al menos una letra tal que $j > i$ o $k > j$ según el caso.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por elegir la condición a cumplir (producciones de S).
- **(0.5 puntos)** Por agregar arbitrariamente letras en una primera condición (producciones de X y C).
- **(1 punto)** Por satisfacer una primera condición (producciones de X' y B).
- **(0.5 punto)** Por toda una segunda condición (producciones de Y , A' e Y').

Pregunta 2.2

Por Lema de Bombeo para CFG. Sea para todo $N > 0$ la palabra $z = a^N b^{N+1} c^{N+2}$. Sean $i = N, j = N + 1$ y $k = N + 2$ se tiene que $i < j < k$, esto es, z está en S y $|z| \geq N$.

Sea $z = uvwxy$ una descomposición cualquiera tal que $vx \neq \epsilon$ y $|vwx| \leq N$. Sea $z' = uv^i wx^i y$ para algún $i \geq 0$. Luego, se tienen los siguientes casos (no necesariamente excluyentes entre sí):

1. Si v o x son combinaciones de 2 letras. Entonces, con $i = 2$, z' ya no está en $\mathcal{L}(a^* b^* c^*)$ ni en S .
2. Si $x \in \mathcal{L}(a^*)$. Entonces, con $i = 2$, se tienen mayor o igual letras a que b y $z' \notin S$.
3. Si $x \in \mathcal{L}(b^*)$. Entonces, con $i = 2$, se tienen mayor o igual letras b que c y $z' \notin S$.
4. Si $v \in \mathcal{L}(b^*)$. Entonces, con $i = 0$, se tienen mayor o igual letras a que b y $z' \notin S$.
5. Si $v \in \mathcal{L}(c^*)$. Entonces, con $i = 0$, se tienen mayor o igual letras b que c y $z' \notin S$.

Por lo tanto, S no es un Lenguaje Libre de Contexto.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por encontrar una palabra en S útil para el Lema.
- **(0.5 puntos)** Por cada ítem desde el ítem 2 hasta el ítem 4 (2 puntos en total).

Pregunta 3

Sea G una gramática en forma normal de Greibach sin variables inútiles, con reglas del tipo $X \rightarrow a\gamma$ y con $\gamma \in V^*$. Recordamos la definición de $LL(1)$, que es la siguiente: G es $LL(1)$ si para todo par de derivaciones

$$\begin{aligned} S &\Rightarrow^* uX\beta \Rightarrow u\gamma_1\beta \Rightarrow^* uv_1 \\ S &\Rightarrow^* uX\beta \Rightarrow u\gamma_2\beta \Rightarrow^* uv_2 \end{aligned}$$

Si $v_1|_1 = v_2|_1$, entonces $\gamma_1 = \gamma_2$.

($\Rightarrow$) Supongamos que G es $LL(1)$. Sea $X \rightarrow a\gamma, X \rightarrow a'\gamma' \in P$, tal que $a = a'$. Por demostrar: $\gamma = \gamma'$. Como G no tiene variables inútiles, entonces

$$\begin{aligned} S &\Rightarrow^* uX\beta \text{ para algún } u \text{ y } \beta \\ S &\Rightarrow^* uX\beta \Rightarrow ua\gamma\beta \\ S &\Rightarrow^* uX\beta \Rightarrow ua'\gamma'\beta \end{aligned}$$

Como G no tiene variables inútiles:

$$\begin{aligned} a\gamma\beta &\Rightarrow^* v_1 \\ a'\gamma'\beta &\Rightarrow^* v_2 \end{aligned}$$

Si $a = a'$, entonces $v_1|_1 = v_2|_1$. Por lo tanto:

$$\begin{aligned} S &\Rightarrow^* uX\beta \Rightarrow ua\gamma\beta \Rightarrow uv_1 \\ S &\Rightarrow^* uX\beta \Rightarrow ua'\gamma'\beta \Rightarrow uv_2 \\ v_1|_1 &= v_2|_1 \end{aligned}$$

Como la gramática es $LL(1)$, entonces $a\gamma = a'\gamma'$ por lo que $\gamma = \gamma'$.

($\Rightarrow$) Suponemos que para dos reglas $x \rightarrow a\gamma$ y $X \rightarrow a'\gamma'$, si $a = a'$ entonces $\gamma = \gamma'$. Suponga que

$$\begin{aligned} S &\Rightarrow^* uX\beta \Rightarrow u\gamma_1\beta \Rightarrow^* uv_1 \\ S &\Rightarrow^* uX\beta \Rightarrow u\gamma_2\beta \Rightarrow^* uv_2 \\ v_1|_1 &= v_2|_2 \end{aligned}$$

Demostraremos que $\gamma_1 = \gamma_2$. Como G está en GNF, $\gamma_1 = a\gamma$ y $\gamma_2 = a'\gamma'$. Por derivación por la izquierda, sabemos que $a = v_1|_1 = v_2|_1 = a'$. Luego, tenemos que:

$$X \rightarrow a\gamma, X \rightarrow a'\gamma' \text{ y } a = a' \Rightarrow \gamma = \gamma'$$

Por lo tanto $\gamma_1 = \gamma_2$ y G es $LL(1)$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** por recalcar que existen dos derivaciones $S \Rightarrow uX\beta \Rightarrow ua\gamma\beta$ al partir con G en $LL(1)$.
- **(1 punto)** por resaltar que a partir de cada $a\gamma\beta$ se puede llegar a una palabra v al partir con G en $LL(1)$. Descontar 0,5 puntos si no se menciona que G no tiene variables inútiles .
- **(1 punto)** por ocupar $LL(1)$ al partir con G en $LL(1)$.
- **(1 punto)** por recalcar que $\gamma_1 = a\gamma$ y $\gamma_2 = a'\gamma'$ al partir con G una gramática que cumple el lado derecho de la equivalencia.
- **(1 punto)** por recalcar que $a = v_1|_1$ y $a' = v_2|_1$ al partir con G una gramática que cumple el lado derecho de la equivalencia.
- **(1 punto)** por concluir que $\gamma_1 = \gamma_2$ al partir con G una gramática que cumple el lado derecho de la equivalencia.

Pregunta 4

Una posible solución para esta pregunta consiste en la siguiente modificación del algoritmo de evaluación de autómatas no deterministas `eval-NFAonthefly`. Notar que aprovechamos el hecho de que su tiempo está en $\mathcal{O}(|\mathcal{A}| \cdot |w|)$.

```

Function followk( $\mathcal{A} = (Q, \Sigma, \Delta, I, F), a, w, k$ )
    if  $|w| > k$  then
         $\perp$  return FALSE
     $\mathcal{A} = \text{remover\_inútiles}(\mathcal{A})$                                 /* Búsqueda sobre los estados  $\mathcal{O}(|\mathcal{A}|)$  */
     $S = \emptyset$ 
    foreach  $(p, t, q) \in \Delta$  do if  $t = a$  then  $S = S \cup \{q\}$ 

     $S_{final} = \text{eval-NFAonthefly}(\mathcal{A} = (Q, \Sigma, \Delta, S, F), w)$ 
    if  $S_{final} \cap F \neq \emptyset$  then
         $\perp$  return TRUE
    else if  $S_{final} \cap F = \emptyset \wedge |w| = k$  then
         $\perp$  return alcanzables( $S_{final}$ )  $\cap F \neq \emptyset$           /* Búsqueda desde  $S_{final}$   $\mathcal{O}(|\mathcal{A}|)$  */
     $\perp$  return FALSE

```

Donde `remove_inútiles` toma un autómata y nos retorna uno nuevo que tiene solo los estados que son alcanzables. Por otro lado `alcanzables`, dado un conjunto de estados, nos entrega todos los estados alcanzables desde estos usando Δ . Para ambos casos se pueden usar algoritmos de búsqueda lineales sobre la cantidad de estados.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por encontrar todos los estados a los que se puede llegar desde una a sin considerar estados inútiles.
- **(1 punto)** `eval-NFAonthe-fly` desde los estados a los que se puede llegar desde una a
- **(2 puntos)** Por ejecutar `eval-NFAonthe-fly` sobre w
- **(1 punto)** Por retornar el valor correcto en el caso que $|w| = |v|$
- **(1 punto)** Por retornar el valor correcto en el caso que $|w| = k$ y $|v| > k$



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
 ESCUELA DE INGENIERÍA
 DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2° 2022

EXAMEN

Preguntas en blanco: Preguntas o ítems entregados en blanco se evaluarán con 0.5 de 6 puntos.

Pregunta 1

Demuestre que para todo autómata apilador $\mathcal{P}$, existe un autómata apilador alternativo $\mathcal{D}$ tal que:

$$\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{D}).$$

Pregunta 2

Considere los siguientes lenguajes sobre el alfabeto $\{0, 1\}$:

$$L_1 = \{a_1 \dots a_{2n} \in \{0, 1\}^* \mid n > 0 \wedge a_1 a_3 \dots a_{2n-1} = a_2 a_4 \dots a_{2n}\}$$

$$L_2 = \{a_1 \dots a_{2n} \in \{0, 1\}^* \mid n > 0 \wedge a_1 a_3 \dots a_{2n-1} = a_{2n} a_{2n-2} \dots a_2\}$$

Uno de los lenguajes es regular y el otro no. ¿Cuál es cuál? Diga cuál lenguaje es regular y cuál no, y demuestre ambas afirmaciones.

Pregunta 3

Sea $\mathcal{G} = (V, \Sigma, P, S)$ una gramática libre de contexto en forma normal de Chomsky.

- Demuestre que si $\mathcal{G}$ no tiene variables inútiles, entonces

$$\mathcal{L}(\mathcal{G}) \text{ es infinito si, y solo si, existe } X \in V \text{ y } \alpha, \beta \in (V \cup \Sigma)^* \text{ tal que } X \xRightarrow{+} \alpha X \beta \quad (1)$$

donde $\xRightarrow{+}$ significa que es una derivación en uno o más pasos.

- Demuestre que si $\mathcal{G}$ tiene variables inútiles, entonces (1) no es necesariamente cierto.

Pregunta 4

Considere los siguientes dos problemas:

Problema: OUTPUTNOVACIO	Problema: OUTPUTNOTRIVIAL
Input: Un AnnA $\mathcal{N}$ y $d \in \Sigma^*$.	Input: Un AnnA $\mathcal{N}$ y $d \in \Sigma^*$.
Output: TRUE ssi $\llbracket \mathcal{N} \rrbracket(d) \neq \emptyset$.	Output: TRUE ssi $\emptyset \neq \llbracket \mathcal{N} \rrbracket(d) \neq \{\epsilon\}$.

Para cada uno de los problemas, escriba un algoritmo que los resuelva en tiempo $\mathcal{O}(|\mathcal{N}| \cdot |d|)$. Explique la correctitud de su algoritmo.



PAUTA EXAMEN

Pregunta 1

Se busca demostrar que para todo autómata apilador $\mathcal{P}$, existe un autómata apilador alternativo $\mathcal{D}$ tal que $\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{D})$.

Sea el autómata apilador $\mathcal{P} = (Q, \Sigma, \Gamma, \Delta, q_0, \perp, F)$. Construimos un PDA alternativo $\mathcal{D} = (Q', \Sigma, \Delta', q'_0, F')$ tal que:

- $Q' = Q \cup \Gamma \cup \{q'_0\}$
- $F' = F$
- $\Delta' = \{(q'_0, \epsilon, q_0 \perp)\} \cup \{(pA, a, q\gamma) \mid (p, a, A, q, \gamma) \in \Delta\}$

Ahora, debemos demostrar la correctitud del PDA alternativo generado:

- $(\Rightarrow)$ Sea una palabra w y la ejecución de aceptación $(q_0 \perp, w) \vdash_{\mathcal{P}}^* (q_f, \epsilon)$ del autómata apilador $\mathcal{P}$ sobre w . Por la construcción del autómata apilador alternativo $\mathcal{D}$, tendremos la siguiente ejecución:

$$(q'_0, w) \vdash_{\mathcal{D}} (q_0 \perp, w) \vdash_{\mathcal{D}}^* (q_f, \epsilon)$$

Lo que es una ejecución de aceptación para $\mathcal{D}$.

- $(\Leftarrow)$ Sea una palabra w y la ejecución de aceptación $(q'_0, w) \vdash_{\mathcal{D}}^* (q_f, \epsilon)$ del autómata apilador alternativo $\mathcal{D}$. Por construcción, tenemos que:

$$(q'_0, w) \vdash_{\mathcal{D}} (q_0 \perp, w) \vdash_{\mathcal{D}}^* (q_f, \epsilon)$$

Lo que implica la existencia de la ejecución del autómata $\mathcal{P}$:

$$(q_0 \perp, w) \vdash_{\mathcal{P}}^* (q_f, \epsilon)$$

La cual también es una ejecución de aceptación.

Finalmente, al haber demostrado la correctitud de lo generado, damos por concluida la demostración. Dado lo anterior, la distribución de puntaje es la siguiente:

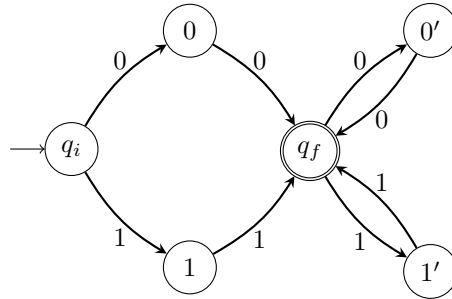
- **(4 puntos)** Por definir correctamente $\mathcal{D}$:
 - (1 punto) por definir correctamente $Q' = Q \cup \Gamma \cup \{q'_0\}$
 - (1 punto) por definir correctamente $F' = F$
 - (2 puntos) por definir correctamente $\Delta' = \{(q'_0, \epsilon, q_0 \perp)\} \cup \{(pA, a, q\gamma) \mid (p, a, A, q, \gamma) \in \Delta\}$
- **(1 punto)** Por demostrar correctitud en la dirección $(\Rightarrow)$.
- **(1 punto)** Por demostrar correctitud en la dirección $(\Leftarrow)$.

Pregunta 2

Se puede notar que el lenguaje L_1 es regular, y que el lenguaje L_2 no lo es. A continuación, se demuestran ambas afirmaciones por separado.

Pregunta 2.1

Una posible demostración de que L_1 es regular, es mediante el siguiente autómata:



La idea general para aceptar el lenguaje $L_1 = \{a_1 \dots a_{2n} \in \{0, 1\}^* \mid n > 0 \wedge a_1 a_3 \dots a_{2n-1} = a_2 a_4 \dots a_{2n}\}$, es que cada vez que se lee una letra en una posición impar, se debe revisar que la letra inmediatamente siguiente sea la misma. Luego, el autómata siempre lee de a dos letras, esperando que sean iguales. Si lo son, vuelve al estado final.

Además, por la condición de que $n > 0$, el ϵ no debe ser aceptado, por lo que el estado inicial no puede ser final, y se añaden estados para revisar las primeras dos letras de la palabra.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1 puntos) Por explicación de autómata
- (4 puntos) Por autómata correcto
- (1 puntos) Por manejar caso de input $w = \epsilon$

Pregunta 2.2

Una posible demostración de que L_2 no es regular, es mediante la siguiente aplicación del contrapositivo del lema de bombeo. Sea un $N > 0$ arbitrario. Existe la siguiente palabra:

$$0^N 110^N \in L_2$$

La cual se puede dividir de la siguiente forma:

$$\begin{aligned} x &= \epsilon \\ y &= 0^N \\ z &= 110^N \end{aligned}$$

Entonces, para cualquier $y = 0^j 0^k 0^l$ con $u = 0^j$, $v = 0^k$, $w = 0^l$ y $k > 0$, podemos tomar $i = 2$ tal que:

$$x \cdot u \cdot v^2 \cdot w \cdot z = 0^{N+k} 110^N \notin L_2$$

Por lo tanto, queda demostrado por el contrapositivo del lema de bombeo que L_2 no es regular.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por elección de palabra perteneciente a L_2
- **(2 puntos)** Por correcta elección de x, y, z y posibles casos de u, v, w
- **(2 puntos)** Por bombear y llegar a palabra que no pertenece a L_2

Pregunta 3

Pregunta 3.1

Se deben demostrar ambas direcciones. En primer lugar demostraremos la dirección *existe* $X \in V$ y $\alpha, \beta \in (V \cup \Sigma)^*$ tal que $X \xRightarrow{+} \alpha X \beta \Rightarrow \mathcal{L}(\mathcal{G})$ es infinito. Sabemos que $X \xRightarrow{+} \alpha X \beta$. Como $\mathcal{G}$ está en CNF, entonces necesariamente $\alpha \neq \varepsilon$ o $\beta \neq \varepsilon$ (no pueden ser vacíos a la vez). También, como la gramática no tiene variables inútiles, sabemos que:

- $\exists \alpha', \beta' \ S \xRightarrow{*} \alpha' X \beta'$ (alcanzable)
- $\alpha \xRightarrow{*} u, \beta \xRightarrow{*} v, \alpha' \xRightarrow{*} u', \beta' \xRightarrow{*} v', X \xRightarrow{*} w$ (generadoras)

Y como $\mathcal{G}$ está en CNF, $u \neq \varepsilon$ o $v \neq \varepsilon$. Luego, podemos realizar la construcción:

$$S \xRightarrow{*} \alpha' X \beta' \xRightarrow{*} \alpha' \alpha^i X \beta^i \beta' \xRightarrow{*} u' u^i w v^i v'$$

Como $u \neq \varepsilon$ o $v \neq \varepsilon$, entonces el subconjunto generado con esta construcción es infinito.

Ahora demostraremos la otra dirección, es decir $\mathcal{L}(\mathcal{G})$ es infinito $\Rightarrow$ existe $X \in V$ y $\alpha, \beta \in (V \cup \Sigma)^*$ tal que $X \xRightarrow{+} \alpha X \beta$. Como $\mathcal{L}(\mathcal{G})$ es un lenguaje infinito, entonces existe $w \in \mathcal{L}(\mathcal{G})$ tal que $|w| \geq 2^{|V|}$. Como la gramática está en forma normal de Chomsky, entonces al hacer el árbol de derivación de esta palabra necesariamente la altura del árbol será mayor a $|V|$. Por lo tanto existe una rama del árbol que tiene al menos $|V| + 1$ variables y por principio del palomar, hay una variable repetida en el árbol de derivación. Luego, existe X tal que

$$X \xRightarrow{+} \alpha X \beta$$

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** En el caso converso, por argumentar que $u \neq \varepsilon$ o $v \neq \varepsilon$.
- **(0.5 puntos)** En el caso converso, por argumentar que X es alcanzable
- **(0.5 puntos)** En el caso converso, por establecer que $X, \alpha, \beta, \alpha', \beta'$ son generadoras
- **(1 punto)** En el caso converso, por encontrar derivaciones para generar palabras infinitas
- **(0.5 puntos)** En el caso converso, por concluir que el lenguaje $\mathcal{L}(\mathcal{G})$ es infinito
- **(1 punto)** En el caso directo, por argumentar que como el lenguaje es infinito entonces existe una palabra suficientemente grande
- **(1 punto)** En el caso directo, por usar el árbol de derivación de la palabra para acotar la cantidad de derivaciones
- **(1 punto)** En el caso directo, por usar principio del palomar para concluir que existe repetición de las palabras

Pregunta 3.2

Una posible solución para esta pregunta es entregar un contraejemplo. Sea la siguiente gramática:

$$\begin{aligned} S &\rightarrow a \\ X &\rightarrow XX \end{aligned}$$

Está claro que el lenguaje generado por esta gramática es $\{a\}$ (lenguaje finito), pero sin embargo existe una variable X recursiva. Como existe al menos un caso en el que se incumple la propiedad del enunciado, entonces la propiedad no es necesariamente cierta si $\mathcal{G}$ tiene variables inútiles.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(4 puntos)** Por encontrar un contraejemplo
- **(2 puntos)** Por concluir que no se cumple la propiedad

Pregunta 4

Pregunta 4.1

Una posible solución para esta pregunta es modificar el algoritmo *eval-NFAonthefly* visto en clases.

#OutputNoVacio-AnnA($\mathcal{N} = (Q, \Sigma, \Lambda, \Delta, I, F)$, $d = a_0a_1 \dots a_{n-1}$)

```
S ← I
for i = 0 to n - 1 do
  Sold ← S
  S ← ∅
  for p ∈ Sold do
    S ← S ∪ {q | (p, ai, q) ∈ Δ}
    S ← S ∪ {q | ∃λ ∈ Λ. (p, ai, λ, q) ∈ Δ}
  end for
end for
return Check(S ∩ F ≠ ∅)
```

El algoritmo realiza lo pedido, ya que verifica si hay una ejecución de aceptación, con o sin anotación. Al igual que el algoritmo *eval-NFAonthefly*, se inicializa tomando los estados iniciales, luego se itera sobre la palabra y se guardan los estados a los que se llega desde el estado actual al leer esa letra. Además, este algoritmo revisa los estados a los que se llega desde el estado actual, leyendo la letra actual y si tiene una anotación. Finalmente, retorna si se llegó a un estado final o no, que es lo pedido.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(2 puntos)** Por construir el algoritmo basado en *eval-NFAonthefly*.
- **(3 puntos)** Por ampliar las transiciones a las que tienen marca.
- **(1 punto)** Por explicación de correctitud.

Pregunta 4.2

Una posible solución para esta pregunta es modificar el algoritmo *eval-NFAonthefly* visto en clases de la siguiente forma:

#OutputNoTrivial-AnnA($\mathcal{N} = (Q, \Sigma, \Lambda, \Delta, I, F)$, $d = a_0 a_1 \dots a_{n-1}$)

```

 $S \leftarrow I \times \{0\}$ 
for  $i = 0$  to  $n - 1$  do
   $S_{old} \leftarrow S$ 
   $S \leftarrow \emptyset$ 
  for  $(p, 0) \in S_{old}$  do
     $S \leftarrow S \cup \{(q, 0) \mid (p, a_i, q) \in \Delta\}$ 
     $S \leftarrow S \cup \{(q, 1) \mid \exists \lambda \in \Lambda. (p, a_i, \lambda, q) \in \Delta\}$ 
  end for
  for  $(p, 1) \in S_{old}$  do
     $S \leftarrow S \cup \{(q, 1) \mid (p, a_i, q) \in \Delta\}$ 
     $S \leftarrow S \cup \{(q, 1) \mid \exists \lambda \in \Lambda. (p, a_i, \lambda, q) \in \Delta\}$ 
  end for
end for
return Check( $S \cap (F \times \{1\}) \neq \emptyset$ )

```

El algoritmo realiza lo pedido, ya que verifica si hay una ejecución de aceptación con alguna anotación. Para esto, el algoritmo mantiene pares (p, b) donde p es un estado y $b \in \{0, 1\}$ donde 0 significa que hay una ejecución que ha llegado hasta p pero que no tiene una anotación, y 1 significa que la ejecución tiene al menos una anotación (y por lo tanto, el resultado si llega a un estado final es distinto de ϵ). Al igual que el algoritmo *eval-NFAonthefly*, se inicializa tomando los estados iniciales y lo marca con 0, que significa que no ha habido una anotación hasta ese momento. Luego se itera sobre la palabra y se consideran dos casos. Si hay un par $(p, 0)$ (el primer for) entonces se agregan los pares $(q, 0)$, si hay una transición sin anotación de p a q , y los pares $(q, 1)$ si hay una transición con anotación. En cambio, si hay un par $(p, 1)$ (el segundo for) entonces se agregan todos los pares $(q, 1)$ si hay una transición de p a q , sin importar si es de anotación o no. Finalmente, el algoritmo verifica si hay un par $(p, 1)$ con $p \in F$ en los estados S . Si es así, retorna TRUE ya que significa que hay una ejecución que entrega una anotación distinta de ϵ que es lo pedido.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (1 punto) Por inicializar $S = I \times \{0\}$
- (1 punto) Por no marcar los estados a los que ha llegado sin marcas previas.
- (1 punto) Por marcar los estados a los que ha llegado sin marcas previas, pero tienen marca en esa transición.
- (1 punto) Por marcar los estados a los que ha llegado con marcas previas y mantenerles las marcas.
- (1 punto) Por devolver **Check**($S \cap (F \times \{1\}) \neq \emptyset$).
- (1 punto) Por explicación de correctitud.



Examen

Pregunta 1

Sea $\mathcal{P} = (Q, \Sigma, \Gamma, \Delta, q_0, \perp, \{q_f\})$ un autómata apilador con un único estado final y tal que todas sus transiciones en Δ son de la forma $(p, a, A, q, B_1 B_2)$ o de la forma (p, a, A, q, ϵ) con $p, q \in Q, a \in \Sigma \cup \{\epsilon\}$ y $A, B_1, B_2 \in \Gamma$. Demuestre que existe un autómata apilador $\mathcal{P}'$ con UN solo estado tal que $\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{P}')$.

Solución

La solución corresponde a la demostración vista en la clase 27. Definiremos un nuevo PDA de un solo estado q^* :

$$\mathcal{P}' = (Q', \Sigma, \Gamma', \Delta', q^*, \perp', F')$$

de tal forma que:

- $Q' = F' = \{q^*\}$ es el único estado, inicial y final.
- Σ es el alfabeto de lectura (el mismo que $\mathcal{P}$).
- $\Gamma' = Q \times \Gamma \times Q$ será el alfabeto de stack.
“ $(p, A, q) \in \Gamma'$ si desde p leyendo A en el tope de stack llegamos a q al hacer pop de A .”
- $\Delta' \subseteq (Q' \times (\Sigma \cup \{\epsilon\}) \times \Gamma') \times (Q' \times \Gamma'^*)$ es la relación de transición.
- $\perp' = (q_0, \perp, q_f)$ es el símbolo inicial del stack.
“El autómata parte en q_0 y al hacer pop de $\perp$ llegará a q_f .”

Básicamente, estamos guardando la información de los estados de $\mathcal{P}$ como parte del stack de $\mathcal{P}'$, en tuplas de forma (p, A, q) , donde $p \in Q$ representaría nuestro estado actual, $A \in \Gamma^*$ el stack de $\mathcal{P}$ y $q \in Q$ el estado que “adivinamos” está por debajo en el stack de $\mathcal{P}'$. Definiremos Δ' de la siguiente manera:

$$\Delta' = \overbrace{\{(q^*, a, (p, A, q_2), q^*, (q, B_1, q_1)(q_1, B_2, q_2)) \mid (p, a, A, q, B_1 B_2) \in \Delta \wedge q_1, q_2 \in Q\}}^{\text{Transiciones push}} \cup \underbrace{\{(q^*, a, (p, A, q), q^*, \epsilon) \mid (p, a, A, q, \epsilon) \in \Delta\}}_{\text{Transiciones pop}}$$

Ahora demostramos que tanto $\mathcal{P}$ como $\mathcal{P}'$ definen el mismo lenguaje. Para esto, demostraremos la siguiente afirmación para todo $p \in Q, A \in \Gamma, w \in \Sigma^*$ y $k \geq 1$:

$$(pA, w) \vdash_{\mathcal{P}}^k (q, \epsilon) \quad \text{si y solo si} \quad (q^*(p, A, q), w) \vdash_{\mathcal{P}'}^k (q^*, \epsilon)$$

Si esta afirmación es verdadera tendremos como caso particular que:

$$(q_0 \perp, w) \vdash_{\mathcal{P}}^* (q_f, \epsilon) \quad \text{si y solo si} \quad (q^*(q_0, \perp, q_f), w) \vdash_{\mathcal{P}'}^* (q^*, \epsilon)$$

lo que significa que: $\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{P}')$. Demostraremos lo anterior por inducción en el largo de sus ejecuciones.

Caso base Para una ejecución de un solo paso, sabemos que la única transición existente será pop. Digamos que $c \in \Sigma$ una letra cualquiera:

$$\begin{aligned} & (pA, c) \vdash_{\mathcal{P}} (q, \epsilon) \\ \Leftrightarrow & (p, c, A, q, \epsilon) \in \Delta \\ \Leftrightarrow & (q^*, c, (p, A, q), q^*, \epsilon) \in \Delta' \quad \text{por construcción} \\ \Leftrightarrow & (q^*(p, A, q), c) \vdash_{\mathcal{P}'} (q^*, \epsilon) \end{aligned}$$

Hipótesis inductiva Supongamos se cumple la siguiente propiedad para $k < n$

$$(pA, w) \vdash_{\mathcal{P}}^k (q, \epsilon) \quad \text{si y solo si} \quad (q^*(p, A, q), w) \vdash_{\mathcal{P}'}^k (q^*, \epsilon)$$

Paso inductivo Digamos que tenemos una palabra $w \in \Sigma^*$ tal que $|w| = n$ y $w = c \cdot u \cdot v$. Luego analicemos la ejecución por partes:

$$\begin{aligned} & (pA, c) \vdash_{\mathcal{P}}^n (q, \epsilon) \\ \Leftrightarrow & \underbrace{(pA, c \cdot u \cdot v) \vdash_{\mathcal{P}} (p_1 B_1 B_2, u \cdot v)} \vdash_{\mathcal{P}}^i (p_2 B_2, v) \vdash_{\mathcal{P}}^j (q, \epsilon) \\ \Leftrightarrow & (p, c, A, p_1, B_1 B_2) \in \Delta \\ \Leftrightarrow & (q^*, c, (p, A, q_2), q^*, (p_1, B_1, q_1)(q_1, B_2, q_2)) \in \Delta' \quad \text{por construcción} \\ \Leftrightarrow & (q^*(p, A, p_1), c) \vdash_{\mathcal{P}'} (q^*, \epsilon) \end{aligned}$$

Por otra parte, tomemos una sección de tamaño $i < n$ de la ejecución,

$$\begin{aligned} & (pA, c) \vdash_{\mathcal{P}}^n (q, \epsilon) \\ \Leftrightarrow & (pA, c \cdot u \cdot v) \vdash_{\mathcal{P}} \underbrace{(p_1 B_1 B_2, u \cdot v)} \vdash_{\mathcal{P}}^i (p_2 B_2, v) \vdash_{\mathcal{P}}^j (q, \epsilon) \\ \Leftrightarrow & (p_1 B_1, u) \vdash_{\mathcal{P}}^i (p_2, \epsilon) \\ \Leftrightarrow & (q^*(p_1, B_1, p_2), u) \vdash_{\mathcal{P}'}^i (p_2, \epsilon) \quad \text{ya que } i < n, \text{ por H.I.} \end{aligned}$$

Similarmente, tomemos una sección final de tamaño $j < n$,

$$\begin{aligned} & (pA, c) \vdash_{\mathcal{P}}^n (q, \epsilon) \\ \Leftrightarrow & (pA, c \cdot u \cdot v) \vdash_{\mathcal{P}} (p_1 B_1 B_2, u \cdot v) \vdash_{\mathcal{P}}^i \underbrace{(p_2 B_2, v)} \vdash_{\mathcal{P}}^j (q, \epsilon) \\ \Leftrightarrow & (p_2 B_2, v) \vdash_{\mathcal{P}}^j (q, \epsilon) \\ \Leftrightarrow & (q^*(p_2, B_2, q), v) \vdash_{\mathcal{P}'}^j (q, \epsilon) \quad \text{ya que } j < n, \text{ por H.I.} \end{aligned}$$

Finalmente, al juntar estas tres conclusiones podemos ver que

$$(q^*(p, A, q), c \cdot u \cdot v) \vdash_{\mathcal{P}'}^n (q^*, \epsilon)$$

Finalmente, hemos demostrado por inducción que $\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{P}')$, ya que las ejecuciones de ambos PDA son equivalentes.

Distribución de puntaje

- (1 pto.) Por plantear el concepto de construcción y funcionamiento del nuevo PDA $\mathcal{P}'$.
- (1 pto.) Por definir correctamente el alfabeto de stack Γ' .
- (1 pto.) Por definir correctamente el símbolo inicial del stack $\perp'$.
- (1 pto.) Por definir en Δ' las transiciones *push* correctamente.
- (1 pto.) Por definir en Δ' las transiciones *pop* correctamente.
- (1 pto.) Por demostrar que ambos PDA definen el mismo lenguaje, $\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{P}')$

Pregunta 2

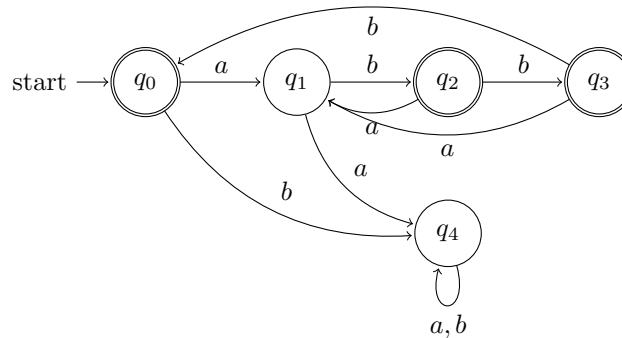
Considere la siguiente expresión regular sobre el alfabeto $\{a, b\}$:

$$R = (ab + abb + abbb)^*$$

1. Construya un autómata finito determinista $\mathcal{A}$ tal que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(R)$. Explique su respuesta.
2. Demuestre que NO existe un autómata finito determinista $\mathcal{A}'$ con 4 estados tal que $\mathcal{L}(\mathcal{A}') = \mathcal{L}(R)$.

Solución

Problema 2.1. La expresión R representa cadenas en repeticiones de ab , abb y $abbb$. Un AFD (autómata finito determinista) que acepte esta expresión regular, debe reconocer cada uno de estos patrones. El siguiente autómata cumple con lo anterior:



Explicación: El autómata tiene cinco estados, q_0, q_1, q_2, q_3 y q_4 . El estado inicial es q_0 y los estados finales son q_0, q_2 y q_3 . El estado q_4 es un estado de error, que se alcanza si se lee un símbolo distinto de b o a en los estados q_0 o q_1 respectivamente. En el estado q_0 se espera leer un a para pasar al estado q_1 y luego un b para pasar al estado q_2 . En el estado q_2 se espera leer un b para pasar al estado q_3 y luego un a para volver al estado q_1 . En el estado q_3 se espera leer un b para volver al estado q_0 y un a para volver al estado q_1 . En el estado q_4 se espera leer un a o un b para quedarse en el mismo estado.

Distribución de puntaje

- (1 pto.) Por aceptar ϵ en el estado inicial que además es un estado de reinicio y aceptación. Es decir, reconocer la secuencia vacía y $abbb$.
- (1 pto.) Por aceptar a en el estado inicial y pasar al estado q_1 .
- (1 pto.) Por aceptar b en el estado q_1 y pasar al estado q_2 . Es decir, reconocer la secuencia ab .
- (1 pto.) Por aceptar b en el estado q_2 y pasar al estado q_3 . Es decir, reconocer la secuencia abb .
- (1 pto.) Por estado de rechazo en el estado q_4 . Es decir, rechazar cualquier secuencia que no sea ab, abb o $abbb$.
- (1 pto.) Por la explicación.

Problema 2.2. El lenguaje $\mathcal{L}(R)$ tiene estructuras repetitivas que dependen de la longitud de las cadenas ab , abb , $abbb$. Cada uno de estos patrones requiere distinguir entre diferentes posiciones dentro de las cadenas. Con sólo 4 estados no es posible rastrear suficientes configuraciones, pues es necesario considerar las palabras que no son aceptadas como se vio anteriormente.

PD: Todas las clases de equivalencia de la relación de Myhill Nerode $\equiv_{\mathcal{L}(R)}$ son distintas, por lo que no es posible reducir el autómata.

Para esta demostración, se puede usar el concepto de clases de equivalencia de Myhill-Nerode. Este método se basa en demostrar que el lenguaje tiene más de 4 clases de equivalencia, lo que implica que no es posible reducir el autómata a menos de 5 estados.

Sabemos que dos cadenas x y y están en la misma clase de equivalencia respecto a $\mathcal{L}(R)$ si, para cualquier cadena z , la concatenación de $xz \in \mathcal{L}(R)$ implica que $yz \in \mathcal{L}(R)$ y viceversa. En otras palabras $x \equiv_{\mathcal{L}(R)} y \iff (\forall z \in \Sigma^*. xz \in \mathcal{L}(R) \iff yz \in \mathcal{L}(R))$.

Para el lenguaje, las clases de equivalencia dependen de qué prefijos de la cadena x se alinean con los patrones ab , abb o $abbb$. Necesitamos distinguir cuántos estados posibles puede tener la máquina.

Los patrones posibles en $\mathcal{L}(R)$ generan 5 clases de equivalencia diferentes:

- $[\epsilon]$: Cadenas vacías o listas para empezar un patrón.
- $[a]$: Cadenas que han iniciado un patrón pero necesitan un b .
- $[ab]$: Cadenas que están en medio de un patrón válido, esperando completarse o reiniciarse.
- $[abb]$: Cadenas que casi completan un patrón válido, faltando solo un b .
- $[aa]$: Cadenas inválidas para el lenguaje porque no pueden derivar en un patrón aceptable.

Para mostrar que estas clases de equivalencia son distintas, se puede usar el siguiente argumento:

	ϵ	a	ab	abb
ϵ				
a	b			
ab	b	bbb		
abb	b	bb	bb	
aa	ab	b	b	b

En la tabla, se muestra que las clases de equivalencia son distintas, pues para cada par de cadenas, se puede encontrar una cadena z que distinga entre ellas. Por ejemplo, para las cadenas a y ab , se puede usar $z = bbb$ para distinguirlas. Para las cadenas ab y abb , se puede usar $z = bb$ para distinguirlas. Para las cadenas abb y aa , se puede usar $z = b$ para distinguirlas. Por lo tanto, como tengo por lo menos 5 clases de equivalencia de Myhill-Nerode, entonces cualquier DFA que acepte $\mathcal{L}(R)$ debe tener al menos 5 estados.

Distribución de puntaje

- **(1 pto.)** Por identificar las clases de equivalencia.
- **(1.5 ptos.)** Por comparar ab^i versus ab^j .
- **(1.5 ptos.)** Por comparar ab^i versus ϵ .
- **(1.5 ptos.)** Por comparar aa con cualquier otra cadena.
- **(0.5 ptos.)** Por la conclusión de usar la relación de Myhill-Nerode lleva a que el autómata tiene que tener más de 4 estados.

Problema 3

Una gramática libre de contexto $\mathcal{G}$ se dice *lineal* si todas sus producciones son de la forma $X \rightarrow aY$, $X \rightarrow Ya$ o $X \rightarrow a$ con $X, Y \in V$ y $a \in \Sigma$. En cambio, $\mathcal{G}$ se dice que es lineal *por la izquierda* si es lineal y NO tiene reglas de la forma $X \rightarrow aY$.

1. Demuestre que si $\mathcal{G}$ es una gramática lineal por la izquierda, entonces $\mathcal{L}(\mathcal{G})$ es regular.
2. Demuestre que existe una gramática lineal $\mathcal{G}$ tal que $\mathcal{L}(\mathcal{G})$ NO es regular.

Solución

Problema 3.1. Sea la gramática $G = (V, \Sigma, P, S)$ lineal por la izquierda. Es decir, sus producciones son de la forma $X \rightarrow Ya$ o $X \rightarrow a$. Al ser lineal por la izquierda, su árbol derivación será de la forma:

$$S \Rightarrow X_{n-1}a_n \Rightarrow X_{n-2}a_{n-1}a_n \Rightarrow \dots \Rightarrow X_1a_2\dots a_n \Rightarrow a_1a_2\dots a_n$$

La idea es construir un autómata que adivinará (no determinismo) la derivación anterior desde la derecha hacia la izquierda. O sea, empezando con $X_1 \rightarrow a_1$ de ahí $X_2 \rightarrow X_1a_2$, etc, hasta llegar a $S \rightarrow X_{n-1}a_n$. Para esto, los estados del autómata serán las variables V con un estado adicional q_0 para partir.

Dado lo anterior, construimos el autómata A como sigue. Sea $A = (V \cup \{q_0\}, \Sigma, \Delta, \{q_0\}, \{S\})$, con $\Delta = \{(q_0, a, X) \mid X \rightarrow a \in P\} \cup \{(Y, a, X) \mid X \rightarrow Ya \in P\}$. Por demostrar: $\mathcal{L}(G) = \mathcal{L}(A)$.

$$\mathcal{L}(G) \subseteq \mathcal{L}(A)$$

Sea $w = a_1a_2\dots a_n \in \mathcal{L}(G)$. Con G lineal por la izquierda. La derivación de G sobre w es de la forma:

$$S \Rightarrow X_{n-1}a_n \Rightarrow X_{n-2}a_{n-1}a_n \Rightarrow \dots \Rightarrow X_1a_2\dots a_n \Rightarrow a_1a_2\dots a_n$$

De esto, es claro que se puede construir una ejecución ρ de la forma:

$$\rho: q_0 \xrightarrow{a_1} X_1 \xrightarrow{a_2} X_2 \longrightarrow \dots \longrightarrow X_{n-1} \xrightarrow{a_n} S$$

$$\mathcal{L}(A) \subseteq \mathcal{L}(G)$$

La demostración es exactamente igual para este lado que el anterior, se construye la misma ejecución por construcción de Δ y se concluye la misma derivación de G .

Distribución de puntaje

- (1 pto.) Por la idea de cómo es el árbol de derivación de la gramática y como esto puede llevar a construir un autómata.
- (1 pto.) Por dar el conjunto final del autómata de manera correcta.
- (1.5 ptos.) Por el primer subconjunto de Δ .
- (1.5 ptos.) Por el segundo subconjunto de Δ .
- (1 pto.) Por demostrar, para ambos lados, que los lenguajes son iguales.

Problema 3.2. Sea el lenguaje $L = \{a^n b^n \mid n \geq 1\}$, que, como se vió en clases, es no regular. Sea la gramática G que lo produce tal que:

$$\begin{array}{lcl} S & \rightarrow & aX \mid aY \\ X & \rightarrow & Sb \\ Y & \rightarrow & b \end{array}$$

Como se puede notar, G es lineal y produce el lenguaje L . Por lo que queda demostrado.

Distribución de puntaje

- (2 ptos.) Por dar el lenguaje.
- (2 ptos.) Por demostrar que es no regular. Si se usa uno visto en clases, es válido no demostrarlo.
- (2 ptos.) Por dar la gramática que construya al lenguaje.

Problema 4

Considere el siguiente lenguaje sobre el alfabeto $\Sigma = \{a, b\}$:

$$L = \{a^n b^m a^n b^m \mid n \geq 0 \wedge m \geq 0\}.$$

Sea $L^c = \Sigma^* \setminus L$ el complemento de L .

1. ¿Es L libre de contexto? Demuestre su afirmación.
2. ¿Es L^c libre de contexto? Demuestre su afirmación.

Solución

Problema 4.1. El lenguaje **no es libre de contexto**. Para demostrar esto, usamos el lema del bombeo de lenguajes libres de contexto. Sea $N > 0$. Definimos:

$$z = a^N b^N a^N b^N.$$

Considere una partición cualquiera $uvwxy = z$ tal que: $vx \neq \varepsilon$ y $|vwx| \leq N$. Como $|vwx| \leq N$, se tiene que (1) $vwx \in \mathcal{L}(a^* b^*)$ o (2) $vwx \in \mathcal{L}(b^* a^*)$. Asimismo, tomamos $i = 2$. A continuación analizaremos uno de los casos, ya que el otro es análogo:

Sin pérdida de generalidad, suponemos que $v \in \mathcal{L}(a^*)$ y $x \in \mathcal{L}(b^*)$. Los otros casos son análogos. Entonces, se tiene:

$$\underbrace{a^j}_u \underbrace{a^k}_v \underbrace{a^l b^m}_w \underbrace{b^n}_x \underbrace{b^o a^N b^N}_y$$

Para algún $j, k, l, m, n, o \in \mathbb{N}$ tal que $j + k + l = N \wedge m + n + o = N \wedge (k \neq 0 \vee n \neq 0)$ Recordemos que escogimos $i = 2$, con lo que tenemos:

$$u v^2 w x^2 y = a^{j+2k+l} b^{m+2n+o} a^N b^N = a^{N+k} b^{N+n} a^N b^N$$

Como $k \neq 0$ o $n \neq 0$, entonces $N + k \neq N \vee N + n \neq N$. Por lo tanto, $uv^2wx^2y \notin L$.

Distribución de puntaje

- (1 pto.) Por decir que el lenguaje NO es libre de contexto.
- (1 pto.) Por definir z de manera correcta.
- (1 pto.) Por descomponer z correctamente y usar $i = 2$ o similar.
- (1 pto.) Por mencionar y/o analizar los dos casos, dado que $|vwx| \leq N$.
- (1 pto.) Por armar $uv^2wx^2y \notin L$ dado que se escogió $i = 2$ o similar.
- (1 pto.) Por concluir que $uv^2wx^2y \notin L$ dado que $k \neq 0$ o $n \neq 0$.

Problema 4.2. El lenguaje **es libre de contexto**. Para esto, dado que el lenguaje se forma utilizando la operación complemento, notemos que tenemos dos casos:

I) $w \neq a^* b^* a^* b^*$, es decir, no es de la forma solicitada para $\mathcal{L}$. Esto corresponde a casos como $a...ab...ba...ab...ba...$ o $b...ba...ab...ba...$, es decir, $a^* b^+ a^+ b^+ a \Sigma$

II) $w = a^k b^l a^m b^n$ tal que $k \neq m \vee l \neq n$

Dado esto, debemos formar una gramática libre de contexto que contenga ambos casos. Un ejemplo es la siguiente gramática S , donde S_I corresponde al primer caso y S_{II} corresponde al segundo:

$$\begin{aligned} S &\rightarrow S_I \mid S_{II} \\ S_I &\rightarrow a S_I \mid b B_1 \\ B_1 &\rightarrow b B_1 \mid a A_2 \\ A_2 &\rightarrow a A_2 \mid b B_2 \\ B_2 &\rightarrow b B_2 \mid a E \end{aligned}$$

$$\begin{aligned}
E &\rightarrow a E \mid b E \mid \epsilon \\
S_{II} &\rightarrow X B \mid A Y \\
A &\rightarrow a A \mid \epsilon \\
B &\rightarrow b B \mid \epsilon \\
X &\rightarrow a X a \mid a A b B \mid b B a A \mid a \\
Y &\rightarrow b Y b \mid a A b B \mid b B a A \mid b
\end{aligned}$$

De la definición de S uno puede ver que las derivaciones de S_I serán de la forma:

$$w = a^* b^+ a^+ b^+ a \Sigma$$

y las derivaciones de S_{II} serán de la forma:

$$w = a^k b^l a^m b^n \text{ tal que } k \neq m \vee l \neq n$$

Por lo tanto, $\mathcal{L}(S) = \mathcal{L}$.

Distribución de puntaje

- **(1 pto.)** Por decir que el lenguaje es libre de contexto.
- **(0.5 ptos.)** Por el caso I.
- **(2 ptos.)** Por el caso II.
- **(0.5 ptos.)** Por las reglas de S_I .
- **(1 pto.)** Por las reglas $S_{II} \rightarrow X B \mid A Y$, es decir, por hacer un camino que permita llegar a $w = a^k b^l a^m b^n$ tal que $k \neq m$ y otro que permita llegar a $w = a^k b^l a^m b^n$ tal que $\forall l \neq n$.
- **(1 pto.)** Por el resto de las reglas de S_{II} .